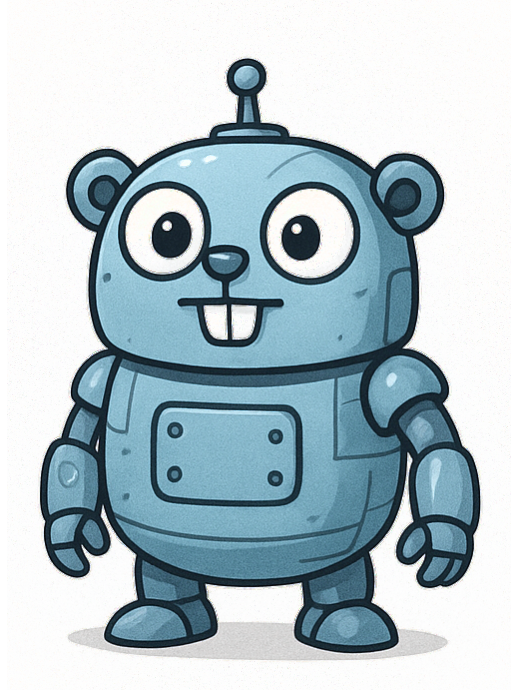


Gorai

Building Modern Robots with Go and NATS



Greg Herlein & Luca Herlein

Version 0.2.0

2025

Introduction

About This Book

Gorai: Building Modern Robots with Go and NATS is a practical guide to a new approach in robotics software. Whether you're a software developer curious about robotics, a robotics enthusiast tired of fighting complex frameworks, or an experienced engineer looking for something better, this book will get you building.

We wrote this book because we believe robotics software should be simpler. The tools exist—Go's elegant concurrency, NATS's battle-tested messaging, modern AI accelerators—but nobody had put them together in a way that prioritized developer experience. *Gorai* is our answer, and this book is your guide to using it.

Who We Are

Greg Herlein

My path to robotics took a circuitous route through some of the most demanding technical environments you can imagine.

I started my career as a US Navy Submarine Nuclear Power Plant Operator and then Supervisor. When you're responsible for a nuclear reactor hundreds of feet underwater, you learn quickly that systems must be simple enough to understand completely, robust enough to never fail, and designed so that the right action is the obvious action. Those lessons never left me.

After the Navy, I spent decades in Silicon Valley leading engineering teams at companies you've heard of—Rackspace, Cisco, AWS—and plenty of startups you haven't. I built distributed systems before "distributed systems" was a buzzword. I learned what works at scale and what doesn't.

But robotics was always my passion on the side. I coached middle school and high school robotics teams in FIRST LEGO League and VEX competitions, watching students struggle with the same software complexity that frustrated professional engineers. At home, I built robots for fun—and ran into those same frustrations myself.

Gorai grew from a simple question: why is robotics software so much harder than it needs to be? The distributed systems lessons from my career, the simplicity requirements from nuclear power, and the accessibility needs from coaching young roboticists—they all pointed to the same answer. We needed something new.

Luca Herlein

I grew up building robots. Not as a hobby – I picked that up later — as the thing I did from elementary school through college.

My FIRST LEGO League team made it to the World Championships. I spent years in VEX competitions, learning what it takes to build machines that actually work under pressure. Eight years of competition robotics teaches you things that textbooks can't: that the simple solution usually beats the complex one, that testing matters more than theory, and that the robot that runs reliably beats the robot that runs impressively (sometimes).

I studied Aerospace Engineering at CU Boulder, where I learned the formal foundations—dynamics, control systems, embedded programming. I served as Aerodynamics Lead Engineer on the university's 2021-22 Design Build Fly (DBF) competition team, applying those foundations to aircraft that had to actually fly. But honestly, the competition experience—from FLL through DBF—taught me more about building things that work than any textbook. Academic exercises have known solutions. Competition robots and aircraft face unknown challenges with hard deadlines.

Why We Wrote This Together

A robotics framework needs two perspectives: the software architect who thinks in distributed systems and long-term maintainability, and the roboticist who thinks in actuators and sensors and "will this work when it matters."

Greg brings decades of building systems that scale and survive. Luca brings years of building robots that compete and win. *Gorai* exists at the intersection—software engineering rigor applied to practical robotics.

This book reflects both perspectives. The architectural discussions come from hard-won experience with distributed systems. The practical examples come from actually building robots. When we disagree (and we do), we usually find that both viewpoints have merit—and the synthesis is better than either alone.

What You'll Learn

By the end of this book, you'll understand:

- **The Gorai mental model:** How to think about robot software as distributed systems
- **NATS messaging:** Pub/sub, request/reply, and streaming for robotics
- **Component architecture:** Sensors, actuators, cameras—building blocks that compose
- **Service design:** Vision, navigation, and custom capabilities
- **Testing strategies:** From unit tests to hardware validation
- **AI integration:** Running ML models on edge hardware
- **Project organization:** Structuring code that grows with your robot

More importantly, you'll have built things. The hello-sensor example runs real code. The custom component chapter produces working drivers. The testing chapter creates tests that actually catch bugs.

How to Read This Book

If you're new to robotics: Read sequentially. Each chapter builds on the previous. By Chapter 9, you'll understand a complete working system.

If you're experienced with ROS/ROS 2: Skim Chapters 1–2 for the philosophical differences, then dive into Chapter 3 (NATS) and Chapter 9 (hello-sensor). The patterns will feel familiar; the simplicity will feel liberating.

If you're a Go developer exploring robotics: Chapter 2 (architecture) and Chapter 4–6 (components) will orient you. The code will feel natural; the domain concepts will be new.

If you just want to build something: Start with Chapter 8 (environment setup) and Chapter 9 (hello-sensor). Get code running, then circle back to understand why it works.

A Note on Style

We write the way we talk. Technical concepts deserve clear explanations, not academic obfuscation. Code examples are complete and runnable, not excerpts that require imagination to compile.

When we don't know something, we say so. When multiple approaches work, we explain the tradeoffs rather than pretending one is obviously correct. Robotics is hard enough without authors pretending otherwise.

Let's Build

The best way to learn robotics is to build robots. The best way to learn Gorai is to use it.

Fire up your terminal. Clone the repository. Let's get started.

Greg Herlein & Luca Herlein 2024

Chapter 1: Why Gorai?

1.1 The Robotics Software Landscape

The world of robotics software has evolved dramatically over the past two decades. What began as custom, hand-rolled solutions for individual robots has grown into a rich ecosystem of frameworks, libraries, and platforms. Yet despite this maturation, building robot software remains more difficult than it should be.

A Brief History

ROS (Robot Operating System) emerged from Stanford and Willow Garage in 2007, becoming the de facto standard for research robotics. Its publish-subscribe architecture, standardized message types, and vast package ecosystem revolutionized how robots were built. However, ROS was designed for a different era—single robots, research labs, and developers comfortable with C++ build systems.

ROS 2 arrived in 2017 to address ROS's limitations. Built on DDS (Data Distribution Service), it brought real-time capabilities, better security, and multi-robot support. But ROS 2 also brought complexity: multiple DDS implementations to choose from, a steep learning curve, and build times that can stretch into hours.

YARP (Yet Another Robot Platform) took a different approach, focusing on middleware for humanoid robots. It excels at connecting heterogeneous systems but requires significant investment to master its idioms and patterns.

Viam represents the modern cloud-connected approach: a managed platform where robots connect to cloud services for configuration, monitoring, and ML inference. It's elegant but introduces cloud dependencies that not every robot application can accept.

Common Pain Points

After years of working with these platforms, recurring frustrations emerge:

C++ Complexity: ROS and ROS 2 are fundamentally C++ frameworks. While Python bindings exist, performance-critical code requires C++. This means grappling with CMake, colcon, header dependencies, and compilation times measured in tens of minutes. Memory safety issues lurk in every pointer.

Python Performance Limitations: Many teams escape to Python for faster development, only to hit walls when their control loops can't keep up or their image processing saturates a single core. The "prototype in Python, rewrite in C++" cycle wastes enormous effort.

Heavy Framework Overhead: Modern ROS 2 installations consume gigabytes. Starting a simple node pulls in layers of middleware. The abstraction cost—both in binary size and mental overhead—grows with each release.

Steep Learning Curves: New developers face months of ramp-up time. Understanding launch files, parameter servers, lifecycle management, QoS profiles, and the interaction between nodes requires dedicated study. Documentation assumes familiarity with concepts that aren't explained.

Build System Complexity: colcon, CMake, ament, package.xml, setup.py—the tooling stack has grown organically and shows it. Cross-compilation for embedded targets requires arcane knowledge. Reproducible builds demand containerization.

The Gap Gorai Fills

These pain points aren't inevitable. They reflect choices made in different contexts—academic research, enterprise middleware, cloud platforms—that don't always align with building practical robots.

What if we started fresh? What if we took the best ideas from distributed systems and cloud computing, combined them with Go's simplicity and performance, and designed specifically for modern robotics development?

That's the question Gorai answers.

1.2 Design Philosophy

Gorai isn't just another robotics framework—it's a deliberate set of choices about how robot software should be built. These principles guide every design decision.

Go-First

Go was designed at Google to solve exactly the problems that plague robotics development: C++ complexity, build system nightmares, and dependency hell. It compiles to native binaries in seconds, not minutes. It has built-in concurrency primitives that match how robots actually work—many things happening at once. It produces single, statically-linked binaries that deploy trivially.

```
// A complete Gorai node in ~20 lines
func main() {
    n, _ := node.New("my_robot", node.WithNATS("nats://localhost:4222"))
    defer n.Close()

    pub := pub.New[*sensor.Temperature](n, "sensors.temp")

    for reading := range readSensor() {
        pub.Publish(context.Background(), reading)
    }
}
```

Go's type system catches errors at compile time without the ceremony of C++. Its garbage collector eliminates memory leaks without runtime overhead that matters for robotics. Its tooling—`go build`, `go test`, `go mod`—just works.

For microcontrollers, TinyGo brings the same language to resource-constrained devices. Write your robot's brain in Go, write your motor controller in Go—same language, same patterns, same mental model.

NATS-Native

While ROS 2 chose DDS—a complex enterprise middleware with multiple competing implementations—Gorai builds on NATS, a messaging system designed for cloud-native applications.

NATS brings:

- **Simplicity:** A single binary, zero configuration to start
- **Performance:** Millions of messages per second on modest hardware
- **Flexibility:** Pub/sub, request/reply, and streaming in one system
- **JetStream:** Persistence when you need it, fire-and-forget when you don't
- **Clustering:** Built-in distribution across nodes and networks

NATS was battle-tested at companies processing billions of messages daily before Gorai adopted it. That operational maturity matters when your robot needs to work reliably.

AI-Optimized

Modern robots increasingly rely on ML inference—object detection, pose estimation, voice recognition, path planning. Gorai treats AI as a first-class capability rather than an afterthought.

The acceleration layer (`accel/`) provides a unified interface across different hardware:

- **NPU:** Rockchip RK3588's 6 TOPS neural processing unit
- **GPU:** NVIDIA CUDA for Jetson platforms
- **TPU:** Google Coral edge TPU
- **CPU:** Optimized fallback that works everywhere

Load a model, run inference, get results—the same code works whether you're on a laptop testing or deployed on edge hardware:

```
acc, _ := rknn.New()
model, _ := acc.Load(ctx, "yolov5s.rknn")
outputs, _ := model.Infer(ctx, inputs)
```

Modular by Default

Gorai components communicate through messages, not method calls. This isn't just architecture astronautics—it has practical consequences:

- **Hot swapping:** Replace a motor driver without restarting the navigation stack
- **Distributed deployment:** Run vision processing on a GPU node, control on a Pi
- **Testing:** Inject fake components without modifying production code
- **Monitoring:** Observe any data flow with standard NATS tools

Every component implements the same `Resource` interface. Every resource can be accessed locally or remotely with the same code. The system composes naturally.

Low Barrier to Entry

Getting started with Gorai should take minutes, not days:

```
# Install Go (if needed)
# Install NATS (single binary)
# Clone and run
git clone https://github.com/gorai/gorai
cd gorai
go run ./examples/hello-sensor
```

No colcon builds. No CMake configuration. No ROS workspace setup. No Docker containers (unless you want them). The examples compile and run immediately.

Fun!

This might seem frivolous, but it matters. Robotics should spark joy. When build systems frustrate and frameworks confuse, that joy disappears.

Gorai aims to bring back the fun: write code, see it run on your robot, iterate quickly, and spend your time solving robotics problems rather than fighting tools.

1.3 Who Should Use Gorai

Gorai isn't trying to be everything to everyone. It's designed for a specific kind of developer and a specific kind of project.

You Should Use Gorai If You're...

Building new, modern robotics projects. Gorai shines on greenfield projects where you're not constrained by existing code. If you're starting a new robot from scratch, Gorai lets you move fast without inheriting technical debt.

Not dependent on ROS ecosystem packages. The ROS ecosystem has thousands of packages—SLAM algorithms, navigation stacks, manipulation libraries. If your project critically depends on specific ROS packages with no alternatives, staying in ROS makes sense. But if you need standard capabilities (sensor interfaces, motor control, basic vision), Gorai provides clean implementations without the baggage.

Open to experimentation. Gorai is young. APIs may evolve. Best practices are still emerging. If you need a framework certified for production medical robots today, look elsewhere. If you're excited to shape a framework's future while building your robot, welcome aboard.

Preferring Go's simplicity to C++ complexity. If you love template metaprogramming and consider CMake a reasonable build system, Gorai might feel constrained. But if you've ever spent an afternoon debugging a segfault or wrestling with linking errors, Go's guardrails are liberating.

Valuing extensibility and performance. Gorai's architecture makes adding new components straightforward. Its Go foundation means you get native performance without unsafe memory access. When you need more speed, the profiler tells you exactly where, and optimization is tractable.

Interested in AI-assisted development. Gorai's codebase is designed to work well with AI coding assistants. Clear interfaces, consistent patterns, and comprehensive specifications mean AI tools can help write components, generate tests, and explain behavior. This isn't just documentation—it's a development philosophy.

Targeting Linux-based robot compute. Gorai runs on Linux: Raspberry Pi, Jetson, Orange Pi, or any ARM or x86 board. It doesn't require ROS's specific Ubuntu LTS versions—any modern Linux works. If your primary compute is Windows or macOS, Gorai isn't the right choice.

Wanting to use TinyGo for microcontrollers. For low-level hardware—motor drivers, sensor interfaces, real-time control—Gorai supports TinyGo on microcontrollers. Same language on your Raspberry Pi brain and your RP2040 motor controller. Same patterns, same skills.

Gorai is Not For You If...

You need certified, production-ready software today. Gorai is under active development. It hasn't been validated for safety-critical applications. Medical robots, autonomous vehicles on public roads, industrial automation with human safety implications—these deserve mature, certified frameworks.

You need specific ROS packages. If your project depends on MoveIt for manipulation, Nav2 for navigation, or specific SLAM implementations only available in ROS, the switching cost is too high. Gorai will eventually have equivalents, but "eventually" doesn't help today.

Your team is deeply invested in ROS/ROS 2. Migration costs are real. If your team knows ROS inside and out, has years of custom packages, and a deployment pipeline that works, the productivity gain from Gorai may not justify retraining.

You need hard real-time guarantees. Go's garbage collector, while excellent, introduces unpredictable pauses. For microsecond-level control loops (some motor commutation, force control), dedicated real-time systems are appropriate. Gorai works alongside these systems—the serial gateway pattern connects TinyGo microcontrollers for real-time tasks—but it doesn't replace them.

1.4 What You'll Build

This book is hands-on. By the end, you'll have built real, working robot components and understand Gorai deeply enough to build your own.

The Hello Sensor Example

Our primary teaching example is `hello-sensor`: a CPU temperature sensor that reads system thermal data and publishes it over NATS. It sounds simple, but it demonstrates everything you need to know:

- Creating a Gorai node and connecting to NATS
- Implementing the `Sensor` interface
- Platform-specific code (Linux thermal zones, macOS system calls)
- Publishing Protocol Buffer messages
- Configuration and command-line flags
- Statistics collection and diagnostics
- Graceful shutdown and resource cleanup
- Fake implementations for testing

By Chapter 9, you'll understand every line of this example and be ready to adapt it for your own sensors.

Along the Way

Each chapter builds practical skills:

Chapter 2-3: You'll run NATS, observe message flow, and understand how Gorai's distributed architecture works in practice.

Chapter 4-7: You'll explore component interfaces—sensors, actuators, cameras, and services—understanding the contracts that make components interchangeable.

Chapter 8: You'll set up a complete development environment, from Go installation to hardware connections.

Chapter 10: You'll implement a custom component from scratch, following patterns established in the framework.

Chapter 11: You'll write tests at every level—unit tests with fakes, component tests with embedded NATS, integration tests across systems.

Chapter 12: You'll run ML inference on accelerated hardware, integrating AI capabilities into robot behaviors.

What You Won't Build

This book focuses on foundations. We won't build:

- A complete autonomous robot (that's a book unto itself)
- Production navigation or SLAM systems
- Detailed manipulation pipelines
- Fleet management and cloud integration

These are important topics, but they build on the foundations this book establishes. Master the fundamentals here, and those advanced topics become tractable.

Cross-reference: The complete hello-sensor implementation is covered in Chapter 9.

1.5 Prerequisites

Gorai is designed to be approachable, but some background knowledge will help you get the most from this book.

Required: Basic Go Knowledge

You should be comfortable with Go fundamentals:

- **Variables and types:** `var` , `:=` , basic types (`int` , `string` , `float64`)
- **Functions:** Declaration, multiple return values, error handling
- **Structs:** Field definition, methods with receivers
- **Interfaces:** How they work, implicit satisfaction
- **Slices and maps:** Creation, access, iteration
- **Goroutines and channels:** Basic concurrent patterns
- **Packages and imports:** Go module structure

If you're new to Go, spend a few hours with the [Go Tour](#) before diving in. The concepts translate quickly, especially if you know Python, JavaScript, or C.

Required: Command-Line Familiarity

You'll spend time in the terminal:

- Navigating directories (`cd` , `ls` , `pwd`)
- Running commands with flags
- Understanding stdout, stderr, and exit codes
- Basic environment variables

Nothing exotic—if you've used a Unix-like terminal, you're prepared.

Helpful: Networking Basics

Understanding helps but isn't required:

- IP addresses and ports
- TCP vs UDP (NATS uses TCP)
- Client-server vs peer-to-peer models
- What "localhost" means

The book explains what you need when you need it.

Helpful: Basic Electronics/Hardware

If you want to connect real hardware:

- What GPIO, I2C, SPI, and UART mean
- How to read a pinout diagram
- Basic electrical safety (don't short 5V to ground)

For the first several chapters, you'll work with simulated and fake components. Hardware comes later, and we'll explain what you need.

Optional: Prior Robotics Experience

Experience with ROS, ROS 2, or other robotics frameworks helps you appreciate Gorai's design choices. But it's not required—we explain concepts from first principles.

If you're coming from ROS, you'll recognize familiar patterns: nodes, topics, publishers, subscribers, services. Gorai's versions are simpler but serve the same purposes.

Development Environment

You'll need:

- A computer running Linux, macOS, or Windows (with WSL2 for Linux compatibility)
- Go 1.21 or later installed
- A text editor or IDE (VS Code with Go extension recommended)
- Git for cloning repositories
- Network access for downloading dependencies

Chapter 8 covers setup in detail. For now, confirm you can run `go version` and see output like `go version go1.22.0 linux/amd64`.

With these foundations in place, you're ready to understand how Gorai thinks about robotics. Chapter 2 introduces the mental model and architecture that makes everything else make sense.

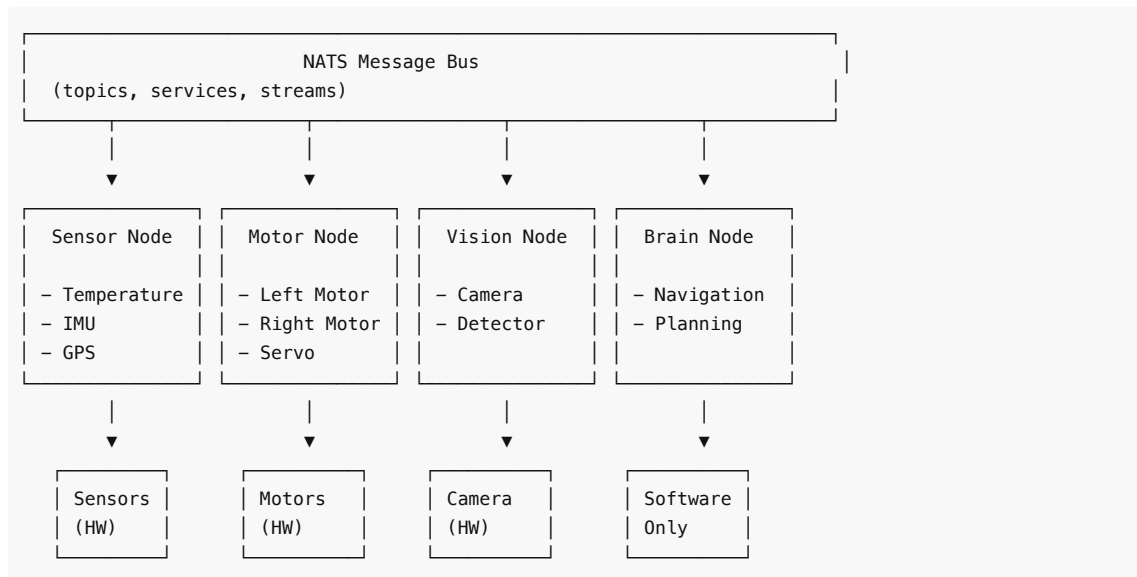
Chapter 2: Mental Model & Architecture

Understanding Gorai's architecture isn't about memorizing components—it's about internalizing a way of thinking about robot software. This chapter establishes the mental model that makes everything else click.

2.1 The Big Picture

A Gorai robot is a collection of independent processes communicating through messages. This sounds abstract, so let's make it concrete.

System Architecture



Each box is a **node**—an independent process that manages one or more **resources** (components or services). Nodes communicate exclusively through the NATS message bus. This separation has profound implications:

- **Failure isolation:** If the vision node crashes, motors keep running
- **Independent scaling:** Run vision on a GPU, control on a Pi
- **Hot updates:** Restart a node without stopping the robot
- **Clean testing:** Replace real nodes with fake ones

The Three-Layer Model

Gorai robots typically span three computational layers:

Layer 1: Primary Compute

The main robot brain—a Linux single-board computer (SBC) running Go:

- Raspberry Pi 5, Orange Pi 5, Jetson Orin Nano
- Runs high-level logic: navigation, planning, behavior
- Connects to NATS server (often running locally)
- Has network access for updates, remote monitoring, fleet coordination

Layer 2: Secondary Nodes

Smaller Linux boards for specialized tasks:

- Dedicated vision processing on a board with GPU/NPU
- Sensor fusion node close to physical sensors
- Isolated control loops for manipulator arms
- Each runs its own nodes, connects to the same NATS bus

Layer 3: Microcontrollers

TinyGo on resource-constrained devices:

- RP2040, ESP32, STM32 for real-time motor control
- Direct GPIO/PWM/ADC for hardware interfaces
- Communicate with Layer 1/2 via serial gateway
- Handle microsecond-level timing requirements

Not every robot needs all three layers. A simple robot might have a single Raspberry Pi running everything. A complex robot might have dozens of nodes across multiple boards. The architecture scales gracefully.

Message Flow

Let's trace a concrete example: a robot detecting an obstacle and stopping.

1. **Camera publishes image:** `gorai.vision.camera.image`
2. **Detector subscribes, processes, publishes:** `gorai.vision.detector.detections`
3. **Navigation subscribes, sees obstacle, publishes velocity:** `gorai.control.cmd_vel`
4. **Motor controller subscribes, applies brake:** Hardware stops

Each step is a node doing one thing well. Each message is typed and structured. The flow is observable with standard NATS tools:

```
# Watch all messages in real-time
nats sub "gorai.>"
```

This observability transforms debugging. Instead of adding print statements and recompiling, you watch message flow directly.

2.2 Core Concepts

Three concepts form Gorai's foundation: Nodes, Resources, and the Resource Model. Master these, and the framework becomes intuitive.

2.2.1 Nodes

A **Node** is the fundamental unit of execution in Gorai. It represents a process that:

- Connects to the NATS message bus
- Manages one or more resources
- Handles its own lifecycle (startup, running, shutdown)

Creating a node is straightforward:

```
n, err := node.New("my_node",
    node.WithNATS("nats://localhost:4222"),
    node.WithNamespace("robot1"),
)
if err != nil {
    log.Fatal(err)
}
defer n.Close()
```

Node Lifecycle

Nodes progress through distinct phases:

1. **Creation:** `node.New()` creates the node structure
2. **Connection:** `WithNATS()` establishes NATS connection
3. **Setup:** Create publishers, subscribers, register resources
4. **Running:** `Spin()` blocks, processing messages
5. **Shutdown:** `Shutdown()` signals stop, `Close()` releases resources

```
// The typical node lifecycle
func main() {
```

```

ctx, cancel := context.WithCancel(context.Background())
defer cancel()

// Handle signals for graceful shutdown
go func() {
    sig := make(chan os.Signal, 1)
    signal.Notify(sig, syscall.SIGINT, syscall.SIGTERM)
    <-sig
    cancel()
}()

// Create and configure node
n, _ := node.New("example", node.WithNATS("nats://localhost:4222"))
defer n.Close()

// Setup resources, publishers, subscribers...

// Run until context cancels
n.Spin(ctx)
}

```

Namespacing for Multi-Robot Systems

When multiple robots share a NATS bus, namespacing prevents collisions:

```

// Robot 1
n1, _ := node.New("sensors", node.WithNamespace("robot1"))
// Publishes to: robot1.sensors.*

// Robot 2
n2, _ := node.New("sensors", node.WithNamespace("robot2"))
// Publishes to: robot2.sensors.*

```

The `FullName()` method returns the complete identifier:

```

n.FullName() // Returns "robot1.sensors"

```

Cross-reference: See Chapter 3 for how nodes communicate via NATS.

2.2.2 Resources

A **Resource** is anything managed by Gorai: a motor, a camera, a navigation service, a sensor. All resources implement a common interface defined in `pkg/resource/resource.go` :

```

type Resource interface {
    // Name returns the unique resource identifier
    Name() Name

    // Reconfigure updates the resource with new configuration
    Reconfigure(ctx context.Context, deps Dependencies, conf Config) error

    // DoCommand executes arbitrary commands for extensibility
    DoCommand(ctx context.Context, cmd map[string]any) (map[string]any, error)

    // Close releases all resources and stops background operations
    Close(ctx context.Context) error
}

```

This interface is small by design. Every resource can:

- **Be identified** by its Name
- **Be reconfigured** at runtime without restart
- **Be extended** with custom commands via DoCommand
- **Be cleaned up** when no longer needed

Resource Naming

Resources have structured names with four parts:

```
namespace:type:subtype/instance
```

For example:

- gorai:component:motor/left_wheel
- gorai:component:sensor/cpu_temp
- gorai:service:vision/detector
- robot1:component:camera/front

This structure enables:

- **Discovery:** Find all motors with `*:component:motor/*`
- **Organization:** Group by namespace for multi-robot fleets
- **Clarity:** Names are self-documenting

Creating names in code:

```
name := resource.NewComponentName("gorai", "sensor", "cpu_temp")
// gorai:component:sensor/cpu_temp
```

Components vs Services

Resources divide into two categories:

Components abstract hardware:

- Motors, cameras, sensors, grippers
- Have physical counterparts
- Defined in the `component/` package

Services provide software capabilities:

- Vision processing, navigation, SLAM
- Pure computation, no direct hardware
- Defined in the `service/` package

This distinction matters for organization but not for the core interface—both are Resources.

Cross-reference: Chapters 4-6 detail component types; Chapter 7 covers services.

2.2.3 The Resource Model

Gorai's resource model creates a consistent hierarchy:

```
Resource (base interface)
├─ Component (hardware abstraction)
│   └─ Sensor (provides readings)
│       ├── IMU
│       ├── GPS
│       ├── Encoder
│       ├── Temperature
│       └─ ...
│   └─ Actuator (provides movement)
│       ├── Motor
│       ├── Servo
│       ├── Gripper
│       └─ ...
```

```

├─ Service (software capabilities)
│   ├── Vision
│   ├── Navigation
│   ├── SLAM
│   └── ...
└─ Camera (special case: both sensor and image provider)

```

Each level adds capabilities:

Sensor adds the ability to provide readings:

```

type Sensor interface {
    Resource
    Readings(ctx context.Context) (map[string]any, error)
}

```

Actuator adds motion control:

```

type Actuator interface {
    Resource
    IsMoving(ctx context.Context) (bool, error)
    Stop(ctx context.Context) error
}

```

Specific types like `Motor` add domain-specific methods while inheriting from `Actuator`. This layered approach means code that works with any `Actuator` works with any motor, servo, or gripper—polymorphism through interfaces, the Go way.

2.3 Distributed Architecture

Gorai is distributed by default. Even a single-board robot runs multiple nodes communicating through NATS. This section explains why and how.

Why Distributed Matters for Robotics

Robots are inherently parallel systems:

- Sensors produce data continuously
- Actuators execute commands asynchronously
- Processing happens at different rates (vision at 30Hz, IMU at 1000Hz)
- Failures in one subsystem shouldn't cascade

Traditional monolithic architectures fight this reality. A single-threaded main loop serializes inherently parallel work. Shared memory creates coupling and race conditions. A crash anywhere stops everything.

Distributed architecture embraces the reality:

- Each node runs independently at its natural rate
- Message passing provides clean, typed interfaces between subsystems
- Failure isolation protects the system
- Horizontal scaling is natural—add nodes, add capability

Primary Compute Responsibilities

The primary compute board (typically the most powerful SBC) usually handles:

NATS Server: The message broker runs here, accessible to all nodes:

```

# Start NATS server (often via scripts/start.sh)
nats-server -js # -js enables JetStream

```

High-Level Logic: Navigation planning, behavior trees, mission management:

```
// Brain node coordinates behavior
brain, _ := node.New("brain", node.WithNATS(natsURL))

// Subscribe to sensor fusion output
sub.New(brain, "gorai.perception.state", func(state *State) {
    decision := planner.Decide(state)
    cmdPub.Publish(ctx, decision.Commands)
})
```

User Interfaces: Web dashboards, API endpoints, remote control:

```
// HTTP server for monitoring
http.HandleFunc("/status", func(w http.ResponseWriter, r *http.Request) {
    status := collectStatus()
    json.NewEncoder(w).Encode(status)
})
```

Data Logging: Recording messages for debugging and replay:

```
# NATS CLI can record all traffic
nats sub "gorai.>" --dump messages.log
```

Secondary Node Use Cases

Secondary nodes handle specialized, isolated tasks:

Vision Processing: Cameras generate significant data. Processing on a dedicated node (especially one with GPU/NPU) keeps the main compute responsive:

```
// Vision node on Jetson/RK3588
vision, _ := node.New("vision", node.WithNATS(natsURL))
camera := setupCamera()
detector := loadModel()

for frame := range camera.Frames() {
    detections := detector.Detect(frame)
    detPub.Publish(ctx, detections) // Only send results, not full images
}
```

Sensor Fusion: Combine IMU, encoders, and GPS into coherent state estimates:

```
// Runs at high frequency, isolated from other processing
fusion, _ := node.New("fusion", node.WithNATS(natsURL))

// Subscribe to raw sensors
sub.New(fusion, "gorai.sensors.imu.data", imuHandler)
sub.New(fusion, "gorai.sensors.encoders.data", encoderHandler)
sub.New(fusion, "gorai.sensors.gps.data", gpsHandler)

// Publish fused state at consistent rate
ticker := time.NewTicker(10 * time.Millisecond) // 100Hz
for range ticker.C {
    state := filter.Update()
    statePub.Publish(ctx, state)
}
```

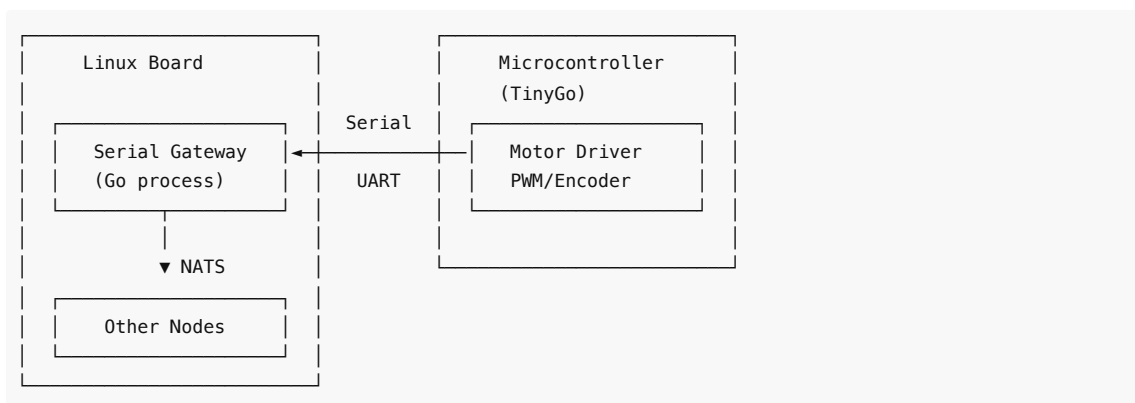
Isolated Control Loops: Arm manipulation, precise motor control:

```
// Arm controller with dedicated timing
arm, _ := node.New("arm_controller", node.WithNATS(natsURL))

// High-frequency servo loop
for {
    joints := readJointStates()
    commands := controller.Compute(target, joints)
    applyCommands(commands)
    time.Sleep(time.Millisecond) // 1kHz loop
}
```

Serial Gateway Pattern for Microcontrollers

TinyGo runs on microcontrollers (RP2040, ESP32) but can't connect directly to NATS. The serial gateway pattern bridges this gap:



The gateway translates between NATS messages and a compact serial protocol:

```
// Gateway code (runs on Linux)
gateway, _ := node.New("motor_gateway", node.WithNATS(natsURL))
serial, _ := openSerial("/dev/ttyUSB0", 115200)

// NATS to Serial
sub.New(gateway, "gorai.motors.+.command", func(cmd *MotorCommand) {
    packet := encodeCommand(cmd)
    serial.Write(packet)
})

// Serial to NATS
go func() {
    for {
        packet := readPacket(serial)
        feedback := decodeFeedback(packet)
        fbPub.Publish(ctx, feedback)
    }
}()
```

The microcontroller side handles real-time control:

```
// TinyGo code on microcontroller
func main() {
    uart := machine.UART0
    uart.Configure(machine.UARTConfig{BaudRate: 115200})

    motor := setupMotor()
```

```

encoder := setupEncoder()

for {
    if uart.Buffered() > 0 {
        cmd := readCommand(uart)
        motor.SetPower(cmd.Power)
    }

    // Send encoder feedback
    position := encoder.Read()
    sendFeedback(uart, position)

    time.Sleep(time.Millisecond)
}
}

```

This pattern gives you:

- Real-time control on dedicated hardware
- NATS integration without microcontroller networking complexity
- Clean separation between real-time and non-real-time code

2.4 Configuration & Hot Reload

Robots need configuration: motor directions, sensor calibrations, network addresses, behavioral parameters. Gorai provides a configuration system that works at runtime, not just at startup.

JSON-Based Configuration

Configuration files are JSON, readable and editable without special tools:

```

{
  "components": [
    {
      "name": "left_motor",
      "type": "motor",
      "model": "gpio",
      "attributes": {
        "pin_forward": 17,
        "pin_reverse": 18,
        "pin_pwm": 12,
        "max_rpm": 200,
        "encoder_pin": 23,
        "ticks_per_revolution": 1200
      }
    },
    {
      "name": "front_camera",
      "type": "camera",
      "model": "v4l2",
      "attributes": {
        "device": "/dev/video0",
        "width": 640,
        "height": 480,
        "fps": 30
      }
    }
  ],
  "services": [
    {
      "name": "detector",

```

```

        "type": "vision",
        "model": "yolox",
        "attributes": {
            "model_path": "/models/yolox_s.onnx",
            "confidence_threshold": 0.5,
            "nms_threshold": 0.4
        },
        "depends_on": ["front_camera"]
    }
}

```

The structure is intentional:

- **name**: Unique identifier within type
- **type**: Component/service category (motor, camera, vision)
- **model**: Specific implementation (gpio motor, v4l2 camera, yolox detector)
- **attributes**: Implementation-specific settings
- **depends_on**: Resources this one needs (for initialization order)

Loading Configuration

The `config` package parses configuration files:

```

import "github.com/gorai/gorai/pkg/config"

cfg, err := config.LoadFile("robot.json")
if err != nil {
    log.Fatal(err)
}

for _, comp := range cfg.Components {
    fmt.Printf("Component: %s (type=%s, model=%s)\n",
        comp.Name, comp.Type, comp.Model)
}

```

Configuration flows into resources through the `Config` type:

```

type Config struct {
    Attributes map[string]any
    Raw        []byte
}

// Resources receive Config during creation and reconfiguration
func (m *Motor) Reconfigure(ctx context.Context, deps Dependencies, conf Config) error {
    // Parse typed configuration
    var cfg MotorConfig
    if err := conf.Unmarshal(&cfg); err != nil {
        return err
    }

    m.maxRPM = cfg.MaxRPM
    m.ticksPerRev = cfg.TicksPerRevolution
    return nil
}

```

Runtime Reconfiguration Without Restart

The `Reconfigure()` method on every resource enables runtime updates:

```
// Change motor parameters without restarting
newConf := resource.NewConfig(map[string]any{
    "max_rpm": 250, // Increase from 200
})
motor.Reconfigure(ctx, deps, newConf)
```

This matters for:

- **Tuning:** Adjust PID gains while watching behavior
- **Adaptation:** Change parameters based on conditions (indoor vs outdoor)
- **Debugging:** Temporarily lower speeds, increase logging
- **Fleet management:** Push configuration updates to deployed robots

Dependency Injection

Resources often depend on other resources. A vision service needs a camera. A navigation service needs motors and sensors. Gorai manages these dependencies explicitly:

```
type Dependencies interface {
    Get(name Name) (Resource, error)
    GetByType(subtype string) ([]Resource, error)
    All() []Resource
}

// Vision service uses dependency injection
func NewVisionService(deps Dependencies, conf Config) (*VisionService, error) {
    cameraName := conf.GetString("camera")
    camera, err := deps.Get(resource.MustParseName(cameraName))
    if err != nil {
        return nil, fmt.Errorf("camera not found: %w", err)
    }

    return &VisionService{
        camera: camera.(Camera),
        // ...
    }, nil
}
```

The configuration's `depends_on` field ensures proper initialization order. Resources start only after their dependencies are ready.

Configuration Best Practices

Keep hardware-specific values in config, not code:

```
// Good: values from config
maxSpeed := cfg.GetFloat("max_speed")

// Bad: hardcoded values
maxSpeed := 1.5
```

Validate early:

```
func (m *Motor) Reconfigure(ctx, deps, conf) error {
    var cfg MotorConfig
    if err := conf.Unmarshal(&cfg); err != nil {
        return fmt.Errorf("invalid config: %w", err)
    }

    if cfg.MaxRPM <= 0 {
```

```

        return fmt.Errorf("max_rpm must be positive, got %f", cfg.MaxRPM)
    }

    // ... apply valid configuration
}

```

Support sensible defaults:

```

type MotorConfig struct {
    MaxRPM      float64 `json:"max_rpm"`
    TicksPerRev int     `json:"ticks_per_revolution"`
    ControlLoopHz int     `json:"control_loop_hz"`
}

func DefaultMotorConfig() MotorConfig {
    return MotorConfig{
        MaxRPM:      100,
        TicksPerRev: 1000,
        ControlLoopHz: 100,
    }
}

```

2.5 Network Transparency (NWS/NWC)

One of Gorai's most powerful features is network transparency: the ability to use resources the same way whether they're local (in the same process) or remote (on another machine).

Local vs Remote Resources

Consider a motor. When it's local, you call methods directly:

```

motor := createMotor()
motor.SetPower(ctx, 0.5) // Direct method call
position, _ := motor.GetPosition(ctx)

```

When the motor runs on a different node (perhaps a microcontroller gateway), you still want the same interface. This is where NWS (Network Wrapper Server) and NWC (Network Wrapper Client) come in.

NWS: Exposing Resources Over NATS

A Network Wrapper Server takes a local resource and exposes its methods over NATS:

```

// On the node with the physical motor
motor := createMotor()

// Wrap it for network access
wrapper := nws.Wrap(node, motor, "gorai.motors.left_wheel")

```

Now method calls arrive as NATS messages. The wrapper:

1. Subscribes to request topics
2. Deserializes incoming requests
3. Calls the actual resource method
4. Serializes and returns the response

The topics follow a pattern:

- `gorai.motors.left_wheel.SetPower` - for SetPower calls
- `gorai.motors.left_wheel.GetPosition` - for GetPosition calls
- And so on for each method

NWC: Consuming Remote Resources

A Network Wrapper Client creates a local proxy that forwards calls over NATS:

```
// On a different node (or same node, doesn't matter)
motor := nwc.Motor(node, "gorai.motors.left_wheel")

// Use it like a local motor
motor.SetPower(ctx, 0.5) // Becomes NATS request/reply
position, _ := motor.GetPosition(ctx)
```

The client proxy:

1. Serializes the method call and arguments
2. Sends a NATS request
3. Waits for the response
4. Deserializes and returns the result

Transparent Location Abstraction

The magic is that consuming code doesn't know (or care) if a resource is local or remote:

```
func RunBehavior(motor motor.Motor) {
    // This function works with local or remote motors
    for i := 0; i < 10; i++ {
        motor.SetPower(ctx, float64(i) / 10)
        time.Sleep(100 * time.Millisecond)
    }
    motor.Stop(ctx)
}

// Works with local motor
localMotor := createMotor()
RunBehavior(localMotor)

// Works with remote motor
remoteMotor := nwc.Motor(node, "gorai.motors.left_wheel")
RunBehavior(remoteMotor)
```

Use Cases

Distributed Robot Architecture: Camera processing on a GPU node, motion control on the main board:

```
// Vision node exposes camera and detector
camera := setupCamera()
nws.Wrap(visionNode, camera, "gorai.cameras.front")

detector := setupDetector()
nws.Wrap(visionNode, detector, "gorai.vision.detector")

// Brain node consumes them remotely
camera := nwc.Camera(brainNode, "gorai.cameras.front")
detector := nwc.Vision(brainNode, "gorai.vision.detector")
```

Remote Monitoring/Control: Operator station accessing robot resources:

```
// On operator laptop
robot := nwc.Connect("nats://robot.local:4222")
motor := nwc.Motor(robot, "gorai.motors.left_wheel")
```

```
// Interactive control
motor.SetPower(ctx, *joystickInput)
```

Testing with Resource Injection: Use fake resources transparently:

```
// In tests, expose a fake motor
fake := fake.NewMotor()
nws.Wrap(testNode, fake, "test.motor")

// Code under test connects to it
motor := nwc.Motor(sutNode, "test.motor")
RunBehavior(motor)

// Verify fake received expected calls
assert.True(t, fake.StopWasCalled())
```

Performance Considerations

Network transparency has overhead:

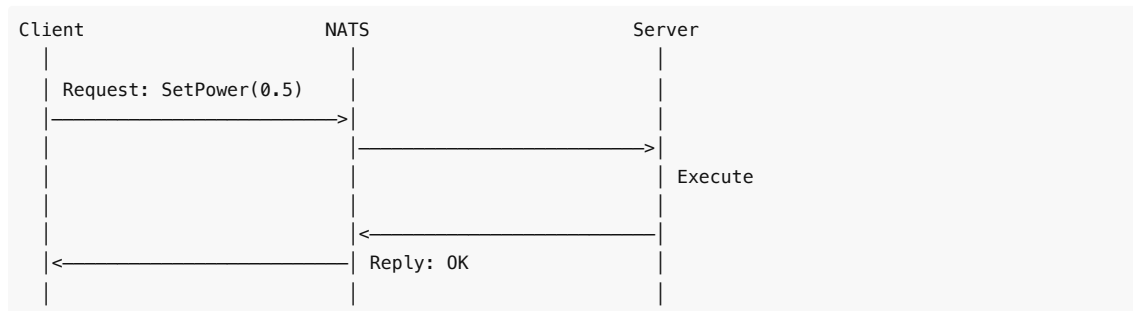
- Serialization/deserialization for each call
- Network latency (microseconds locally, milliseconds across network)
- NATS message processing

For high-frequency operations (1kHz control loops), prefer local resources or the serial gateway pattern. Reserve NWS/NWC for:

- Infrequent operations (configuration, status checks)
- Operations where network latency is acceptable
- Cross-node coordination

Implementation Details

Under the hood, NWS/NWC use NATS request/reply:



Errors propagate correctly—if the remote motor fails, the error returns through the client proxy. Timeouts are configurable. Context cancellation works as expected.

With the mental model established—nodes, resources, distributed architecture, configuration, and network transparency—you're ready to understand Gorai's communication backbone. Chapter 3 dives deep into NATS.

Chapter 3: NATS - The Communication Backbone

NATS is the foundation of Gorai's communication. Understanding NATS deeply transforms how you think about robot architecture.

3.1 Why NATS?

Gorai could have built on many messaging systems: ROS 2's DDS, ZeroMQ, gRPC, MQTT, or custom protocols. NATS won for compelling reasons.

Cloud-Native Messaging for Robotics

NATS was built for cloud infrastructure—systems with thousands of services, unreliable networks, and demanding performance requirements. These constraints mirror robotics:

- **Many producers and consumers:** Sensors publish, multiple nodes subscribe
- **Unreliable connections:** WiFi drops, nodes restart, processes crash
- **Low latency requirements:** Control loops can't wait
- **Simple operations:** No time for complex configuration

NATS brings cloud-hardened solutions to robotics problems.

Performance Characteristics

NATS is fast. Benchmarks show:

- **18+ million messages/second** on modest hardware (single server)
- **Sub-millisecond latency** for typical messages
- **Minimal CPU overhead:** More cycles for your robot logic

For comparison, ROS 2 with DDS can struggle to saturate a gigabit link. NATS handles it trivially.

Memory usage is also lean. The NATS server runs in tens of megabytes. Clients add negligible overhead. This matters when your robot's brain is a Raspberry Pi, not a data center.

Comparison with Alternatives

vs ROS 2 DDS:

- DDS is enterprise middleware designed for defense and aerospace
- Multiple implementations (CycloneDDS, FastDDS, Connexnt) with different behaviors
- Complex QoS configuration with dozens of parameters
- NATS: One implementation, simple config, predictable behavior

vs ZeroMQ:

- ZeroMQ is a library, not a broker—each node manages its own connections
- Discovery requires custom solutions
- NATS: Broker simplifies topology, built-in discovery via subjects

vs gRPC:

- gRPC is point-to-point, not pub/sub
- Requires knowing endpoints ahead of time
- NATS: Loose coupling, dynamic discovery, pub/sub native

vs MQTT:

- MQTT is designed for IoT telemetry—small messages, constrained devices
- Limited pub/sub patterns, no request/reply
- NATS: Full messaging patterns, higher performance, JetStream for persistence

JetStream for Persistence

Core NATS is fire-and-forget: if no subscriber is listening, messages disappear. JetStream adds persistence:

- **Streams:** Store messages durably
- **Consumers:** Track what each subscriber has seen
- **Replay:** New subscribers can catch up on history
- **Acknowledgment:** Ensure messages are processed

Gorai uses core NATS for real-time data (sensor streams, control commands) and JetStream when durability matters (configuration updates, logged data, mission waypoints).

```
// Core NATS: fast, no persistence
pub := pub.New[*sensor.IMU](node, "sensors.imu.data")

// JetStream: reliable, persisted
pub := pub.New[*config.Update](node, "config.updates",
    pub.WithQoS(pub.Reliable))
```

Operational Simplicity

NATS runs as a single binary with zero dependencies:

```
# That's it. NATS is running.
nats-server -js

# Or with a config file
nats-server -c /etc/nats/nats.conf
```

No ZooKeeper, no etcd, no Kubernetes. NATS can run on a Raspberry Pi as easily as in a cloud cluster.

Clustering is straightforward when you need it:

```
# Three-server cluster for high availability
nats-server -c server1.conf
nats-server -c server2.conf
nats-server -c server3.conf
```

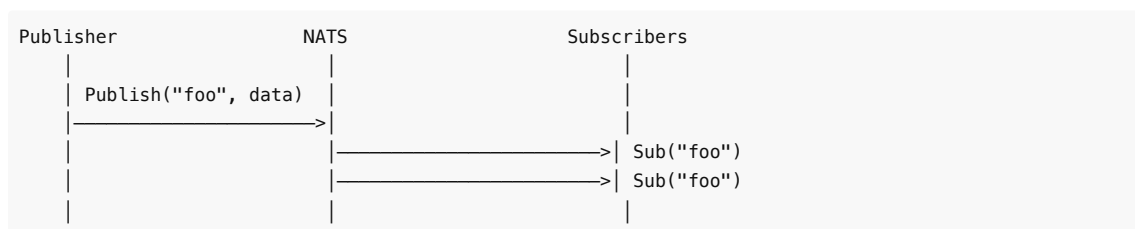
For most robots, a single local NATS server is sufficient. The option to scale exists when needed.

3.2 NATS Fundamentals

Before diving into Gorai's patterns, let's understand NATS primitives.

Publish/Subscribe Basics

NATS pub/sub is simple: publishers send to subjects, subscribers listen on subjects:



Subjects are strings with dot-separated hierarchies:

- `sensors.imu.data`
- `motors.left.command`
- `vision.camera.front.image`

This isn't just convention—it enables wildcard subscriptions.

Request/Reply Pattern

NATS supports RPC-style synchronous calls:

```
// Requester
response, err := nc.Request("services.detector", request, timeout)

// Responder
nc.Subscribe("services.detector", func(msg *nats.Msg) {
    result := process(msg.Data)
    msg.Respond(result)
})
```

Under the hood, NATS creates a temporary inbox subject for the reply. This pattern is perfect for:

- Getting current sensor values
- Querying component status
- Invoking service methods

Wildcards and Subject Hierarchies

NATS wildcards make subscriptions powerful:

Single-level wildcard (*): Matches exactly one token

```
// Matches: sensors.imu.data, sensors.gps.data
// Not: sensors.imu.calibration.data
nc.Subscribe("sensors.*.data", handler)
```

Multi-level wildcard (>): Matches one or more tokens

```
// Matches: sensors.anything, sensors.a.b.c.d
nc.Subscribe("sensors.>", handler)
```

Practical uses:

```
// All motor commands for any motor
nc.Subscribe("motors.*.command", handler)

// Everything from robot1
nc.Subscribe("robot1.>", handler)

// All camera images from any namespace
nc.Subscribe("*.cameras.*.image", handler)
```

Connection Management

NATS clients handle connection lifecycle automatically:

```
nc, err := nats.Connect("nats://localhost:4222",
    nats.Name("my_node"),           // Identify in server logs
    nats.ReconnectWait(time.Second), // Retry interval
    nats.MaxReconnects(-1),         // Retry forever
)
```

The client automatically:

- Reconnects on disconnect
- Re-subscribes after reconnection
- Buffers messages during brief outages

Gorai's `node.New()` configures these sensibly by default:

```
n, err := node.New("my_node", node.WithNATS("nats://localhost:4222"))
// Reconnection, buffering, etc. are configured automatically
```

Observing Messages

The `nats` CLI is invaluable for debugging:

```
# Subscribe to everything
nats sub ">"

# Subscribe to sensor data
nats sub "sensors.>"

# Publish a test message
nats pub "test.topic" "hello world"

# Request/reply
nats request "services.echo" "ping"
```

During development, keep a terminal running `nats sub ">"` to watch all traffic. It's like `tcpdump` for your robot's nervous system.

Subject Naming Conventions

Gorai follows consistent naming:

```
{namespace}.{type}.{name}.{suffix}
```

Examples:

```
gorai.sensors.imu.data
gorai.motors.left.command
gorai.services.detector.request
robot1.cameras.front.image
```

Where:

- **namespace**: Organization or robot identifier
- **type**: Category (sensors, motors, cameras, services)
- **name**: Specific instance
- **suffix**: Data type or operation (data, command, request, response)

This structure enables useful wildcard patterns:

```
// All sensors from this robot
nc.Subscribe("gorai.sensors.>", handler)

// All motor commands
nc.Subscribe("gorai.motors.*.command", handler)

// Everything from robot1
nc.Subscribe("robot1.>", handler)
```

3.3 Gorai's NATS Patterns

Gorai builds three communication patterns on NATS: Topics (pub/sub), Services (request/reply), and Actions (long-running with feedback).

3.3.1 Topics (Pub/Sub)

Topics are the primary pattern for streaming data. Sensors publish continuously; interested nodes subscribe.

Publishing sensor data:

```
// Create a typed publisher
pub := pub.New[*sensor.Temperature](node, "gorai.sensors.temp.data")

// In your reading loop
for reading := range temperatureReadings() {
    msg := &sensor.Temperature{
        Header:    makeHeader(),
        Temperature: reading.Celsius,
        Variance:   reading.Variance,
    }
    pub.Publish(ctx, msg)
}
```

Subscribing to sensor data:

```
sub.New[*sensor.Temperature](node, "gorai.sensors.temp.data",
    func(msg *sensor.Temperature) {
        log.Printf("Temperature: %.1f°C", msg.Temperature)
    })
```

Telemetry publishing follows the same pattern:

```
// Battery monitor
battPub := pub.New[*sensor.BatteryState](node, "gorai.power.battery.state")

ticker := time.NewTicker(time.Second)
for range ticker.C {
    state := readBatteryState()
    battPub.Publish(ctx, state)
}
```

Topic naming conventions:

gorai.{node}.{component}.{datatype}

Examples:

gorai.hello.cpu_temp.data
gorai.sensors.imu.data
gorai.motors.left.feedback
gorai.cameras.front.image

Cross-reference: See Chapter 4 for how sensor data flows over topics.

3.3.2 Services (Request/Reply)

Services handle synchronous operations: "give me the current value" or "execute this command and tell me if it worked."

Implementing a service:

```
// Register a handler for motor commands
nc.Subscribe("gorai.motors.left.set_power", func(msg *nats.Msg) {
    var req MotorPowerRequest
    proto.Unmarshal(msg.Data, &req)

    err := motor.SetPower(ctx, req.Power)

    resp := &MotorPowerResponse{Success: err == nil}
    if err != nil {
```

```

        resp.Error = err.Error()
    }

    data, _ := proto.Marshal(resp)
    msg.Respond(data)
})

```

Calling a service:

```

req := &MotorPowerRequest{Power: 0.5}
data, _ := proto.Marshal(req)

respMsg, err := nc.Request("gorai.motors.left.set_power", data, time.Second)
if err != nil {
    return fmt.Errorf("request failed: %w", err)
}

var resp MotorPowerResponse
proto.Unmarshal(respMsg.Data, &resp)
if !resp.Success {
    return fmt.Errorf("motor error: %s", resp.Error)
}

```

Timeout handling is critical for robotics:

```

// Short timeout for control commands
resp, err := nc.Request(subject, data, 100*time.Millisecond)
if err == nats.ErrTimeout {
    // Handle timeout—maybe stop motors for safety
    emergencyStop()
}

```

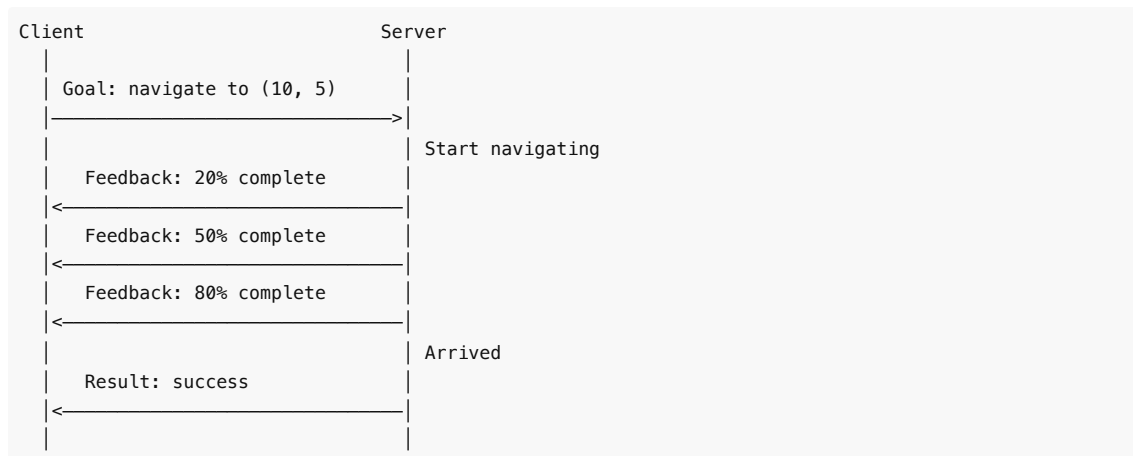
Cross-reference: See Chapter 7 for higher-level service implementations.

3.3.3 Actions (Long-Running)

Actions handle operations that take time and provide progress updates: navigation to a goal, arm movements, scanning routines.

The pattern involves three message types:

- **Goal:** What to do
- **Feedback:** Progress updates during execution
- **Result:** Final outcome



Server implementation:

```
server, _ := action.NewServer[*NavGoal, *NavFeedback, *NavResult](
    node, "navigation.go_to",
    func(ctx context.Context, handle *action.GoalHandle[*NavGoal, *NavFeedback, *NavResult]) {
        goal := handle.Goal()

        for !atGoal(goal.Position) {
            if handle.IsCanceling() {
                handle.SetCanceled(&NavResult{Success: false})
                return
            }

            // Move toward goal
            step := computeStep(goal.Position)
            executeStep(step)

            // Send feedback
            handle.SendFeedback(&NavFeedback{
                DistanceRemaining: distanceTo(goal.Position),
                Progress:           computeProgress(),
            })

            time.Sleep(100 * time.Millisecond)
        }

        handle.SetSucceeded(&NavResult{
            Success:      true,
            FinalPosition: currentPosition(),
        })
    },
)
```

Client usage:

```
client, _ := action.NewClient[*NavGoal, *NavFeedback, *NavResult](
    node, "navigation.go_to")

goal := &NavGoal{Position: &geometry.Point{X: 10, Y: 5}}
handle, _ := client.SendGoal(ctx, goal)

// Monitor feedback
for fb := range handle.Feedback() {
    log.Printf("Progress: %.1f%%, Distance: %.2fm",
        fb.Progress*100, fb.DistanceRemaining)
}

// Get result
result, err := handle.Wait(ctx)
if result.Success {
    log.Printf("Arrived at %v", result.FinalPosition)
}
```

Cancellation support:

```
// Client can cancel
handle.Cancel()

// Server checks for cancellation
if handle.IsCanceling() {
```

```

    cleanup()
    handle.SetCanceled(&NavResult{})
    return
}

```

Use actions for:

- Navigation to waypoints
- Arm trajectory execution
- Scanning/searching behaviors
- Any operation lasting more than a few seconds

3.4 Quality of Service (QoS)

Not all messages have the same requirements. A control command must arrive immediately but can be lost if the subscriber isn't ready. A configuration update must be delivered reliably. Gorai provides QoS levels for these different needs.

BestEffort: Core NATS

The default QoS—simple, fast, no persistence:

```

pub := pub.New[*sensor.IMU](node, "sensors.imu.data")
// Uses core NATS, no JetStream

```

Characteristics:

- **Lowest latency:** Direct publish to subscribers
- **No storage:** If no subscriber is listening, message is lost
- **No acknowledgment:** Publisher doesn't know if anyone received it
- **Minimal overhead:** Just network I/O

Use for:

- High-frequency sensor data (IMU at 1kHz)
- Real-time control commands
- Any data where the next message supersedes the previous

Example: IMU data stream

```

pub := pub.New[*sensor.Imu](node, "gorai.sensors.imu.data")

for reading := range imu.Readings() {
    pub.Publish(ctx, reading)
    // If subscribers miss one, the next arrives in 1ms anyway
}

```

Reliable: JetStream Acknowledgment

Messages are persisted and delivery is guaranteed:

```

pub := pub.New[*config.Update](node, "gorai.config.updates",
    pub.WithQoS(pub.Reliable))

```

Characteristics:

- **Persistence:** Messages stored in JetStream stream
- **Acknowledgment:** Publisher knows message was stored
- **Redelivery:** Failed deliveries are retried
- **Higher overhead:** Storage I/O, acknowledgment round-trip

Use for:

- Configuration updates

- Mission waypoints
- Critical commands that must not be lost
- Logging and telemetry that must be preserved

Example: Configuration distribution

```
pub := pub.New[*config.RobotConfig](node, "gorai.config.robot",
    pub.WithQoS(pub.Reliable))

// When config changes, publish reliably
pub.Publish(ctx, newConfig)
// Returns only after message is persisted
```

Retained: Last-Value Retention

Only the most recent message per subject is kept:

```
pub := pub.New[*sensor.BatteryState](node, "gorai.power.battery",
    pub.WithRetain())
```

Characteristics:

- **Last value available:** New subscribers immediately get current state
- **Automatic cleanup:** Old values are replaced
- **JetStream storage:** Persisted, but only one message per subject

Use for:

- Current status/state
- Configuration that should be available to new subscribers
- "What is X right now?" queries

Example: Battery status

```
pub := pub.New[*sensor.BatteryState](node, "gorai.power.battery",
    pub.WithRetain())

// Publish periodically
ticker := time.NewTicker(time.Second)
for range ticker.C {
    pub.Publish(ctx, readBatteryState())
}

// New subscribers immediately get the latest state
sub.New[*sensor.BatteryState](node, "gorai.power.battery", handler,
    sub.WithDeliverLast())
```

History: Message Buffering

Keep the last N messages for late-joining subscribers:

```
pub := pub.New[*sensor.Odometry](node, "gorai.odom.data",
    pub.WithHistory(100)) // Keep last 100 messages
```

Characteristics:

- **Catch-up:** New subscribers can replay recent history
- **Bounded storage:** Only N messages per subject
- **Ordered delivery:** Messages arrive in sequence

Use for:

- Odometry data (for pose estimation catch-up)
- Event logs
- Any stream where context from recent past matters

Example: Odometry with history

```
// Publisher keeps history
pub := pub.New[*nav.Odometry](node, "gorai.odom.data",
    pub.WithHistory(100))

// Late subscriber catches up
sub.New[*nav.Odometry](node, "gorai.odom.data", handler,
    sub.WithDeliverAll()) // Get all available history first
```

Choosing the Right QoS

Scenario	QoS	Rationale
IMU at 1kHz	BestEffort	Speed matters, missing one is fine
Motor commands	BestEffort	Latest command supersedes previous
Configuration updates	Reliable	Must not be lost
Current battery level	Retained	New nodes need current value
Odometry stream	History	Localization needs recent context
Logged events	Reliable	Must be preserved
Camera images	BestEffort	Too large for persistence at frame rate

Default to **BestEffort**. Only use JetStream QoS when you need its guarantees—the overhead is real.

3.5 JetStream Features

JetStream is NATS's persistence layer. When you need messages to survive restarts, handle late subscribers, or guarantee delivery, JetStream provides the mechanisms.

Streams and Consumers

Streams store messages:

```
// Gorai creates streams automatically when using JetStream QoS
// But you can create them manually for advanced control
js, _ := nc.JetStream()

_, err := js.AddStream(&nats.StreamConfig{
    Name:      "SENSOR_DATA",
    Subjects: []string{"gorai.sensors.>"},
    Storage:   nats.FileStorage,
    MaxMsgs:   1000000,
    MaxAge:    24 * time.Hour,
})
```

Stream configuration options:

- **Storage:** `FileStorage` (persistent) or `MemoryStorage` (faster, volatile)
- **MaxMsgs:** Maximum messages to retain
- **MaxAge:** Maximum message age before deletion
- **MaxBytes:** Maximum storage size
- **Replicas:** Number of copies for HA (in clusters)

Consumers track subscriber progress:

```
// Push consumer: messages delivered as they arrive
sub, _ := js.Subscribe("gorai.sensors.>",
    handler,
    nats.Durable("sensor_processor"),
    nats.DeliverAll(),
)

// Pull consumer: subscriber requests messages
sub, _ := js.PullSubscribe("gorai.sensors.>",
    "batch_processor",
    nats.AckExplicit(),
)
msgs, _ := sub.Fetch(100) // Get up to 100 messages
```

Durable Subscriptions

Durable consumers remember their position across restarts:

```
sub, _ := sub.New[*sensor.Temperature](node, "gorai.sensors.temp.data",
    handler,
    sub.WithDurable("temp_logger"), // Named consumer
)

// After restart, continues from where it left off
```

Without durability, restarting a subscriber loses track of which messages were processed. With durability:

1. First run: Processes messages 1-100
2. Restart
3. Second run: Continues from message 101

Critical for:

- Log processors that shouldn't miss messages
- Event handlers that need exactly-once semantics
- Offline-capable nodes that catch up after reconnection

Replay Capabilities

JetStream enables powerful replay scenarios:

Start from beginning:

```
sub.New(node, topic, handler, sub.WithDeliverAll())
```

Start from last message:

```
sub.New(node, topic, handler, sub.WithDeliverLast())
```

Start from new messages only (default):

```
sub.New(node, topic, handler, sub.WithDeliverNew())
```

Replay by sequence number (advanced):

```
js.Subscribe(subject, handler,
    nats.StartSequence(12345))
```

Replay by time:

```
js.Subscribe(subject, handler,
    nats.StartTime(time.Now().Add(-1*time.Hour)))
```

Use cases:

- Debugging: Replay sensor data through a fixed algorithm
- Testing: Run the same inputs through new code
- Recovery: Re-process events after a crash
- Analysis: Historical data review

Cross-reference: Chapter 11 covers using replay for testing.

Acknowledgment and Redelivery

JetStream tracks message acknowledgment:

```
js.Subscribe(subject, func(msg *nats.Msg) {
    err := process(msg)
    if err != nil {
        msg.Nak() // Negative acknowledgment: redeliver
        return
    }
    msg.Ack() // Success: don't redeliver
})
```

Acknowledgment options:

- **Ack()**: Message processed successfully
- **Nak()**: Processing failed, redeliver soon
- **Term()**: Don't redeliver (poison message)
- **InProgress()**: Still working, extend timeout

Configuration controls redelivery:

```
sub.WithAckWait(30 * time.Second) // Wait before redelivering
sub.WithMaxDeliver(5)             // Give up after 5 attempts
```

Practical JetStream Usage

Recording robot sessions:

```
# Record all messages to a stream
nats stream add RECORDING \
    --subjects "gorai.>" \
    --storage file \
    --max-age 1h

# Later, replay for analysis
nats consumer add RECORDING analyzer \
    --deliver all \
    --replay instant
```

Mission-critical commands:

```
// Ensure waypoints are never lost
waypointPub := pub.New[*nav.Waypoint](node, "gorai.mission.waypoints",
    pub.WithQoS(pub.Reliable))
```

```
// On subscriber side, acknowledge after persisting
sub.New[*nav.Waypoint](node, "gorai.mission.waypoints",
    func(wp *nav.Waypoint) {
        saveToDatabase(wp)
        // JetStream auto-acks after handler returns without error
    },
    sub.WithSubQoS(sub.Reliable))
```

Fleet telemetry collection:

```
// Stream from all robots
js.AddStream(&nats.StreamConfig{
    Name:      "FLEET_TELEMETRY",
    Subjects: []string{"*.telemetry.>"}, // Any robot's telemetry
    Storage:   nats.FileStorage,
    MaxAge:    7 * 24 * time.Hour, // Keep a week
})
```

3.6 The NATS CLI

The `nats` command-line tool is indispensable for Gorai development. It lets you observe, debug, and interact with the message bus directly.

Installation

```
# macOS
brew install nats-io/nats-tools/nats

# Linux (via go install)
go install github.com/nats-io/natscli/nats@latest

# Or download from releases
# https://github.com/nats-io/natscli/releases
```

Verify installation:

```
nats --version
```

Basic Commands

Subscribe to messages:

```
# All messages
nats sub ">"

# All sensor messages
nats sub "gorai.sensors.>"

# Specific topic
nats sub "gorai.sensors.temp.data"

# With timestamps
nats sub "gorai.>" --raw
```

Publish messages:

```
# Simple text
nats pub "test.topic" "hello world"
```

```
# JSON
nats pub "gorai.test" '{"value": 42}'

# From file
nats pub "gorai.config" --file config.json
```

Request/Reply:

```
# Send request, wait for reply
nats request "gorai.services.echo" "ping" --timeout 5s
```

Monitoring with nats server

Check server status:

```
# Server info
nats server info

# Connection list
nats server connections

# Request counts
nats server report connections
```

Real-time monitoring:

```
# Watch message rates
nats server report accounts --top

# Stream activity
nats stream report
```

JetStream Commands

Stream management:

```
# List streams
nats stream list

# Stream info
nats stream info SENSOR_DATA

# View messages in stream
nats stream view SENSOR_DATA

# Purge (delete all messages)
nats stream purge SENSOR_DATA
```

Consumer management:

```
# List consumers
nats consumer list SENSOR_DATA

# Consumer info (shows lag, pending, etc.)
nats consumer info SENSOR_DATA my_consumer
```

```
# Get next message
nats consumer next SENSOR_DATA my_consumer
```

Debugging Robot Communication

Watch all traffic during development:

```
# Terminal 1: Watch everything
nats sub ">" --raw

# Terminal 2: Run your robot
go run ./examples/hello-sensor
```

Filter for specific patterns:

```
# Only motor commands
nats sub "gorai.motors.*.command"

# Only errors/warnings (if you publish them)
nats sub "gorai.*.error" "gorai.*.warn"
```

Interactive testing:

```
# Test a motor service manually
nats request "gorai.motors.left.set_power" \
  '{"power": 0.5}' \
  --timeout 1s

# Simulate sensor data
nats pub "gorai.sensors.fake.data" \
  '{"temperature": 42.5}'
```

Measure latency:

```
# Round-trip time to server
nats rtt

# Latency distribution
nats bench "test.latency" --pub 1000 --sub 1 --size 256
```

Useful One-Liners

```
# Count messages per second on a topic
nats sub "gorai.sensors.imu.data" --count 1000 2>&1 | tail -1

# Dump last N messages from a stream
nats stream view SENSOR_DATA --last 10

# Create a quick test stream
nats stream add TEST --subjects "test.>" --storage memory

# Follow logs with pretty JSON
nats sub "gorai.logs.>" | jq .

# Export stream contents
nats stream view SENSOR_DATA --json > data.jsonl
```

Cross-reference: Chapter 9 uses these commands to observe the hello-sensor example.

With a solid understanding of NATS—its patterns, QoS levels, JetStream features, and debugging tools—you're ready to explore how Gorai uses these capabilities for specific component types. Chapter 4 begins with sensors.

Chapter 4: Components - Sensors

Sensors are the robot's eyes, ears, and proprioception. They transform physical phenomena—light, temperature, acceleration, distance—into data structures your software can reason about.

4.1 The Sensor Interface

Every sensor in Gorai implements a simple interface from `pkg/resource/resource.go` :

```
type Sensor interface {
    Resource

    // Readings returns the current sensor readings as key-value pairs.
    // The keys and value types depend on the specific sensor implementation.
    Readings(ctx context.Context) (map[string]any, error)
}
```

That's it. One method beyond the base Resource interface. This simplicity is intentional.

Why map[string]any for Readings

Different sensors produce radically different data:

- Temperature sensor: Single float (degrees Celsius)
- IMU: Nine floats (3-axis acceleration, gyroscope, magnetometer)
- GPS: Latitude, longitude, altitude, accuracy, satellite count
- LiDAR: Thousands of range measurements

A fixed return type would either be too restrictive or require sensor-specific interfaces for every sensor type. The `map[string]any` approach provides:

- **Flexibility:** Any sensor can return whatever data it produces
- **Discoverability:** Print the map to see what's available
- **Forward compatibility:** New readings can be added without interface changes

```
readings, _ := tempSensor.Readings(ctx)
// Output: map[temperature_celsius:42.5 temperature_fahrenheit:108.5 zone:thermal_zone0]

readings, _ := imu.Readings(ctx)
// Output: map[accel_x:0.01 accel_y:-0.02 accel_z:9.81 gyro_x:0.001 ...]
```

Standard Reading Keys

While sensors can return any keys, conventions enable interoperability:

Key Pattern	Type	Description
temperature_celsius	float64	Temperature in Celsius
temperature_fahrenheit	float64	Temperature in Fahrenheit
accel_x, accel_y, accel_z	float64	Acceleration (m/s²)
gyro_x, gyro_y, gyro_z	float64	Angular velocity (rad/s)
latitude, longitude	float64	GPS coordinates (degrees)
altitude	float64	Altitude (meters)
distance	float64	Range measurement (meters)
battery_percent	float64	Battery level (0-100)

Following conventions enables generic processing:

```
// Works with any temperature sensor
temp, ok := readings["temperature_celsius"].(float64)
if ok && temp > 80 {
    log.Warn("High temperature detected")
}
```

Timestamp Handling

Readings represent a point in time. The sensor implementation should include when the measurement was taken:

```
func (s *TemperatureSensor) Readings(ctx context.Context) (map[string]any, error) {
    reading := readHardware()

    return map[string]any{
        "temperature_celsius": reading.Value,
        "timestamp":           time.Now(),
        "measurement_duration": reading.Duration,
    }, nil
}
```

For Protocol Buffer messages, timestamps are explicit:

```
msg := &sensor.Temperature{
    Header: &std.Header{
        Stamp: timestamppb.Now(),
        FrameId: "thermal_zone0",
    },
    Temperature: reading.Value,
    Variance:    reading.Variance,
}
```

Implementing the Sensor Interface

A minimal sensor implementation:

```
type SimpleSensor struct {
    name resource.Name
}

func (s *SimpleSensor) Name() resource.Name {
    return s.name
}

func (s *SimpleSensor) Reconfigure(ctx context.Context, deps resource.Dependencies, conf resource.Config) error {
    // Update configuration if needed
    return nil
}

func (s *SimpleSensor) DoCommand(ctx context.Context, cmd map[string]any) (map[string]any, error) {
    return nil, fmt.Errorf("no commands supported")
}

func (s *SimpleSensor) Close(ctx context.Context) error {
    return nil
}
```

```
func (s *SimpleSensor) Readings(ctx context.Context) (map[string]any, error) {
    value := readFromHardware()
    return map[string]any{
        "value": value,
    }, nil
}
```

The real complexity lies in:

- Hardware communication (I2C, SPI, GPIO, serial)
- Parsing hardware data formats
- Calibration and unit conversion
- Error handling and recovery

4.2 Built-in Sensor Types

Gorai provides interfaces and implementations for common sensor types. Each builds on the base Sensor interface with domain-specific methods and data structures.

4.2.1 Temperature Sensor

The simplest sensor type—a single scalar measurement:

```
// Standard readings
map[string]any{
    "temperature_celsius": 42.5,
    "temperature_fahrenheit": 108.5,
    "zone": "thermal_zone0",
    "critical_celsius": 105.0, // Optional: thermal limits
    "warning_celsius": 85.0,
}
```

Common sources:

- Linux thermal zones (/sys/class/thermal/thermal_zone*/temp)
- I2C sensors (TMP102, BME280, DS18B20)
- ADC-based thermistors

The hello-sensor example in Chapter 9 implements a complete temperature sensor.

Cross-reference: See Chapter 9 for complete implementation details.

4.2.2 IMU (Inertial Measurement Unit)

IMUs combine multiple sensors measuring motion:

```
// Standard readings
map[string]any{
    // Accelerometer (m/s²)
    "accel_x": 0.01,
    "accel_y": -0.02,
    "accel_z": 9.81, // Gravity

    // Gyroscope (rad/s)
    "gyro_x": 0.001,
    "gyro_y": 0.002,
    "gyro_z": 0.000,

    // Magnetometer (µT) – if available
    "mag_x": 25.3,
    "mag_y": 5.1,
```

```
    "mag_z": 42.7,
}
```

Coordinate frames matter for IMUs. Gorai follows REP 103 conventions:

- X: Forward
- Y: Left
- Z: Up

Document your IMU's native frame and any transformations applied.

Protocol Buffer representation from `sensor.proto` :

```
message Imu {
    std.Header header = 1;

    geometry.Quaternion orientation = 2;
    double orientation_covariance = 3;

    geometry.Vector3 angular_velocity = 4;
    double angular_velocity_covariance = 5;

    geometry.Vector3 linear_acceleration = 6;
    double linear_acceleration_covariance = 7;
}
```

Calibration considerations:

- Accelerometer bias: Subtract offset measured at rest
- Gyroscope drift: Integrate error accumulates over time
- Magnetometer hard/soft iron: Requires figure-8 calibration routine

4.2.3 GPS

GPS sensors provide position on Earth:

```
map[string]any{
    "latitude":      37.4220,      // Degrees
    "longitude":     -122.0841,    // Degrees
    "altitude":      10.5,         // Meters above sea level
    "horizontal_accuracy": 2.5,     // Meters (CEP)
    "vertical_accuracy": 4.0,      // Meters
    "speed":         1.2,         // m/s
    "heading":       45.0,        // Degrees from north
    "satellites":    12,          // Satellites in view
    "fix_type":      "3d",        // "none", "2d", "3d", "rtk"
}
```

NMEA parsing: Most GPS modules output NMEA sentences over serial:

```
$GPGGA,123519,4807.038,N,01131.000,E,1,08,0.9,545.4,M,47.0,M,,*47
```

Gorai GPS implementations parse these into structured data.

Integration with navigation: GPS alone isn't sufficient for robot localization—it's too slow and inaccurate. Combine with IMU and wheel odometry for sensor fusion.

4.2.4 Encoder

Encoders measure rotational position, typically for motors:

```
map[string]any{
    "position":    1234.5,    // Ticks or radians
    "velocity":    10.2,      // Ticks/s or rad/s
    "ticks_per_rev": 1200,    // Encoder resolution
    "direction":    1,        // 1 = forward, -1 = reverse
}
```

Quadrature encoding: Most encoders output two signals (A and B) 90° out of phase, allowing direction detection and 4x resolution.

Velocity calculation: Differentiate position over time, but filter to reduce noise:

```
func (e *Encoder) calculateVelocity() float64 {
    now := time.Now()
    dt := now.Sub(e.lastTime).Seconds()
    dp := e.position - e.lastPosition

    e.lastTime = now
    e.lastPosition = e.position

    // Low-pass filter
    rawVelocity := dp / dt
    e.filteredVelocity = 0.8*e.filteredVelocity + 0.2*rawVelocity

    return e.filteredVelocity
}
```

4.2.5 Range Finders

Distance sensors come in several technologies:

Ultrasonic (e.g., HC-SR04):

```
map[string]any{
    "distance":    0.42,      // Meters
    "min_range":    0.02,      // Minimum detectable
    "max_range":    4.0,       // Maximum range
    "field_of_view": 0.26,     // Radians (~15°)
}
```

Pros: Cheap, works with any surface Cons: Slow (limited update rate), wide beam, temperature sensitive

Infrared (e.g., Sharp GP2Y0A21):

```
map[string]any{
    "distance":    0.25,
    "min_range":    0.10,
    "max_range":    0.80,
}
```

Pros: Fast, narrow beam Cons: Surface-dependent (dark surfaces absorb IR)

LiDAR (e.g., RPLIDAR, Hokuyo):

```
map[string]any{
    "ranges":      []float64{...}, // Array of distances
    "angles":      []float64{...}, // Corresponding angles
    "min_range":    0.15,
    "max_range":    12.0,
    "angle_min":    -3.14159,      // -180°
}
```

```

    "angle_max": 3.14159,          // +180°
    "scan_time": 0.1,             // Seconds per scan
}

```

Point cloud generation: For 3D sensing (RGB-D cameras, rotating LiDAR):

```

message PointCloud2 {
    std.Header header = 1;
    uint32 height = 2;
    uint32 width = 3;
    repeated PointField fields = 4;
    bool is_bigendian = 5;
    uint32 point_step = 6;
    uint32 row_step = 7;
    bytes data = 8;
    bool is_dense = 9;
}

```

4.3 Sensor Data Types (Protocol Buffers)

While `Readings()` returns dynamic maps, structured sensor data uses Protocol Buffers for efficient serialization and type safety.

The sensor.proto Definitions

Gorai defines standard sensor messages in `api/proto/gorai/sensor/sensor.proto` :

```

syntax = "proto3";
package gorai.sensor;

import "gorai/std/std.proto";
import "gorai/geometry/geometry.proto";

// Imu - Inertial Measurement Unit data
message Imu {
    std.Header header = 1;

    geometry.Quaternion orientation = 2;
    repeated double orientation_covariance = 3;

    geometry.Vector3 angular_velocity = 4;
    repeated double angular_velocity_covariance = 5;

    geometry.Vector3 linear_acceleration = 6;
    repeated double linear_acceleration_covariance = 7;
}

// Image - Raw camera image
message Image {
    std.Header header = 1;
    uint32 height = 2;
    uint32 width = 3;
    string encoding = 4;    // "rgb8", "bgr8", "mono8", etc.
    uint32 step = 5;        // Row length in bytes
    bytes data = 6;
}

// LaserScan - 2D laser scan
message LaserScan {
    std.Header header = 1;
}

```

```

    float angle_min = 2;
    float angle_max = 3;
    float angle_increment = 4;
    float time_increment = 5;
    float scan_time = 6;
    float range_min = 7;
    float range_max = 8;
    repeated float ranges = 9;
    repeated float intensities = 10;
}

// Range - Single distance measurement
message Range {
    std.Header header = 1;
    uint32 radiation_type = 2;    // ULTRASOUND=0, INFRARED=1
    float field_of_view = 3;
    float min_range = 4;
    float max_range = 5;
    float range = 6;
}

// NavSatFix - GPS position
message NavSatFix {
    std.Header header = 1;

    int32 status = 2;            // STATUS_NO_FIX=-1, FIX=0, SBAS=1, GBAS=2
    uint32 service = 3;          // SERVICE_GPS=1, GLONASS=2, ...

    double latitude = 4;
    double longitude = 5;
    double altitude = 6;

    repeated double position_covariance = 7;
    uint32 position_covariance_type = 8;
}

// BatteryState - Power source status
message BatteryState {
    std.Header header = 1;
    float voltage = 2;
    float current = 3;
    float charge = 4;
    float capacity = 5;
    float design_capacity = 6;
    float percentage = 7;
    uint32 power_supply_status = 8;
    uint32 power_supply_health = 9;
    uint32 power_supply_technology = 10;
    bool present = 11;
}

```

Timestamps and Headers

Every sensor message includes a Header:

```

message Header {
    Timestamp stamp = 1;
    string frame_id = 2;
    uint32 seq = 3;
}

```

```
message Timestamp {
    int64 seconds = 1;
    int32 nanos = 2;
}
```

stamp: When the measurement was taken (not when it was published) **frame_id**: Coordinate frame reference (e.g., "imu_link", "camera_optical") **seq**: Sequence number for ordering and gap detection

Usage in Go:

```
import "google.golang.org/protobuf/types/known/timestamppb"

msg := &sensor.Imu{
    Header: &std.Header{
        Stamp:    timestamppb.Now(),
        FrameId:  "imu_link",
        Seq:      atomic.AddUint32(&seq, 1),
    },
    LinearAcceleration: &geometry.Vector3{
        X: accel.X,
        Y: accel.Y,
        Z: accel.Z,
    },
    // ...
}
```

Covariance Matrices for Uncertainty

Sensor data is uncertain. Covariance matrices express this uncertainty:

```
// 3x3 covariance matrix as 9 elements, row-major
// [0 1 2]
// [3 4 5]
// [6 7 8]

msg.OrientationCovariance = []float64{
    0.01, 0,    0,    // Roll variance and correlations
    0,    0.01, 0,    // Pitch variance and correlations
    0,    0,    0.02, // Yaw variance and correlations
}
```

Diagonal elements: Variance in each dimension **Off-diagonal elements**: Correlation between dimensions

For uncorrelated sensors, use a diagonal matrix. For unknown covariance, use -1 in the first element as a flag.

Cross-reference: See Chapter 3 for how sensor data flows over NATS.

4.4 Fake Sensors for Testing

Every sensor needs a fake—a test double that simulates sensor behavior without real hardware. Fakes are essential for:

- **Unit testing**: Test logic without hardware dependencies
- **Simulation**: Run the full system on a development laptop
- **CI/CD**: Automated tests in environments without robots
- **Debugging**: Reproduce specific scenarios on demand

Why Fake Implementations Matter

Consider testing a temperature monitoring system:

```
func TestOverheatDetection(t *testing.T) {
    // With a real sensor, you can't control the temperature
    sensor := realSensor.New()
    // How do you trigger the overheat condition?

    // With a fake, you have complete control
    fake := fake.New()
    fake.SetTemperature(95.0) // Simulate overheating

    monitor := NewMonitor(fake)
    status := monitor.Check()

    assert.True(t, status.Overheating)
}
```

Without fakes, you'd need:

- Real hardware connected
- A way to actually heat the sensor
- Tests that take minutes instead of milliseconds
- Flaky results from environmental variation

Fake Implementation Pattern

A good fake implements the same interface as the real sensor plus control methods:

```
// fake/fake.go
package fake

import (
    "context"
    "sync"

    "github.com/gorai/gorai/pkg/resource"
)

// TemperatureSensor is a fake temperature sensor for testing.
type TemperatureSensor struct {
    name      resource.Name
    mu        sync.RWMutex
    temperature float64
    zone      string
    shouldError bool
    errorMsg  string
}

// New creates a new fake temperature sensor.
func New() *TemperatureSensor {
    return &TemperatureSensor{
        name:      resource.NewComponentName("test", "sensor", "fake_temp"),
        temperature: 42.0,
        zone:      "fake_zone",
    }
}

// Implement the Sensor interface
func (f *TemperatureSensor) Name() resource.Name {
    return f.name
}

func (f *TemperatureSensor) Readings(ctx context.Context) (map[string]any, error) {
```

```

    f.mu.RLock()
    defer f.mu.RUnlock()

    if f.shouldError {
        return nil, fmt.Errorf(f.errorMsg)
    }

    return map[string]any{
        "temperature_celsius":    f.temperature,
        "temperature_fahrenheit": f.temperature*9/5 + 32,
        "zone":                   f.zone,
    }, nil
}

func (f *TemperatureSensor) Reconfigure(ctx context.Context, deps resource.Dependencies, conf
resource.Config) error {
    return nil
}

func (f *TemperatureSensor) DoCommand(ctx context.Context, cmd map[string]any) (map[string]any,
error) {
    return nil, nil
}

func (f *TemperatureSensor) Close(ctx context.Context) error {
    return nil
}

// Control methods for testing

// SetTemperature sets the temperature the fake sensor will return.
func (f *TemperatureSensor) SetTemperature(celsius float64) {
    f.mu.Lock()
    defer f.mu.Unlock()
    f.temperature = celsius
}

// SetError makes the sensor return an error on next reading.
func (f *TemperatureSensor) SetError(msg string) {
    f.mu.Lock()
    defer f.mu.Unlock()
    f.shouldError = true
    f.errorMsg = msg
}

// ClearError stops the sensor from returning errors.
func (f *TemperatureSensor) ClearError() {
    f.mu.Lock()
    defer f.mu.Unlock()
    f.shouldError = false
}

// GetReadCount returns how many times Readings was called (for verification).
func (f *TemperatureSensor) GetReadCount() int {
    f.mu.RLock()
    defer f.mu.RUnlock()
    return f.readCount
}

```

Configurable Behavior

Fakes should support various test scenarios:

```
// Configure for specific test scenarios
fake := fake.New()
fake.SetTemperature(25.0) // Normal temperature
fake.SetZone("cpu_thermal")
fake.SetUpdateRate(100 * time.Millisecond)

// Simulate sensor failure
fake.SetError("I2C read timeout")

// Simulate noisy readings
fake.SetNoise(0.5) // ±0.5°C random variation

// Simulate gradual change
fake.SetDrift(0.1) // +0.1°C per reading
```

Error Injection

Testing error handling requires controlled failures:

```
func TestSensorRecovery(t *testing.T) {
    fake := fake.New()
    monitor := NewMonitor(fake)

    // Normal operation
    fake.SetTemperature(25.0)
    status, err := monitor.Check()
    assert.NoError(t, err)

    // Simulate sensor failure
    fake.SetError("hardware disconnected")
    status, err = monitor.Check()
    assert.Error(t, err)
    assert.True(t, status.SensorFailed)

    // Simulate recovery
    fake.ClearError()
    fake.SetTemperature(26.0)
    status, err = monitor.Check()
    assert.NoError(t, err)
    assert.False(t, status.SensorFailed)
}
```

Testing Fakes Themselves

Fakes should have their own tests:

```
// fake/fake_test.go
func TestFakeSensor_ReturnsConfiguredTemperature(t *testing.T) {
    fake := New()
    fake.SetTemperature(50.0)

    readings, err := fake.Readings(context.Background())
    require.NoError(t, err)

    assert.Equal(t, 50.0, readings["temperature_celsius"])
    assert.Equal(t, 122.0, readings["temperature_fahrenheit"])
}
```

```
func TestFakeSensor_ReturnsErrorWhenConfigured(t *testing.T) {  
    fake := New()  
    fake.SetError("test error")  
  
    _, err := fake.Readings(context.Background())  
    assert.Error(t, err)  
    assert.Contains(t, err.Error(), "test error")  
}
```

Cross-reference: See Chapter 11 for comprehensive testing strategies.

Chapter 5: Components - Actuators

Actuators transform electrical signals into physical motion. They're how your robot interacts with the world—wheels that turn, arms that reach, grippers that grasp.

5.1 The Actuator Interface

All actuators share a common base interface from `pkg/resource/resource.go` :

```
type Actuator interface {
    Resource

    // IsMoving returns true if the actuator is currently in motion.
    IsMoving(ctx context.Context) (bool, error)

    // Stop halts all motion immediately.
    Stop(ctx context.Context) error
}
```

The Actuator interface adds two critical safety methods:

IsMoving(): Query motion state. Essential for:

- Waiting for motion to complete before next action
- Safety interlocks (don't close gripper while arm is moving)
- State machine transitions

Stop(): Emergency halt. Every actuator must implement immediate stop:

- Called during emergency shutdowns
- Triggered by safety systems
- Available for manual intervention

Safety-First Design

Actuators can cause harm. Gorai's actuator design prioritizes safety:

```
func (m *Motor) SetPower(ctx context.Context, power float64) error {
    // Clamp power to safe limits
    if power > m.maxPower {
        power = m.maxPower
    }
    if power < -m.maxPower {
        power = -m.maxPower
    }

    // Check safety conditions
    if m.overTemp {
        return fmt.Errorf("motor overtemperature, refusing to run")
    }

    return m.driver.SetPower(power)
}
```

Emergency Stop Patterns

Implement robust stop behavior:

```
func (m *Motor) Stop(ctx context.Context) error {
    // Stop is best-effort-try multiple approaches
```

```

var errs []error

// Try graceful stop first
if err := m.driver.SetPower(0); err != nil {
    errs = append(errs, err)
}

// Engage brake if available
if m.hasBrake {
    if err := m.driver.EngageBrake(); err != nil {
        errs = append(errs, err)
    }
}

// Cut power as last resort
if len(errs) > 0 {
    m.driver.CutPower()
}

m.mu.Lock()
m.moving = false
m.mu.Unlock()

if len(errs) > 0 {
    return fmt.Errorf("stop encountered errors: %v", errs)
}
return nil
}

```

5.2 Motor Interface

Motors are the most common actuators. The Motor interface from `component/motor/motor.go` provides comprehensive control:

```

type Motor interface {
    component.Actuator

    // SetPower sets the motor power from -1.0 (full reverse) to 1.0 (full forward).
    SetPower(ctx context.Context, power float64) error

    // SetVelocity sets target velocity (rad/s or m/s depending on motor type).
    SetVelocity(ctx context.Context, velocity float64) error

    // GoTo moves the motor to the specified absolute position at the given velocity.
    GoTo(ctx context.Context, position, velocity float64) error

    // GoFor moves the motor for the specified number of revolutions at the given RPM.
    // Positive RPM moves forward, negative moves backward.
    GoFor(ctx context.Context, rpm, revolutions float64) error

    // GetPosition returns the current position (in revolutions from zero).
    GetPosition(ctx context.Context) (float64, error)

    // GetVelocity returns the current velocity.
    GetVelocity(ctx context.Context) (float64, error)

    // ResetZeroPosition sets the current position as the zero position.
    ResetZeroPosition(ctx context.Context, offset float64) error

    // IsPowered returns whether the motor is currently receiving power

```

```

// and the current power level.
IsPowered(ctx context.Context) (bool, float64, error)

// Properties returns the motor's properties.
Properties(ctx context.Context) (Properties, error)
}

```

Power Control: SetPower

The simplest control mode—direct power/duty cycle:

```

// Full forward
motor.SetPower(ctx, 1.0)

// Half speed reverse
motor.SetPower(ctx, -0.5)

// Stop (coast)
motor.SetPower(ctx, 0)

```

Power is normalized to [-1.0, 1.0]:

- +1.0 = maximum forward voltage/PWM
- -1.0 = maximum reverse voltage/PWM
- 0 = no power (motor coasts)

Use cases: Direct joystick control, simple behaviors where speed precision doesn't matter.

Velocity Control: SetVelocity

Closed-loop speed control using encoder feedback:

```

// Rotate at 10 rad/s
motor.SetVelocity(ctx, 10.0)

// For linear actuators, this might be m/s
motor.SetVelocity(ctx, 0.5) // 0.5 m/s

```

Velocity control requires:

- Encoder feedback
- PID controller (typically in driver or motor controller)
- Proper tuning

Position Control: GoTo and GoFor

Move to absolute or relative positions:

```

// Move to position 5.0 revolutions at velocity 2.0 rad/s
motor.GoTo(ctx, 5.0, 2.0)

// Move forward 2 revolutions at 60 RPM
motor.GoFor(ctx, 60, 2.0)

// Move backward 1 revolution at 30 RPM
motor.GoFor(ctx, -30, 1.0)

```

GoTo: Absolute position (requires knowing current position) **GoFor:** Relative motion (useful for "move X distance" commands)

Both methods are typically blocking or provide completion feedback.

State Queries

```
// Current position in revolutions
pos, _ := motor.GetPosition(ctx)

// Current velocity
vel, _ := motor.GetVelocity(ctx)

// Is motor powered and at what level?
powered, level, _ := motor.IsPowered(ctx)
```

Motor Properties

Motors report their capabilities:

```
type Properties struct {
    // PositionReporting indicates whether the motor can report its position.
    PositionReporting bool

    // VelocityReporting indicates whether the motor can report its velocity.
    VelocityReporting bool

    // SupportsGoTo indicates whether the motor supports absolute positioning.
    SupportsGoTo bool
}
```

Check properties before using advanced features:

```
props, _ := motor.Properties(ctx)

if !props.PositionReporting {
    // Can't use GoTo without position feedback
    // Fall back to timed motion or power control
}
```

5.3 Motor Types

Different motor technologies have different characteristics. Understanding them helps you choose the right motor and write appropriate control code.

DC Motors

Simple, cheap, high-power motors controlled by voltage/PWM:

Characteristics:

- Continuous rotation
- Speed proportional to voltage (roughly)
- Torque proportional to current
- Reversible by polarity swap
- Require H-bridge driver for bidirectional control

Gorai implementation considerations:

```
type DCMotor struct {
    pwmPin    gpio.PWMPin
    dir1Pin   gpio.Pin
    dir2Pin   gpio.Pin
    encoder   *Encoder // Optional
}
```

```

}

func (m *DCMotor) SetPower(ctx context.Context, power float64) error {
    // Set direction
    if power >= 0 {
        m.dir1Pin.High()
        m.dir2Pin.Low()
    } else {
        m.dir1Pin.Low()
        m.dir2Pin.High()
        power = -power
    }

    // Set PWM duty cycle
    return m.pwmPin.SetDutyCycle(uint32(power * 65535))
}

```

Typical applications: Wheel drive, simple conveyors, fans

Stepper Motors

Precise positioning without encoders:

Characteristics:

- Move in discrete steps (typically 200 steps/revolution)
- Microstepping for finer resolution (1600, 3200+ steps/rev)
- Position known by counting steps (open-loop)
- Lower top speed than DC motors
- Holding torque at rest

Gorai implementation considerations:

```

type StepperMotor struct {
    stepPin  gpio.Pin
    dirPin   gpio.Pin
    stepsPerRev int
    position int64
}

func (m *StepperMotor) GoFor(ctx context.Context, rpm, revolutions float64) error {
    steps := int(revolutions * float64(m.stepsPerRev))
    stepDelay := time.Duration(60e9 / (rpm * float64(m.stepsPerRev)))

    // Set direction
    if steps < 0 {
        m.dirPin.Low()
        steps = -steps
    } else {
        m.dirPin.High()
    }

    for i := 0; i < steps; i++ {
        select {
        case <-ctx.Done():
            return ctx.Err()
        default:
            m.stepPin.High()
            time.Sleep(stepDelay / 2)
            m.stepPin.Low()
            time.Sleep(stepDelay / 2)
            m.position++
        }
    }
}

```

```
    }  
    }  
    return nil  
}
```

Typical applications: 3D printers, CNC machines, camera gimbals

Servo Motors

Position-controlled motors with built-in feedback:

Characteristics:

- Integrated controller, encoder, and motor
- Position or velocity commands via protocol (PWM, serial, CAN)
- High precision and repeatability
- More expensive than DC/stepper

Types:

- **RC Servos:** PWM control, limited rotation (180° typical)
- **Smart Servos:** Serial protocol (Dynamixel, Herkulex), full rotation
- **Industrial Servos:** CAN/EtherCAT, high power

Gorai implementation for RC servo:

```
type RCServo struct {  
    pwmPin gpio.PWMPin  
    minPulse time.Duration // 1ms typical  
    maxPulse time.Duration // 2ms typical  
}  
  
func (s *RCServo) SetAngle(ctx context.Context, degrees float64) error {  
    // Map degrees to pulse width  
    pulse := s.minPulse + time.Duration(  
        (degrees/180.0)*float64(s.maxPulse-s.minPulse))  
  
    return s.pwmPin.SetPulseWidth(pulse)  
}
```

Brushless Motors (BLDC)

High-performance motors requiring electronic commutation:

Characteristics:

- Higher efficiency than brushed DC
- Higher speed capability
- Longer lifespan (no brushes to wear)
- Require ESC (Electronic Speed Controller) or FOC driver

Control modes:

- **PWM/Throttle:** Like DC motors, proportional control
- **Closed-loop:** Encoder feedback for precise velocity/position

Typical applications: Drones, high-performance wheels, industrial robots

Choosing Motor Type

Requirement	Best Choice
Cheap, simple drive	DC motor
Precise positioning, low speed	Stepper

Servo-like behavior, budget	DC + encoder
High precision positioning	Smart servo
High speed, efficiency	Brushless
Simple angle control	RC servo

5.4 Control Patterns

Motor control ranges from simple power commands to sophisticated closed-loop control. Understanding these patterns helps you implement appropriate control for your application.

Open-Loop vs Closed-Loop

Open-loop control: Command without feedback

```
// "Run at 50% power"
motor.SetPower(ctx, 0.5)
// No verification that actual speed matches intent
```

Pros: Simple, no sensors required Cons: No compensation for load changes, friction, battery voltage

Closed-loop control: Command with feedback

```
// "Run at 100 RPM" - controller adjusts power to maintain speed
motor.SetVelocity(ctx, 100)
// Encoder feedback allows continuous adjustment
```

Pros: Precise, handles disturbances Cons: Requires sensors, tuning, more complex

PID Control Basics

PID (Proportional-Integral-Derivative) is the workhorse of motor control:

```
type PIDController struct {
    Kp, Ki, Kd float64 // Gains
    integral   float64 // Accumulated error
    lastError  float64 // Previous error
    lastTime   time.Time
}

func (p *PIDController) Compute(setpoint, measurement float64) float64 {
    now := time.Now()
    dt := now.Sub(p.lastTime).Seconds()
    p.lastTime = now

    error := setpoint - measurement

    // Proportional term
    proportional := p.Kp * error

    // Integral term (accumulated error)
    p.integral += error * dt
    integral := p.Ki * p.integral

    // Derivative term (rate of change)
    derivative := p.Kd * (error - p.lastError) / dt
    p.lastError = error
```

```
    return proportional + integral + derivative
}
```

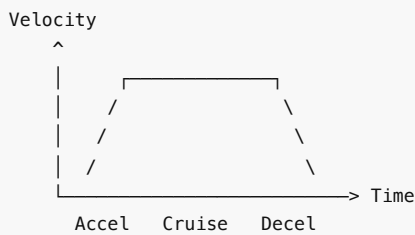
Tuning PID:

1. Start with $K_i = K_d = 0$
2. Increase K_p until oscillation, then halve it
3. Add K_i to eliminate steady-state error
4. Add K_d to reduce overshoot

Velocity Profiles

For smooth motion, use velocity profiles instead of instantaneous commands:

Trapezoidal profile: Accelerate, cruise, decelerate



```
type TrapezoidalProfile struct {
    maxVel    float64
    maxAccel  float64
}

func (t *TrapezoidalProfile) Generate(distance float64) []VelocityPoint {
    // Calculate times for each phase
    accelTime := t.maxVel / t.maxAccel
    accelDist := 0.5 * t.maxAccel * accelTime * accelTime

    if 2*accelDist > distance {
        // Triangle profile - never reach max velocity
        accelTime = math.Sqrt(distance / t.maxAccel)
        return t.triangleProfile(distance, accelTime)
    }

    cruiseDist := distance - 2*accelDist
    cruiseTime := cruiseDist / t.maxVel

    return t.trapezoidProfile(accelTime, cruiseTime)
}
```

S-curve profile: Smoother acceleration (jerk-limited)

- Better for delicate operations
- Reduces mechanical stress

Position Feedback

For position control, combine velocity control with position feedback:

```
type PositionController struct {
    velocityPID *PIDController
    positionGain float64
}

func (c *PositionController) GoTo(target float64) {
```

```

for {
    current := c.motor.GetPosition()
    error := target - current

    if math.Abs(error) < c.tolerance {
        break // Close enough
    }

    // Position error -> velocity setpoint
    velocitySetpoint := c.positionGain * error

    // Clamp to max velocity
    velocitySetpoint = clamp(velocitySetpoint, -c.maxVel, c.maxVel)

    // Velocity PID computes power
    currentVel := c.motor.GetVelocity()
    power := c.velocityPID.Compute(velocitySetpoint, currentVel)

    c.motor.SetPower(power)
    time.Sleep(10 * time.Millisecond) // Control loop rate
}
}

```

When Control Happens

Control loops can run at different levels:

Motor controller hardware: Many motor drivers have built-in PID

- Fastest response (microseconds)
- Configure via registers/protocol

TinyGo on microcontroller: Real-time control in Go

- Fast response (100µs - 1ms)
- Custom control algorithms

Gorai on Linux: Higher-level control

- Slower response (1-10ms)
- Suitable for position control, trajectory tracking
- Not suitable for commutation, current control

Choose the right level for your control loop requirements.

5.5 Servo Interface

Servos are position-controlled actuators, typically for angular positioning. They differ from motors in their control paradigm: you command angles, not speeds.

```

type Servo interface {
    component.Actuator

    // Move moves the servo to the specified angle (degrees).
    Move(ctx context.Context, angleDeg float64) error

    // GetPosition returns the current angle (degrees).
    GetPosition(ctx context.Context) (float64, error)
}

```

Angle-Based Positioning

```
// Center position
servo.Move(ctx, 90)

// Full left
servo.Move(ctx, 0)

// Full right
servo.Move(ctx, 180)
```

Angle ranges vary by servo:

- Standard RC servos: 0-180°
- Extended range: 0-270° or 0-360°
- Continuous rotation: Angle maps to speed/direction

PWM Control

RC servos use PWM pulse width for position:

```
type RCServo struct {
    pwm      gpio.PWMPin
    minAngle float64    // Typically 0
    maxAngle float64    // Typically 180
    minPulse time.Duration // Typically 500µs - 1ms
    maxPulse time.Duration // Typically 2ms - 2.5ms
}

func (s *RCServo) Move(ctx context.Context, angleDeg float64) error {
    // Clamp to valid range
    if angleDeg < s.minAngle {
        angleDeg = s.minAngle
    }
    if angleDeg > s.maxAngle {
        angleDeg = s.maxAngle
    }

    // Map angle to pulse width
    fraction := (angleDeg - s.minAngle) / (s.maxAngle - s.minAngle)
    pulse := s.minPulse + time.Duration(fraction*float64(s.maxPulse-s.minPulse))

    return s.pwm.SetPulseWidth(pulse)
}
```

Calibration: Real servos rarely match spec exactly:

```
// Measure actual endpoints and center
servo.minPulse = 600 * time.Microsecond // Actual minimum
servo.maxPulse = 2400 * time.Microsecond // Actual maximum
```

Multi-Servo Coordination

Robots often have multiple servos that must move together:

```
type ServoGroup struct {
    servos map[string]Servo
}

func (g *ServoGroup) MoveAll(ctx context.Context, positions map[string]float64) error {
    // Start all moves simultaneously
    var wg sync.WaitGroup
```

```

errs := make(chan error, len(positions))

for name, angle := range positions {
    servo := g.servos[name]
    wg.Add(1)
    go func(s Servo, a float64) {
        defer wg.Done()
        if err := s.Move(ctx, a); err != nil {
            errs <- err
        }
    }(servo, angle)
}

wg.Wait()
close(errs)

// Collect errors
var allErrs []error
for err := range errs {
    allErrs = append(allErrs, err)
}

if len(allErrs) > 0 {
    return fmt.Errorf("servo errors: %v", allErrs)
}
return nil
}

```

Synchronized motion: For smooth coordinated movement:

```

func (g *ServoGroup) Interpolate(from, to map[string]float64, duration time.Duration) {
    steps := int(duration / (20 * time.Millisecond)) // 50Hz update

    for i := 0; i <= steps; i++ {
        t := float64(i) / float64(steps)
        positions := make(map[string]float64)

        for name := range to {
            // Linear interpolation
            positions[name] = from[name] + t*(to[name]-from[name])
        }

        g.MoveAll(ctx, positions)
        time.Sleep(20 * time.Millisecond)
    }
}

```

5.6 Gripper Interface

Grippers are end effectors for grasping objects:

```

type Gripper interface {
    component.Actuator

    // Open fully opens the gripper.
    Open(ctx context.Context) error

    // Close fully closes the gripper.
    Close(ctx context.Context) error
}

```

```

// Grab closes until resistance is felt (object grasped).
Grab(ctx context.Context) (bool, error)

// IsOpen returns true if gripper is fully open.
IsOpen(ctx context.Context) (bool, error)
}

```

Basic Open/Close

```

// Open gripper before approaching object
gripper.Open(ctx)

// Close to grasp
gripper.Close(ctx)

```

Force Sensing

Advanced grippers detect when they've grasped something:

```

func (g *Gripper) Grab(ctx context.Context) (bool, error) {
    // Start closing
    g.motor.SetPower(ctx, -0.5) // Close direction

    timeout := time.After(5 * time.Second)
    ticker := time.NewTicker(10 * time.Millisecond)

    for {
        select {
        case <-ctx.Done():
            g.motor.Stop(ctx)
            return false, ctx.Err()

        case <-timeout:
            g.motor.Stop(ctx)
            return false, fmt.Errorf("grab timeout")

        case <-ticker.C:
            // Check for resistance (current spike, stall)
            current := g.motor.GetCurrent()
            if current > g.grabThreshold {
                g.motor.Stop(ctx)
                return true, nil // Object grasped
            }

            // Check if fully closed without object
            if g.atClosedLimit() {
                g.motor.Stop(ctx)
                return false, nil // No object
            }
        }
    }
}

```

Grasp Detection

Beyond simple force sensing:

```

type GraspStatus struct {
    Grasping bool
}

```

```

    ObjectWidth float64 // Estimated from encoder position
    GripForce   float64 // From current or force sensor
}

func (g *Gripper) GetGraspStatus(ctx context.Context) (GraspStatus, error) {
    position := g.encoder.GetPosition()
    current := g.motor.GetCurrent()

    return GraspStatus{
        Grasping:    current > g.holdThreshold,
        ObjectWidth: g.positionToWidth(position),
        GripForce:   g.currentToForce(current),
    }, nil
}

```

5.7 Base Interface (Mobile Robots)

The Base interface abstracts mobile robot locomotion:

```

type Base interface {
    component.Actuator

    // SetVelocity sets linear and angular velocity.
    SetVelocity(ctx context.Context, linear, angular float64) error

    // MoveStraight moves the robot forward by the specified distance.
    MoveStraight(ctx context.Context, distanceMm int, velocity float64) error

    // Spin rotates the robot in place by the specified angle.
    Spin(ctx context.Context, angleDeg, velocity float64) error

    // GetVelocities returns current linear and angular velocities.
    GetVelocities(ctx context.Context) (linear, angular float64, err error)
}

```

Drive Types

Differential Drive: Two independently controlled wheels

```

// Convert linear/angular to wheel velocities
func (b *DiffDrive) setWheelVelocities(linear, angular float64) {
    // v_left = linear - angular * (wheelbase / 2)
    // v_right = linear + angular * (wheelbase / 2)
    vLeft := linear - angular*b.wheelbase/2
    vRight := linear + angular*b.wheelbase/2

    b.leftMotor.SetVelocity(ctx, vLeft/b.wheelRadius)
    b.rightMotor.SetVelocity(ctx, vRight/b.wheelRadius)
}

```

Mecanum Wheels: Omnidirectional movement

```

// Four-wheel mecanum kinematics
func (b *Mecanum) setWheelVelocities(vx, vy, angular float64) {
    // Each wheel contributes differently to motion
    fl := vx - vy - angular*(b.lx+b.ly) // Front left
    fr := vx + vy + angular*(b.lx+b.ly) // Front right
    rl := vx + vy - angular*(b.lx+b.ly) // Rear left
    rr := vx - vy + angular*(b.lx+b.ly) // Rear right
}

```

```

    b.motors["fl"].SetVelocity(ctx, fl)
    b.motors["fr"].SetVelocity(ctx, fr)
    b.motors["rl"].SetVelocity(ctx, rl)
    b.motors["rr"].SetVelocity(ctx, rr)
}

```

Ackermann Steering: Car-like steering

```

func (b *Ackermann) SetVelocity(ctx context.Context, linear, angular float64) error {
    // Convert angular velocity to steering angle
    // Using bicycle model: angular = linear * tan(steering) / wheelbase
    if linear != 0 {
        steering := math.Atan(angular * b.wheelbase / linear)
        b.steeringServo.Move(ctx, steeringToDegrees(steering))
    }

    b.driveMotor.SetVelocity(ctx, linear)
    return nil
}

```

Velocity Commands

Typically expressed as Twist (linear + angular):

```

message Twist {
    Vector3 linear = 1; // Linear velocity (x=forward, y=left, z=up)
    Vector3 angular = 2; // Angular velocity (roll, pitch, yaw)
}

```

For ground robots, typically only use:

- linear.x: Forward/backward
- angular.z: Turn left/right

5.8 Arm Interface (Manipulators)

Robotic arms require sophisticated interfaces:

```

type Arm interface {
    component.Actuator

    // EndPosition returns the current end effector pose.
    EndPosition(ctx context.Context) (*spatialmath.Pose, error)

    // MoveToPosition moves the end effector to the target pose.
    MoveToPosition(ctx context.Context, pose *spatialmath.Pose) error

    // JointPositions returns current joint angles.
    JointPositions(ctx context.Context) ([]float64, error)

    // MoveToJointPositions sets joint angles directly.
    MoveToJointPositions(ctx context.Context, positions []float64) error
}

```

Joint Space vs Task Space

Joint space: Direct control of each joint angle

```
// Move each joint to specific angle
positions := []float64{0, -45, 90, 0, 45, 0} // degrees
arm.MoveToJointPositions(ctx, positions)
```

- Direct, predictable
- Requires knowing valid configurations
- Good for predefined poses

Task space: Control end effector position/orientation

```
// Move end effector to position
pose := spatialmath.NewPoseFromPoint(r3.Vector{X: 0.3, Y: 0.1, Z: 0.4})
arm.MoveToPosition(ctx, pose)
```

- More intuitive for applications
- Requires inverse kinematics
- May have multiple solutions or none

Forward/Inverse Kinematics Concepts

Forward kinematics: Joints → End effector position

Given: Joint angles [θ₁, θ₂, θ₃, ...]
Find: End effector pose (x, y, z, rotation)

Always has a unique solution.

Inverse kinematics: End effector position → Joints

Given: Desired end effector pose
Find: Joint angles to achieve it

May have:

- Multiple solutions (elbow up vs elbow down)
- No solution (target unreachable)
- Singularities (infinite solutions along an axis)

Trajectory Planning

Moving from A to B requires planning:

```
type Trajectory struct {
    Points []TrajectoryPoint
}

type TrajectoryPoint struct {
    Time      time.Duration
    Joints    []float64
    Velocity  []float64
}

func (a *Arm) ExecuteTrajectory(ctx context.Context, traj *Trajectory) error {
    start := time.Now()

    for _, point := range traj.Points {
        // Wait for point time
        elapsed := time.Since(start)
        if point.Time > elapsed {
            time.Sleep(point.Time - elapsed)
        }
    }
}
```

```
    // Move to point
    if err := a.MoveToJointPositions(ctx, point.Joints); err != nil {
        return err
    }
}
return nil
}
```

Trajectory types:

- Point-to-point: Direct joint interpolation
- Cartesian: Straight line in task space
- Spline: Smooth curves through waypoints

With sensors and actuators covered, Chapter 6 explores vision—the intersection of sensors and AI that enables robots to perceive and understand their environment.

Chapter 6: Components - Vision

Vision gives robots the ability to perceive and understand their environment. From simple obstacle detection to complex object recognition, cameras are increasingly central to robotic systems.

6.1 The Camera Interface

Cameras bridge the physical world and digital processing:

```
type Camera interface {
    component.Component

    // Image captures and returns the current frame.
    Image(ctx context.Context) (image.Image, error)

    // Stream returns a channel of continuous frames.
    Stream(ctx context.Context) (<-chan image.Image, error)

    // Properties returns camera intrinsics and capabilities.
    Properties(ctx context.Context) (Properties, error)
}
```

Image Capture

Single frame capture for on-demand processing:

```
camera, _ := camera.New(node, cameraConfig)

// Capture a single frame
img, err := camera.Image(ctx)
if err != nil {
    log.Printf("capture failed: %v", err)
    return
}

// img is a Go standard library image.Image
bounds := img.Bounds()
log.Printf("Captured %dx%d image", bounds.Dx(), bounds.Dy())
```

Image Formats

Go's `image.Image` interface supports multiple formats:

```
switch img := img.(type) {
case *image.RGBA:
    // 8-bit RGBA
    pixel := img.RGBAAt(x, y)

case *image.Gray:
    // 8-bit grayscale
    pixel := img.GrayAt(x, y)

case *image.YCbCr:
    // YUV format (common from cameras)
    y := img.Y[img.YOffset(x, y)]
}
```

Resolution and Frame Rate

Camera properties describe capabilities:

```
type Properties struct {
    Width      int
    Height     int
    FrameRate  float64
    PixelFormat string // "rgb8", "bgr8", "yuv422", etc.

    // Intrinsic parameters for 3D projection
    FocalLengthX float64
    FocalLengthY float64
    PrincipalX   float64
    PrincipalY   float64
    DistortionK  []float64 // Radial distortion
    DistortionP  []float64 // Tangential distortion
}
```

Intrinsic Parameters

Camera intrinsics describe how 3D points project to 2D pixels:

Camera Intrinsic Matrix (K)

$$\begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}$$

f_x, f_y : Focal lengths (pixels)
 c_x, c_y : Principal point

Use intrinsics to:

- Convert pixel coordinates to rays in 3D
- Undistort images
- Compute 3D positions from stereo or depth

```
// Project 3D point to pixel
func (c *Camera) Project(point r3.Vector) (x, y float64) {
    props := c.Properties(ctx)
    x = props.FocalLengthX*point.X/point.Z + props.PrincipalX
    y = props.FocalLengthY*point.Y/point.Z + props.PrincipalY
    return
}
```

6.2 Camera Types

Different camera technologies serve different needs.

USB Cameras

The simplest option—plug and play via V4L2 (Video4Linux2):

```
type USBCamera struct {
    device string // "/dev/video0"
    width  int
    height int
}
```

```

    frameRate int
    v4l2Device *v4l2.Device
}

func NewUSBCamera(device string, width, height, fps int) (*USBCamera, error) {
    dev, err := v4l2.Open(device)
    if err != nil {
        return nil, err
    }

    // Set format
    err = dev.SetFormat(v4l2.PixelFormatMJPEG, width, height)
    if err != nil {
        return nil, err
    }

    // Set frame rate
    err = dev.SetFrameRate(fps)
    if err != nil {
        return nil, err
    }

    return &USBCamera{
        device:    device,
        width:     width,
        height:    height,
        frameRate: fps,
        v4l2Device: dev,
    }, nil
}

```

Common USB cameras:

- Logitech C920/C930: Good quality, wide compatibility
- ELP cameras: Inexpensive, various form factors
- Intel RealSense: RGB-D cameras

CSI Cameras (Raspberry Pi)

Camera Serial Interface provides higher bandwidth:

```

// Raspberry Pi camera via libcamera
type CSICamera struct {
    width    int
    height   int
    camera    *libcamera.Camera
}

func NewCSICamera(width, height int) (*CSICamera, error) {
    cam, err := libcamera.Open()
    if err != nil {
        return nil, err
    }

    config := libcamera.VideoConfiguration{
        Width:    width,
        Height:   height,
        PixelFmt: libcamera.RGB888,
        BufferCnt: 4,
    }

    if err := cam.Configure(config); err != nil {

```

```

        return nil, err
    }

    return &CSICamera{
        width: width,
        height: height,
        camera: cam,
    }, nil
}

```

Advantages of CSI:

- Lower CPU overhead (direct memory access)
- Higher frame rates (60+ fps)
- Lower latency
- GPU acceleration on Pi

Common CSI cameras:

- Raspberry Pi Camera Module v2/v3
- Arducam variety
- IMX477 (HQ Camera)

IP Cameras

Network cameras for remote or distributed sensing:

```

type IPCamera struct {
    url      string
    stream *gocv.VideoCapture
}

func NewIPCamera(rtspURL string) (*IPCamera, error) {
    stream, err := gocv.OpenVideoCapture(rtspURL)
    if err != nil {
        return nil, err
    }

    return &IPCamera{
        url:      rtspURL,
        stream: stream,
    }, nil
}

func (c *IPCamera) Image(ctx context.Context) (image.Image, error) {
    mat := gocv.NewMat()
    defer mat.Close()

    if ok := c.stream.Read(&mat); !ok {
        return nil, fmt.Errorf("failed to read frame")
    }

    return mat.ToImage()
}

```

Use cases:

- Remote monitoring
- Multi-camera systems
- PTZ (pan-tilt-zoom) cameras

Depth Cameras (RGB-D)

Cameras that provide depth information:

```
type DepthCamera interface {
    Camera

    // DepthImage returns per-pixel depth values.
    DepthImage(ctx context.Context) (*DepthMap, error)

    // PointCloud returns 3D point cloud.
    PointCloud(ctx context.Context) (*PointCloud, error)
}

type DepthMap struct {
    Width  int
    Height int
    Data   []float32 // Depth in meters per pixel
}
```

Technologies:

- **Stereo**: Two cameras, compute depth from disparity
- **Structured light**: Project pattern, measure deformation
- **ToF (Time of Flight)**: Measure light round-trip time
- **LiDAR**: Scanning laser measurement

Common depth cameras:

- Intel RealSense D415/D435/D455
- Azure Kinect
- Orbbec Astra
- OAK-D (with NPU)

6.3 Image Data Flow

Images are large—a 1920x1080 RGB image is 6MB uncompressed. Efficient handling matters.

Protocol Buffer Representation

From `vision.proto` :

```
message Image {
    std.Header header = 1;
    uint32 height = 2;
    uint32 width = 3;
    string encoding = 4;    // "rgb8", "bgr8", "mono8", "jpeg", etc.
    uint32 step = 5;        // Row length in bytes
    bytes data = 6;
}

message CompressedImage {
    std.Header header = 1;
    string format = 2;      // "jpeg", "png", "h264"
    bytes data = 3;
}
```

Compression Considerations

Raw images for processing:

```
// Full quality for vision algorithms
img, _ := camera.Image(ctx)
```

```
detections := detector.Detect(img)
```

Compressed images for transport:

```
// JPEG for network transmission (10-20x smaller)
var buf bytes.Buffer
jpeg.Encode(&buf, img, &jpeg.Options{Quality: 80})

pub.Publish(ctx, &vision.CompressedImage{
    Format: "jpeg",
    Data:   buf.Bytes(),
})
```

When to compress:

- Sending over network (NATS, remote nodes)
- Logging/recording
- Display/streaming to operators

When to keep raw:

- Local processing pipeline
- Algorithms sensitive to compression artifacts
- Stereo matching, feature detection

Streaming vs On-Demand

On-demand (pull model):

```
// Get frame when needed
for {
    img, _ := camera.Image(ctx)
    processFrame(img)
    time.Sleep(100 * time.Millisecond)
}
```

Pros: Simple, process at your rate Cons: May miss frames, inconsistent timing

Streaming (push model):

```
// Subscribe to continuous frames
stream, _ := camera.Stream(ctx)
for img := range stream {
    processFrame(img)
}
```

Pros: All frames available, consistent timing Cons: Must keep up or drop frames

Frame Rate Management

Cameras produce frames faster than processing can handle:

```
type FrameDropper struct {
    input  <-chan image.Image
    output chan image.Image
    latest image.Image
}

func (d *FrameDropper) Run(ctx context.Context) {
    for {
        select {
```

```

    case <-ctx.Done():
        return

    case img := <-d.input:
        // Always keep the latest
        d.latest = img

    case d.output <- d.latest:
        // Deliver when consumer is ready
        d.latest = nil
    }
}
}

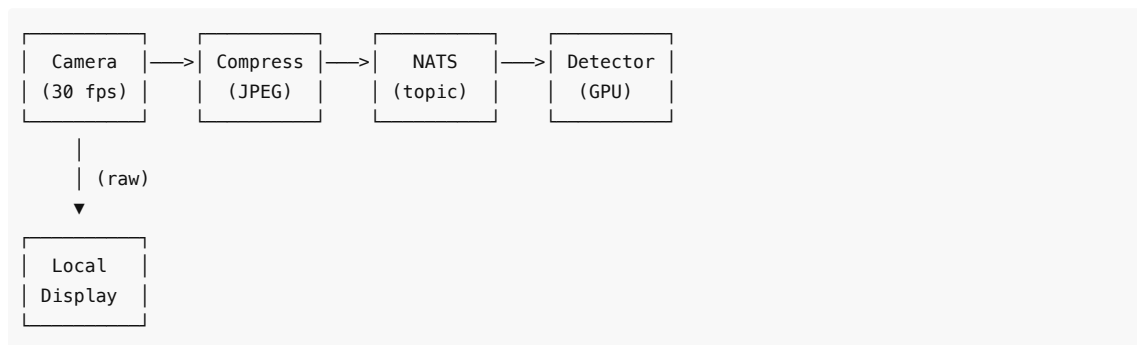
```

This pattern:

- Never blocks the camera
- Always provides the most recent frame
- Drops frames when consumer is slow

Pipeline Architecture

Vision pipelines separate acquisition from processing:



```

// Camera node
func cameraNode(ctx context.Context) {
    camera := setupCamera()
    pub := pub.New[*vision.CompressedImage](node, "gorai.cameras.front.compressed")

    for img := range camera.Stream(ctx) {
        compressed := compress(img)
        pub.Publish(ctx, compressed)
    }
}

// Detector node (on GPU machine)
func detectorNode(ctx context.Context) {
    sub.New[*vision.CompressedImage](node, "gorai.cameras.front.compressed",
        func(msg *vision.CompressedImage) {
            img := decompress(msg)
            detections := model.Detect(img)
            detPub.Publish(ctx, detections)
        })
}

```

6.4 Computer Vision Integration

Go isn't traditionally known for computer vision, but excellent bindings exist.

OpenCV with Go (GoCV)

[GoCV](#) provides comprehensive OpenCV bindings:

```
import "gocv.io/x/gocv"

// Read image
mat := gocv.IMRead("image.jpg", gocv.IMReadColor)
defer mat.Close()

// Convert color space
gray := gocv.NewMat()
defer gray.Close()
gocv.CvtColor(mat, &gray, gocv.ColorBGRTToGray)

// Edge detection
edges := gocv.NewMat()
defer edges.Close()
gocv.Canny(gray, &edges, 50, 100)

// Save result
gocv.IMWrite("edges.jpg", edges)
```

Frame Processing Pipelines

Common vision operations:

```
type VisionPipeline struct {
    camera Camera
    stages []ProcessingStage
}

type ProcessingStage interface {
    Process(img gocv.Mat) gocv.Mat
}

// Preprocessing stage
type Preprocessor struct {
    targetWidth int
    targetHeight int
}

func (p *Preprocessor) Process(img gocv.Mat) gocv.Mat {
    // Resize
    resized := gocv.NewMat()
    gocv.Resize(img, &resized, image.Point{p.targetWidth, p.targetHeight}, 0, 0,
    gocv.InterpolationLinear)

    // Normalize
    normalized := gocv.NewMat()
    resized.ConvertTo(&normalized, gocv.MatTypeCV32F)
    normalized.DivideFloat(255.0)

    resized.Close()
    return normalized
}
```

Common Operations

Color detection:

```
func detectColor(img gocv.Mat, lower, upper gocv.Scalar) gocv.Mat {
    hsv := gocv.NewMat()
    defer hsv.Close()
    gocv.CvtColor(img, &hsv, gocv.ColorBGRToHSV)

    mask := gocv.NewMat()
    gocv.InRangeWithScalar(hsv, lower, upper, &mask)

    return mask
}

// Find red objects
redMask := detectColor(img,
    gocv.Scalar{Val1: 0, Val2: 100, Val3: 100}, // Lower HSV
    gocv.Scalar{Val1: 10, Val2: 255, Val3: 255}) // Upper HSV
```

Feature detection:

```
func detectFeatures(img gocv.Mat) []gocv.KeyPoint {
    gray := gocv.NewMat()
    defer gray.Close()
    gocv.CvtColor(img, &gray, gocv.ColorBGRToGray)

    orb := gocv.NewORB()
    defer orb.Close()

    keypoints := orb.Detect(gray)
    return keypoints
}
```

Contour finding:

```
func findContours(binary gocv.Mat) [][]image.Point {
    contours := gocv.FindContours(binary, gocv.RetrievalExternal, gocv.ChainApproxSimple)

    // Filter by area
    var significant [][]image.Point
    for _, contour := range contours {
        area := gocv.ContourArea(contour)
        if area > 100 { // Minimum area threshold
            significant = append(significant, contour)
        }
    }
    return significant
}
```

Cross-reference: See Chapter 12 for ML-based vision using the acceleration layer.

6.5 Depth Sensing

Depth cameras unlock 3D perception.

Point Cloud Generation

Convert depth images to 3D points:

```
func depthToPointCloud(depth *DepthMap, intrinsics *CameraIntrinsics) *PointCloud {
    points := make([]r3.Vector, 0, depth.Width*depth.Height)
```

```

    for y := 0; y < depth.Height; y++ {
        for x := 0; x < depth.Width; x++ {
            d := depth.At(x, y)
            if d <= 0 || d > 10.0 { // Invalid or too far
                continue
            }

            // Back-project to 3D
            px := (float64(x) - intrinsics.Cx) / intrinsics.Fx * d
            py := (float64(y) - intrinsics.Cy) / intrinsics.Fy * d
            pz := d

            points = append(points, r3.Vector{X: px, Y: py, Z: pz})
        }
    }

    return &PointCloud{Points: points}
}

```

Depth Image Formats

```

type DepthMap struct {
    Width  int
    Height int
    Data   []float32 // Meters per pixel
    MinDist float32 // Minimum valid distance
    MaxDist float32 // Maximum valid distance
}

func (d *DepthMap) At(x, y int) float32 {
    return d.Data[y*d.Width+x]
}

func (d *DepthMap) IsValid(x, y int) bool {
    v := d.At(x, y)
    return v >= d.MinDist && v <= d.MaxDist
}

```

Registration with RGB

Aligning color and depth images:

```

type RGBDFrame struct {
    Color    image.Image
    Depth    *DepthMap
    Transform mat4.Mat4 // Depth to color transform
}

// Project depth point to color pixel
func (f *RGBDFrame) DepthToColor(x, y int) (cx, cy int) {
    d := f.Depth.At(x, y)

    // 3D point in depth frame
    point := backProject(x, y, d, f.depthIntrinsics)

    // Transform to color frame
    colorPoint := f.Transform.MulVec(point)

    // Project to color image

```

```
    cx, cy = project(colorPoint, f.colorIntrinsics)
    return
}
```

With component types covered—sensors, actuators, and cameras—Chapter 7 explores services: the software capabilities that process component data and coordinate robot behavior.

Chapter 7: Services

Services are the brains of a robot—software capabilities that process sensor data, make decisions, and coordinate actions. Unlike components that abstract hardware, services are pure software.

7.1 Components vs Services

The distinction matters for architecture:

Aspect	Components	Services
Purpose	Hardware abstraction	Software capabilities
Examples	Motors, cameras, sensors	Vision, navigation, SLAM
Dependencies	Physical hardware	Other resources (components/services)
Location	Close to hardware	Anywhere on network
State	Hardware state	Computed state

Components answer: "What does this hardware do?" **Services** answer: "What can this robot accomplish?"

A robot might have:

- Camera component: Captures images
- Vision service: Detects objects in those images
- Motor components: Spin wheels
- Navigation service: Plans paths and drives motors

7.2 The Service Interface

Services implement the same base Resource interface:

```
// From service/service.go
type Service interface {
    resource.Resource
}
```

Specific service types add domain-specific methods:

```
type VisionService interface {
    Service

    // Detect finds objects in an image.
    Detect(ctx context.Context, img image.Image) (*Detections, error)

    // Classify identifies what an image contains.
    Classify(ctx context.Context, img image.Image) (*Classifications, error)
}
```

Service Registration

Services register themselves for discovery:

```
func init() {
    registry.RegisterService("vision", "yolox", NewYOLOXVision)
}

func NewYOLOXVision(ctx context.Context, deps resource.Dependencies, conf resource.Config) (Service,
```

```

error) {
    // Get camera dependency
    camName := conf.GetString("camera")
    camera, err := deps.Get(resource.MustParseName(camName))
    if err != nil {
        return nil, fmt.Errorf("camera %s not found: %w", camName, err)
    }

    // Load model
    modelPath := conf.GetString("model_path")
    model, err := loadModel(modelPath)
    if err != nil {
        return nil, err
    }

    return &YOLOXVision{
        camera: camera.(Camera),
        model:  model,
    }, nil
}

```

Discovery Mechanisms

Find services by type:

```

// Get all vision services
visionServices, _ := deps.GetByType("vision")

// Get specific service by name
detector, _ := deps.Get(resource.MustParseName("gorai:service:vision/detector"))

```

7.3 Built-in Service Types

7.3.1 Vision Service

Process images to extract semantic information:

```

type VisionService interface {
    Service

    // Object detection
    DetectObjects(ctx context.Context, img image.Image) (*Detections, error)

    // Classification
    Classify(ctx context.Context, img image.Image) (*Classifications, error)

    // Segmentation
    Segment(ctx context.Context, img image.Image) (*SegmentationMask, error)
}

type Detection struct {
    Label      string
    Confidence  float64
    BoundingBox Rectangle
}

type Detections struct {
    Detections []Detection
}

```

Usage:

```
img, _ := camera.Image(ctx)
detections, _ := visionService.DetectObjects(ctx, img)

for _, det := range detections.Detections {
    if det.Label == "person" && det.Confidence > 0.7 {
        log.Printf("Person detected at %v", det.BoundingBox)
    }
}
```

Cross-reference: See Chapter 12 for ML models powering vision services.

7.3.2 Navigation Service

Plan and execute paths:

```
type NavigationService interface {
    Service

    // SetGoal sets a navigation target.
    SetGoal(ctx context.Context, goal *Pose) error

    // GetPath returns the planned path to current goal.
    GetPath(ctx context.Context) (*Path, error)

    // GetPosition returns current estimated position.
    GetPosition(ctx context.Context) (*Pose, error)

    // Cancel stops current navigation.
    Cancel(ctx context.Context) error

    // IsNavigating returns true if actively navigating.
    IsNavigating(ctx context.Context) (bool, error)
}
```

Navigation services typically:

- Subscribe to sensor data (LiDAR, odometry)
- Maintain an internal map or use provided map
- Plan paths avoiding obstacles
- Publish velocity commands to base

7.3.3 SLAM Service

Simultaneous Localization and Mapping:

```
type SLAMService interface {
    Service

    // GetMap returns the current map.
    GetMap(ctx context.Context) (*Map, error)

    // GetPosition returns position within the map.
    GetPosition(ctx context.Context) (*Pose, error)

    // SaveMap persists the current map.
    SaveMap(ctx context.Context, path string) error

    // LoadMap loads a previously saved map.
```

```
LoadMap(ctx context.Context, path string) error
}
```

SLAM fuses:

- LiDAR scans
- Camera images
- Odometry
- IMU data

To produce:

- 2D or 3D map of environment
- Robot's position within that map

7.3.4 Motion Planning

Generate collision-free trajectories:

```
type MotionService interface {
    Service

    // Plan generates a trajectory from current to goal pose.
    Plan(ctx context.Context, goal *Pose) (*Trajectory, error)

    // PlanWithConstraints plans respecting constraints.
    PlanWithConstraints(ctx context.Context, goal *Pose, constraints *Constraints) (*Trajectory,
error)

    // Execute runs a trajectory on the arm/base.
    Execute(ctx context.Context, trajectory *Trajectory) error
}

type Constraints struct {
    MaxVelocity      float64
    MaxAcceleration float64
    Obstacles        []*Obstacle
    JointLimits      []JointLimit
}
```

7.4 Custom Services

When to Create a Service

Create a service when you have:

- Pure software functionality (no hardware)
- Processing that depends on multiple components
- Stateful logic that persists across calls
- Capability that should be reusable

Examples:

- Object tracking: Maintains object identities across frames
- Battery monitor: Watches battery levels, triggers alerts
- Behavior coordinator: Implements state machine for robot behavior

Service Implementation Example

```
// Custom service: object tracking
type ObjectTracker struct {
    name    resource.Name
    camera  Camera
}
```

```

    vision VisionService
    tracked map[int]*TrackedObject
    nextID  int
    mu      sync.RWMutex
}

func NewObjectTracker(deps resource.Dependencies, conf resource.Config) (*ObjectTracker, error) {
    camName := conf.GetString("camera")
    camera, _ := deps.Get(resource.MustParseName(camName))

    visName := conf.GetString("vision")
    vision, _ := deps.Get(resource.MustParseName(visName))

    return &ObjectTracker{
        name:      resource.NewServiceName("gorai", "tracking", "tracker"),
        camera:    camera.(Camera),
        vision:    vision.(VisionService),
        tracked:   make(map[int]*TrackedObject),
    }, nil
}

func (t *ObjectTracker) Name() resource.Name {
    return t.name
}

func (t *ObjectTracker) Update(ctx context.Context) ([]TrackedObject, error) {
    img, err := t.camera.Image(ctx)
    if err != nil {
        return nil, err
    }

    detections, err := t.vision.DetectObjects(ctx, img)
    if err != nil {
        return nil, err
    }

    t.mu.Lock()
    defer t.mu.Unlock()

    // Match detections to existing tracks
    // Update positions, assign IDs to new objects
    // Remove stale tracks
    // ...

    return t.getTrackedObjects(), nil
}

func (t *ObjectTracker) GetObject(id int) (*TrackedObject, error) {
    t.mu.RLock()
    defer t.mu.RUnlock()

    obj, ok := t.tracked[id]
    if !ok {
        return nil, fmt.Errorf("object %d not found", id)
    }
    return obj, nil
}

// Resource interface methods
func (t *ObjectTracker) Reconfigure(ctx context.Context, deps resource.Dependencies, conf resource.Config) error {

```

```
        return nil
    }

    func (t *ObjectTracker) DoCommand(ctx context.Context, cmd map[string]any) (map[string]any, error) {
        return nil, nil
    }

    func (t *ObjectTracker) Close(ctx context.Context) error {
        return nil
    }
}
```

Service Lifecycle

Services follow the same lifecycle as components:

1. **Creation:** Factory function called with dependencies
2. **Configuration:** Initial config applied
3. **Running:** Service processes requests
4. **Reconfiguration:** Config updates applied live
5. **Shutdown:** Close() called for cleanup

Cross-reference: See Chapter 10 for complete custom service implementation guide.

Chapter 8 covers setting up your development environment to build components and services.

Chapter 8: Development Environment

A well-configured development environment accelerates learning and productivity. This chapter covers everything you need to start building with Gorai.

8.1 Prerequisites

Go 1.21+

Gorai requires Go 1.21 or later for generics support.

Linux (apt):

```
sudo add-apt-repository ppa:longsleep/golang-backports
sudo apt update
sudo apt install golang-go
```

Linux (manual):

```
wget https://go.dev/dl/go1.22.0.linux-amd64.tar.gz
sudo rm -rf /usr/local/go
sudo tar -C /usr/local -xzf go1.22.0.linux-amd64.tar.gz
echo 'export PATH=$PATH:/usr/local/go/bin' >> ~/.bashrc
source ~/.bashrc
```

macOS:

```
brew install go
```

Verify:

```
go version
# go version go1.22.0 linux/amd64
```

NATS Server

Linux/macOS:

```
# Via package manager
brew install nats-server # macOS
# or
go install github.com/nats-io/nats-server/v2@latest

# Or download binary
curl -L https://github.com/nats-io/nats-server/releases/download/v2.10.7/nats-server-v2.10.7-linux-amd64.zip -o nats-server.zip
unzip nats-server.zip
sudo mv nats-server-v2.10.7-linux-amd64/nats-server /usr/local/bin/
```

Verify:

```
nats-server --version
```

Protocol Buffers Toolchain

For working with proto files:

```
# Install protoc compiler
# Linux
sudo apt install -y protobuf-compiler

# macOS
brew install protobuf

# Install Go plugins
go install google.golang.org/protobuf/cmd/protoc-gen-go@latest
go install google.golang.org/grpc/cmd/protoc-gen-go-grpc@latest

# Install buf (recommended)
go install github.com/bufbuild/buf/cmd/buf@latest
```

Optional: TinyGo

For microcontroller development:

```
# Linux
wget https://github.com/tinygo-org/tinygo/releases/download/v0.30.0/tinygo_0.30.0_amd64.deb
sudo dpkg -i tinygo_0.30.0_amd64.deb

# macOS
brew tap tinygo-org/tools
brew install tinygo

# Verify
tinygo version
```

8.2 Project Setup

Clone the Repository

```
git clone https://github.com/gorai/gorai
cd gorai
```

Download Dependencies

```
go mod download
```

Verify Build

```
go build ./...
```

Run Tests

```
go test ./...
```

If tests pass, your environment is ready.

8.3 Essential Tools

nats-server

The message broker:

```
# Start with JetStream enabled
nats-server -js

# With verbose logging
nats-server -js -V

# With config file
nats-server -c nats.conf
```

nats CLI

The command-line client:

```
go install github.com/nats-io/natscli/nats@latest

# Basic commands
nats sub ">"      # Subscribe to all
nats pub foo bar  # Publish message
nats server info  # Server status
```

buf

Protocol buffer management:

```
# Generate Go code from protos
buf generate

# Lint proto files
buf lint

# Check for breaking changes
buf breaking --against .git#branch=main
```

air (Hot Reload)

Automatic rebuilds during development:

```
go install github.com/cosmtrek/air@latest

# Run with hot reload
air
```

Configure with `.air.toml` :

```
[build]
cmd = "go build -o ./tmp/main ./cmd/myapp"
bin = "./tmp/main"
include_ext = ["go", "proto"]
```

dlv (Debugger)

Go debugger:

```
go install github.com/go-delve/delve/cmd/dlv@latest

# Debug a program
dlv debug ./examples/hello-sensor
```

```
# Attach to running process
dlv attach <pid>
```

8.4 IDE Configuration

VS Code

Recommended extensions:

- **Go** (golang.go): Essential Go support
- **NATS** (nats-io.vscode-nats): NATS syntax and tools
- **Proto 3** (zxh404.vscode-proto3): Protocol buffer support

Settings (`.vscode/settings.json`):

```
{
  "go.useLanguageServer": true,
  "go.lintTool": "golangci-lint",
  "go.testFlags": ["-v"],
  "editor.formatOnSave": true,
  "[go]": {
    "editor.defaultFormatter": "golang.go"
  }
}
```

Launch config (`.vscode/launch.json`):

```
{
  "version": "0.2.0",
  "configurations": [
    {
      "name": "Hello Sensor",
      "type": "go",
      "request": "launch",
      "mode": "debug",
      "program": "${workspaceFolder}/examples/hello-sensor",
      "args": ["-fake", "-fake-temp", "50"]
    }
  ]
}
```

GoLand

GoLand works out of the box with Go projects. Recommended settings:

- Enable "Optimize imports on the fly"
- Configure "Go | Go Modules" for the project
- Set up run configurations for examples

8.5 Scripts and Automation

Gorai includes helper scripts:

scripts/start.sh

Start all development services:

```
#!/bin/bash
# Start NATS server in background
```

```
nats-server -js &  
echo "NATS started on port 4222"
```

scripts/stop.sh

Stop services:

```
#!/bin/bash  
pkill nats-server  
echo "Services stopped"
```

scripts/hello.sh

Run the hello-sensor example:

```
#!/bin/bash  
cd "$(dirname "$0")/.." &&  
go run ./examples/hello-sensor "$@"
```

Makefile Targets

```
.PHONY: build test run clean  
  
build:  
    go build ./...  
  
test:  
    go test ./...  
  
test-quick:  
    go test -tags=component ./...  
  
test-all:  
    go test -tags="component integration" ./...  
  
run-hello:  
    go run ./examples/hello-sensor -fake  
  
proto:  
    buf generate  
  
lint:  
    golangci-lint run  
  
nats-start:  
    nats-server -js &  
  
nats-stop:  
    pkill nats-server  
  
clean:  
    rm -rf tmp/ bin/
```

8.6 Hardware Setup

Reference Platform: Raspberry Pi 5

The recommended starting platform:

OS Installation:

- 1. Download Raspberry Pi OS (64-bit)
- 2. Flash with Raspberry Pi Imager
- 3. Enable SSH in settings
- 4. Boot and connect

Go Installation:

```
# On the Pi
wget https://go.dev/dl/go1.22.0.linux-arm64.tar.gz
sudo tar -C /usr/local -xzf go1.22.0.linux-arm64.tar.gz
echo 'export PATH=$PATH:/usr/local/go/bin' >> ~/.bashrc
source ~/.bashrc
```

Clone and Build:

```
git clone https://github.com/gorai/gorai
cd gorai
go build ./...
```

GPIO Access:

```
# Add user to gpio group
sudo usermod -aG gpio $USER
# Logout and login for group change
```

Other Supported Boards

Board	CPU	RAM	NPU	Notes
Raspberry Pi 5	Cortex-A76	8GB	No	Best starter board
Orange Pi 5	RK3588S	8-16GB	6 TOPS	Best for AI
Jetson Orin Nano	Cortex-A78	8GB	GPU	CUDA support
BeagleBone AI-64	TDA4VM	4GB	8 TOPS	Real-time PRUs

8.7 Microcontroller Development

TinyGo Setup

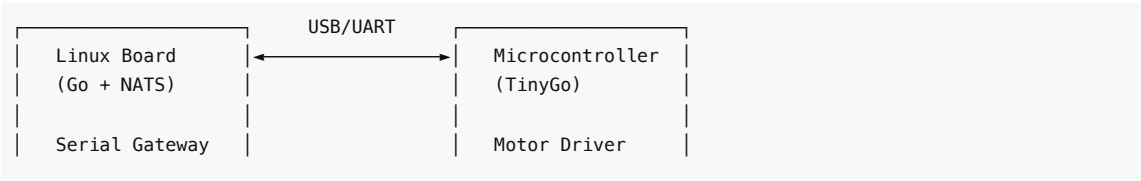
Install TinyGo for your platform, then:

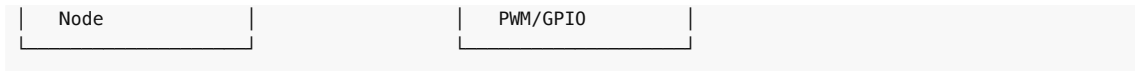
```
# Flash to RP2040 (Raspberry Pi Pico)
tinygo flash -target=pico ./examples/tinygo/blink

# Flash to ESP32
tinygo flash -target=esp32 ./examples/tinygo/blink
```

Serial Gateway

Connect microcontroller to Linux board:





The serial gateway translates NATS messages to a compact serial protocol.

Cross-reference: See [specs/serial-interfaces.md](#) for protocol details.

With your development environment configured, Chapter 9 provides a deep dive into the hello-sensor example, showing how all these pieces come together.

Chapter 9: Hello Sensor Deep Dive

This chapter walks through a complete, working Gorai component: the `hello-sensor` example. By understanding every line, you'll be ready to build your own components.

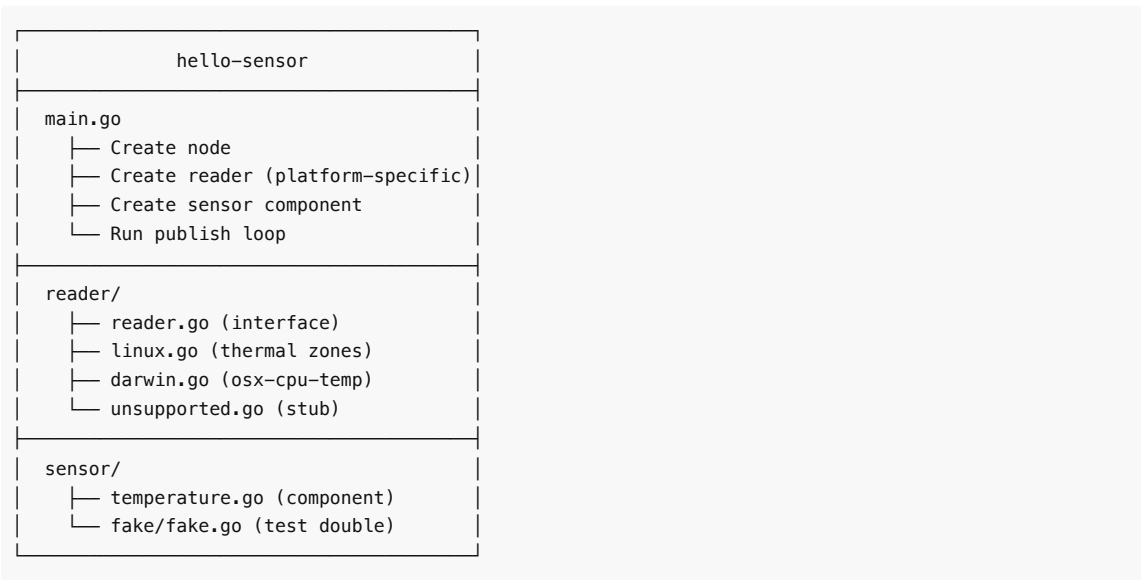
9.1 What We're Building

The `hello-sensor` reads CPU temperature from the host system and publishes it to NATS. It demonstrates:

- Creating a Gorai node
- Platform-specific hardware access
- Implementing the Sensor interface
- Publishing structured messages
- Configuration and command-line flags
- Statistics collection
- Graceful shutdown
- Fake implementations for testing

The complete code is in `examples/hello-sensor/`.

9.2 Architecture Overview



Separation of concerns:

- `reader/` : Platform-specific temperature reading
- `sensor/` : Gorai component wrapping the reader
- `main.go` : Entry point orchestrating everything

Cross-reference: See Chapter 4 for the sensor interface this implements.

9.3 The Reader Package

The reader package abstracts platform-specific temperature reading behind a common interface.

9.3.1 Interface Design

From `reader/reader.go` :

```
// Reading represents a temperature reading from a thermal zone.
type Reading struct {
```

```

    Zone          string
    TemperatureC  float64
    CriticalC     float64 // 0 if unknown
    WarningC      float64 // 0 if unknown
}

// Reader reads temperature from the host system.
type Reader interface {
    // Platform returns the platform name (e.g., "linux", "darwin").
    Platform() string

    // Zones returns available thermal zones.
    Zones(ctx context.Context) ([]string, error)

    // Read returns temperature for a specific zone.
    // Use "" or "default" for the primary zone.
    Read(ctx context.Context, zone string) (Reading, error)

    // Close releases resources.
    Close() error
}

```

Design decisions:

- **Platform()**: Identify which implementation is running
- **Zones()**: Discover available temperature sources
- **Read()**: Get temperature for a specific zone
- **Close()**: Clean shutdown pattern

The factory function selects the right implementation:

```

func New() (Reader, error) {
    switch runtime.GOOS {
    case "linux":
        return newLinuxReader()
    case "darwin":
        return newDarwinReader()
    default:
        return nil, fmt.Errorf("unsupported platform: %s", runtime.GOOS)
    }
}

```

9.3.2 Linux Implementation

Linux exposes thermal data via sysfs. From `reader/linux.go` :

```

//go:build linux

package reader

const thermalBasePath = "/sys/class/thermal"

type linuxReader struct {
    zones []string
}

func newLinuxReader() (Reader, error) {
    r := &linuxReader{}

    // Discover thermal zones
    entries, err := os.ReadDir(thermalBasePath)

```

```

    if err != nil {
        return nil, fmt.Errorf("failed to read thermal directory: %w", err)
    }

    for _, entry := range entries {
        if strings.HasPrefix(entry.Name(), "thermal_zone") {
            r.zones = append(r.zones, entry.Name())
        }
    }

    if len(r.zones) == 0 {
        return nil, fmt.Errorf("no thermal zones found")
    }

    return r, nil
}

func (r *linuxReader) Read(ctx context.Context, zone string) (Reading, error) {
    if zone == "" || zone == "default" {
        zone = r.zones[0]
    }

    reading := Reading{Zone: zone}

    // Read temperature (in millidegrees Celsius)
    tempPath := filepath.Join(thermalBasePath, zone, "temp")
    tempData, err := os.ReadFile(tempPath)
    if err != nil {
        return reading, fmt.Errorf("failed to read temperature: %w", err)
    }

    tempMilliC, err := strconv.ParseInt(strings.TrimSpace(string(tempData)), 10, 64)
    if err != nil {
        return reading, fmt.Errorf("failed to parse temperature: %w", err)
    }
    reading.TemperatureC = float64(tempMilliC) / 1000.0

    // Try to read trip points (optional)
    reading.CriticalC = r.readTripPoint(zone, "critical")
    reading.WarningC = r.readTripPoint(zone, "hot")

    return reading, nil
}

```

Key points:

- Temperature is in millidegrees (divide by 1000)
- Multiple thermal zones exist (CPU, GPU, WiFi, etc.)
- Trip points indicate thermal limits
- File reads can fail—handle errors gracefully

9.3.3 Build Tags for Platform-Specific Code

Go build tags select which files compile:

```
//go:build linux
```

This file only compiles on Linux. The darwin.go file has:

```
//go:build darwin
```

This pattern provides:

- Compile-time platform selection
- Clean separation of platform code
- No runtime overhead
- IDE support (shows correct file for platform)

Stubs for unsupported platforms:

When linux.go compiles, it includes a stub for darwin:

```
// In linux.go
func newDarwinReader() (Reader, error) {
    return nil, fmt.Errorf("darwin reader not available on linux")
}
```

This ensures the `New()` function compiles on all platforms.

Helper Functions

Temperature conversion:

```
func CelsiusToFahrenheit(celsius float64) float64 {
    return celsius*9/5 + 32
}
```

Simple, but consistency matters—define it once, use everywhere.

9.4 The Sensor Component

The sensor package wraps the reader in a Gorai component.

9.4.1 Implementing resource.Resource

From `sensor/temperature.go` :

```
type TemperatureSensor struct {
    name    resource.Name
    config  Config
    reader  reader.Reader
    nc      *nats.Conn

    mu      sync.RWMutex
    running bool
    cancel  context.CancelFunc
    readingCount uint64
    errorCount  uint64
    lastError   string
    lastReading *TemperatureReading

    // Stats for diagnostics
    minTemp float64
    maxTemp float64
    sumTemp float64
    statCount int
}
```

State management:

- `mu` : Protects concurrent access
- `running` : Tracks if publishing is active

- `cancel` : For stopping the publish loop
- Statistics for debugging and monitoring

Name() implementation:

```
func (s *TemperatureSensor) Name() resource.Name {
    return s.name
}
```

The name is created during construction:

```
name := resource.NewComponentName("gorai", "sensor", cfg.Name)
```

9.4.2 Implementing resource.Sensor

The key method for sensors:

```
func (s *TemperatureSensor) Readings(ctx context.Context) (map[string]any, error) {
    reading, err := s.reader.Read(ctx, s.config.Zone)
    if err != nil {
        s.mu.Lock()
        s.errorCount++
        s.lastError = err.Error()
        s.mu.Unlock()
        return nil, err
    }

    s.mu.Lock()
    s.readingCount++
    s.updateStats(reading.TemperatureC)
    s.mu.Unlock()

    return map[string]any{
        "temperature_celsius":    reading.TemperatureC,
        "temperature_fahrenheit": reader.CelsiusToFahrenheit(reading.TemperatureC),
        "zone":                   reading.Zone,
        "critical_celsius":       reading.CriticalC,
        "warning_celsius":        reading.WarningC,
        "platform":               s.reader.Platform(),
    }, nil
}
```

Pattern: Read hardware, update stats, return map.

9.4.3 Reconfigure() for Hot Reload

```
func (s *TemperatureSensor) Reconfigure(ctx context.Context, deps resource.Dependencies, conf
resource.Config) error {
    var cfg Config
    if err := conf.Unmarshal(&cfg); err != nil {
        return fmt.Errorf("failed to parse config: %w", err)
    }

    s.mu.Lock()
    defer s.mu.Unlock()

    if cfg.Interval > 0 {
        s.config.Interval = cfg.Interval
    }
}
```

```

    if cfg.Topic != "" && cfg.Topic != s.config.Topic {
        s.config.Topic = cfg.Topic
    }

    return nil
}

```

Configuration changes take effect without restart.

9.4.4 DoCommand() for Extensibility

Custom commands beyond the standard interface:

```

func (s *TemperatureSensor) DoCommand(ctx context.Context, cmd map[string]any) (map[string]any,
error) {
    if cmdName, ok := cmd["command"].(string); ok {
        switch cmdName {
        case "get_last_reading":
            s.mu.RLock()
            reading := s.lastReading
            s.mu.RUnlock()

            if reading == nil {
                return nil, fmt.Errorf("no reading available")
            }

            return map[string]any{
                "temperature_celsius":    reading.TemperatureCelsius,
                "temperature_fahrenheit": reading.TemperatureFahrenheit,
                "zone":                   reading.Zone,
            }, nil

        case "get_stats":
            s.mu.RLock()
            defer s.mu.RUnlock()

            avg := 0.0
            if s.statCount > 0 {
                avg = s.sumTemp / float64(s.statCount)
            }

            return map[string]any{
                "reading_count": s.readingCount,
                "error_count":   s.errorCount,
                "last_error":    s.lastError,
                "min_celsius":   s.minTemp,
                "max_celsius":   s.maxTemp,
                "avg_celsius":    avg,
            }, nil
        }
    }
    return nil, fmt.Errorf("unknown command: %v", cmd)
}

```

Use cases:

- Diagnostics and debugging
- Custom operations not in standard interface
- Integration with management tools

9.4.5 Publishing Loop

The sensor publishes periodically:

```
func (s *TemperatureSensor) Start(ctx context.Context) error {
    s.mu.Lock()
    if s.running {
        s.mu.Unlock()
        return fmt.Errorf("already running")
    }
    s.running = true

    ctx, cancel := context.WithCancel(ctx)
    s.cancel = cancel
    s.mu.Unlock()

    go s.run(ctx)
    return nil
}

func (s *TemperatureSensor) run(ctx context.Context) {
    ticker := time.NewTicker(s.config.Interval)
    defer ticker.Stop()

    for {
        select {
        case <-ctx.Done():
            s.mu.Lock()
            s.running = false
            s.mu.Unlock()
            return

        case <-ticker.C:
            s.publishReading(ctx)
        }
    }
}
```

Pattern: Ticker-based loop with context cancellation.

9.4.6 Statistics Tracking

```
func (s *TemperatureSensor) updateStats(temp float64) {
    if s.statCount == 0 {
        s.minTemp = temp
        s.maxTemp = temp
    } else {
        if temp < s.minTemp {
            s.minTemp = temp
        }
        if temp > s.maxTemp {
            s.maxTemp = temp
        }
    }
    s.sumTemp += temp
    s.statCount++
}
```

Simple but useful for monitoring temperature over time.

9.5 The Fake Reader

Test doubles enable testing without hardware.

From `sensor/fake/fake.go` :

```
package fake

type FakeReader struct {
    mu          sync.RWMutex
    temperature float64
    zones       []string
    shouldError bool
    errorMsg    string
}

func New() *FakeReader {
    return &FakeReader{
        temperature: 42.0,
        zones:       []string{"fake_zone0"},
    }
}

func (f *FakeReader) SetTemperature(celsius float64) {
    f.mu.Lock()
    defer f.mu.Unlock()
    f.temperature = celsius
}

func (f *FakeReader) SetError(msg string) {
    f.mu.Lock()
    defer f.mu.Unlock()
    f.shouldError = true
    f.errorMsg = msg
}

func (f *FakeReader) Read(ctx context.Context, zone string) (reader.Reading, error) {
    f.mu.RLock()
    defer f.mu.RUnlock()

    if f.shouldError {
        return reader.Reading{}, fmt.Errorf(f.errorMsg)
    }

    return reader.Reading{
        Zone:         zone,
        TemperatureC: f.temperature,
        CriticalC:     100.0,
        WarningC:      80.0,
    }, nil
}
```

Key features:

- `SetTemperature()` : Control returned value
- `SetError()` : Simulate hardware failures
- Thread-safe with mutex
- Implements full Reader interface

9.6 Main Entry Point

From `main.go` :

```

func main() {
    // Parse flags
    natsURL := flag.String("nats", "nats://localhost:4222", "NATS server URL")
    interval := flag.Duration("interval", time.Second, "Publishing interval")
    topic := flag.String("topic", "gorai.hello.cpu_temp.data", "NATS topic")
    zone := flag.String("zone", "", "Thermal zone to read")
    useFake := flag.Bool("fake", false, "Use fake reader")
    fakeTemp := flag.Float64("fake-temp", 42.0, "Temperature for fake reader")
    flag.Parse()

    // Create context with signal handling
    ctx, cancel := context.WithCancel(context.Background())
    defer cancel()

    sigCh := make(chan os.Signal, 1)
    signal.Notify(sigCh, syscall.SIGINT, syscall.SIGTERM)
    go func() {
        <-sigCh
        log.Println("Shutting down...")
        cancel()
    }()

    // Create node
    n, err := node.New("hello_sensor", node.WithNATS(*natsURL))
    if err != nil {
        log.Fatalf("Failed to create node: %v", err)
    }
    defer n.Close()

    // Create reader
    var r reader.Reader
    if *useFake {
        fr := fake.New()
        fr.SetTemperature(*fakeTemp)
        r = fr
        log.Printf("Using fake reader with temperature %.1f°C", *fakeTemp)
    } else {
        r, err = reader.New()
        if err != nil {
            log.Fatalf("Failed to create reader: %v", err)
        }
    }
    defer r.Close()

    // Create sensor
    cfg := sensor.Config{
        Name:      "cpu_temp",
        Zone:      *zone,
        Interval: *interval,
        Topic:     *topic,
    }

    tempSensor, err := sensor.New(n, r, cfg)
    if err != nil {
        log.Fatalf("Failed to create sensor: %v", err)
    }
    defer tempSensor.Close(ctx)

    // Start publishing
    log.Printf("Starting sensor, publishing to %s every %v", cfg.Topic, cfg.Interval)
}

```

```

    if err := tempSensor.Start(ctx); err != nil {
        log.Fatalf("Failed to start sensor: %v", err)
    }

    // Wait for shutdown
    <-ctx.Done()
}

```

Patterns demonstrated:

1. **Flag parsing:** Configurable without recompiling
2. **Signal handling:** Graceful shutdown on Ctrl+C
3. **Dependency injection:** Reader selected at runtime
4. **Resource cleanup:** defer statements ensure cleanup

9.7 Running the Example

Start NATS

```

./scripts/start.sh
# or
nats-server -js &

```

Run with Fake Reader

```

./scripts/hello.sh -fake -fake-temp 65.0

# or directly
go run ./examples/hello-sensor -fake -fake-temp 65.0

```

Output:

```

Using fake reader with temperature 65.0°C
Platform: fake, Available zones: [fake_zone0]
Initial reading: 65.0°C (149.0°F)
Starting sensor, publishing to gorai.hello.cpu_temp.data every 1s

```

Subscribe to Readings

In another terminal:

```

nats sub "gorai.hello.cpu_temp.data"

```

Output:

```

[#1] Received on "gorai.hello.cpu_temp.data"
{"timestamp":"2024-01-15T10:30:00Z","temperature_celsius":65,"temperature_fahrenheit":149,"zone":"fake_zone0","source":"cpu"}

```

Run with Real Hardware

```

go run ./examples/hello-sensor

```

On Linux, this reads from `/sys/class/thermal/thermal_zone*/temp`.

9.8 Observing the Output

JSON Message Format

Published messages are JSON:

```
{
  "timestamp": "2024-01-15T10:30:00.123Z",
  "temperature_celsius": 42.5,
  "temperature_fahrenheit": 108.5,
  "zone": "thermal_zone0",
  "source": "cpu",
  "critical_celsius": 105.0,
  "warning_celsius": 85.0
}
```

Statistics via DoCommand

Query sensor statistics:

```
# In your code or via test
stats, _ := tempSensor.DoCommand(ctx, map[string]any{"command": "get_stats"})
fmt.Printf("Readings: %v, Errors: %v\n", stats["reading_count"], stats["error_count"])
fmt.Printf("Min: %.1f°C, Max: %.1f°C, Avg: %.1f°C\n",
  stats["min_celsius"], stats["max_celsius"], stats["avg_celsius"])
```

Monitoring Message Flow

```
# Watch all messages
nats sub ">" --raw

# Count messages per second
watch -n1 "nats sub gorai.hello.cpu_temp.data --count 1 2>/dev/null"
```

Cross-reference: See Chapter 3 for NATS CLI usage.

With hello-sensor thoroughly understood, Chapter 10 shows how to build your own custom components following these same patterns.

Chapter 10: Building Custom Components

Now that you understand Gorai's patterns through hello-sensor, let's build custom components from scratch.

10.1 When to Create a Component

Create a component when you have:

- **Hardware to abstract:** Specific sensor, motor, or device
- **Reusable functionality:** Will be used across projects
- **Standard interface compliance:** Fits existing component types
- **Need for fakes:** Testing requires simulation

Don't create a component for:

- One-off scripts
- Pure software logic (use a service instead)
- Simple utilities (use functions)

10.2 Component Structure

Standard layout:

```
component/
├─ mycomponent/
│   ├── mycomponent.go      # Interface definition
│   ├── mycomponent_test.go # Unit tests
│   └─ fake/
│       ├── fake.go         # Test double
│       └─ fake_test.go     # Fake tests
```

Or for a standalone driver:

```
github.com/myorg/gorai-mymotor/
├─ mymotor.go
├─ mymotor_test.go
├─ fake/
│   └─ fake.go
├─ go.mod
└─ README.md
```

10.3 Step-by-Step: Custom Motor Driver

Let's build a motor driver for a DRV8833 dual motor controller.

10.3.1 Define the Interface

First, understand what capabilities we need:

```
// drv8833/drv8833.go
package drv8833

import (
    "context"
    "github.com/gorai/gorai/component/motor"
)

// DRV8833Motor implements motor.Motor for DRV8833 controller.
type DRV8833Motor interface {
    motor.Motor
```

```

// SetDecay sets the decay mode (fast or slow).
SetDecay(ctx context.Context, fast bool) error

// GetFault returns true if fault pin is active.
GetFault(ctx context.Context) (bool, error)
}

```

10.3.2 Implement the Driver

```

// drv8833/motor.go
package drv8833

import (
    "context"
    "fmt"
    "sync"

    "github.com/gorai/gorai/driver/gpio"
    "github.com/gorai/gorai/pkg/resource"
)

type Config struct {
    In1Pin    int    `json:"in1_pin"`
    In2Pin    int    `json:"in2_pin"`
    PWMPin    int    `json:"pwm_pin"`
    FaultPin  int    `json:"fault_pin"`
    MaxPower  float64 `json:"max_power"`
    PWMFreqHz int    `json:"pwm_freq_hz"`
}

type Motor struct {
    name      resource.Name
    config    Config
    in1       gpio.Pin
    in2       gpio.Pin
    pwm       gpio.PWMPin
    fault     gpio.Pin

    mu      sync.RWMutex
    power   float64
    moving  bool
    fastDecay bool
}

func New(name string, cfg Config, pins gpio.Provider) (*Motor, error) {
    if cfg.MaxPower <= 0 || cfg.MaxPower > 1.0 {
        cfg.MaxPower = 1.0
    }
    if cfg.PWMFreqHz <= 0 {
        cfg.PWMFreqHz = 20000 // 20kHz default
    }

    m := &Motor{
        name:    resource.NewComponentName("gorai", "motor", name),
        config:  cfg,
    }

    var err error
    m.in1, err = pins.OutputPin(cfg.In1Pin)
    if err != nil {

```

```

        return nil, fmt.Errorf("failed to configure IN1: %w", err)
    }

    m.in2, err = pins.OutputPin(cfg.In2Pin)
    if err != nil {
        return nil, fmt.Errorf("failed to configure IN2: %w", err)
    }

    m.pwm, err = pins.PWMPin(cfg.PWMPin, cfg.PWMFreqHz)
    if err != nil {
        return nil, fmt.Errorf("failed to configure PWM: %w", err)
    }

    if cfg.FaultPin > 0 {
        m.fault, err = pins.InputPin(cfg.FaultPin)
        if err != nil {
            return nil, fmt.Errorf("failed to configure FAULT: %w", err)
        }
    }

    return m, nil
}

// Resource interface

func (m *Motor) Name() resource.Name {
    return m.name
}

func (m *Motor) Reconfigure(ctx context.Context, deps resource.Dependencies, conf resource.Config) error {
    var cfg Config
    if err := conf.Unmarshal(&cfg); err != nil {
        return err
    }
    m.mu.Lock()
    m.config.MaxPower = cfg.MaxPower
    m.mu.Unlock()
    return nil
}

func (m *Motor) DoCommand(ctx context.Context, cmd map[string]any) (map[string]any, error) {
    return nil, nil
}

func (m *Motor) Close(ctx context.Context) error {
    m.Stop(ctx)
    return nil
}

// Actuator interface

func (m *Motor) IsMoving(ctx context.Context) (bool, error) {
    m.mu.RLock()
    defer m.mu.RUnlock()
    return m.moving, nil
}

func (m *Motor) Stop(ctx context.Context) error {
    m.mu.Lock()
    defer m.mu.Unlock()

```

```

    // Coast stop: both inputs low
    m.in1.Low()
    m.in2.Low()
    m.pwm.SetDutyCycle(0)
    m.power = 0
    m.moving = false

    return nil
}

// Motor interface

func (m *Motor) SetPower(ctx context.Context, power float64) error {
    m.mu.Lock()
    defer m.mu.Unlock()

    // Check for faults
    if m.fault != nil && m.fault.Read() {
        return fmt.Errorf("motor fault detected")
    }

    // Clamp power
    if power > m.config.MaxPower {
        power = m.config.MaxPower
    }
    if power < -m.config.MaxPower {
        power = -m.config.MaxPower
    }

    // Set direction
    if power > 0 {
        m.in1.High()
        m.in2.Low()
    } else if power < 0 {
        m.in1.Low()
        m.in2.High()
        power = -power
    } else {
        // Brake (both high with zero PWM) or coast
        if m.fastDecay {
            m.in1.Low()
            m.in2.Low()
        } else {
            m.in1.High()
            m.in2.High()
        }
    }

    // Set PWM duty cycle
    duty := uint32(power * 65535)
    if err := m.pwm.SetDutyCycle(duty); err != nil {
        return fmt.Errorf("failed to set PWM: %w", err)
    }

    m.power = power
    m.moving = power != 0

    return nil
}

```

```
// Simplified implementations for remaining methods...

func (m *Motor) SetVelocity(ctx context.Context, velocity float64) error {
    return fmt.Errorf("velocity control not supported without encoder")
}

func (m *Motor) GoTo(ctx context.Context, position, velocity float64) error {
    return fmt.Errorf("position control not supported without encoder")
}

func (m *Motor) GoFor(ctx context.Context, rpm, revolutions float64) error {
    return fmt.Errorf("GoFor not supported without encoder")
}

func (m *Motor) GetPosition(ctx context.Context) (float64, error) {
    return 0, fmt.Errorf("position reporting not supported")
}

func (m *Motor) GetVelocity(ctx context.Context) (float64, error) {
    return 0, fmt.Errorf("velocity reporting not supported")
}

func (m *Motor) ResetZeroPosition(ctx context.Context, offset float64) error {
    return fmt.Errorf("not supported")
}

func (m *Motor) IsPowered(ctx context.Context) (bool, float64, error) {
    m.mu.RLock()
    defer m.mu.RUnlock()
    return m.power != 0, m.power, nil
}

func (m *Motor) Properties(ctx context.Context) (motor.Properties, error) {
    return motor.Properties{
        PositionReporting: false,
        VelocityReporting: false,
        SupportsGoTo:     false,
    }, nil
}
```

10.3.3 Create the Fake

```
// drv8833/fake/fake.go
package fake

import (
    "context"
    "sync"

    "github.com/gorai/gorai/component/motor"
    "github.com/gorai/gorai/pkg/resource"
)

type Motor struct {
    name    resource.Name
    mu      sync.RWMutex
    power   float64
    moving  bool
    fault   bool
}
```

```

func New(name string) *Motor {
    return &Motor{
        name: resource.NewComponentName("test", "motor", name),
    }
}

// Test helpers

func (m *Motor) SetFault(fault bool) {
    m.mu.Lock()
    m.fault = fault
    m.mu.Unlock()
}

func (m *Motor) GetPowerSet() float64 {
    m.mu.RLock()
    defer m.mu.RUnlock()
    return m.power
}

// Motor interface implementation...
func (m *Motor) SetPower(ctx context.Context, power float64) error {
    m.mu.Lock()
    defer m.mu.Unlock()

    if m.fault {
        return fmt.Errorf("motor fault")
    }

    m.power = power
    m.moving = power != 0
    return nil
}

// ... other methods similar to real but tracking state

```

10.3.4 Write Tests

```

// drv8833/drv8833_test.go
package drv8833_test

import (
    "context"
    "testing"

    "github.com/myorg/gorai-drv8833/fake"
    "github.com/stretchr/testify/assert"
)

func TestMotor_SetPower(t *testing.T) {
    motor := fake.New("test")

    err := motor.SetPower(context.Background(), 0.5)
    assert.NoError(t, err)
    assert.Equal(t, 0.5, motor.GetPowerSet())

    moving, _ := motor.IsMoving(context.Background())
    assert.True(t, moving)
}

```

```
func TestMotor_SetPower_Fault(t *testing.T) {
    motor := fake.New("test")
    motor.SetFault(true)

    err := motor.SetPower(context.Background(), 0.5)
    assert.Error(t, err)
    assert.Contains(t, err.Error(), "fault")
}

func TestMotor_Stop(t *testing.T) {
    motor := fake.New("test")
    motor.SetPower(context.Background(), 0.5)

    err := motor.Stop(context.Background())
    assert.NoError(t, err)

    moving, _ := motor.IsMoving(context.Background())
    assert.False(t, moving)
}
```

10.4 Registration and Discovery

Adding to Registry

```
func init() {
    registry.RegisterComponent("motor", "drv8833", func(ctx context.Context, deps
resource.Dependencies, conf resource.Config) (any, error) {
        var cfg Config
        if err := conf.Unmarshal(&cfg); err != nil {
            return nil, err
        }
        pins := gpio.DefaultProvider()
        return New(conf.Name, cfg, pins)
    })
}
```

Configuration Schema

Document your configuration:

```
{
  "name": "left_motor",
  "type": "motor",
  "model": "drv8833",
  "attributes": {
    "in1_pin": 17,
    "in2_pin": 18,
    "pwm_pin": 12,
    "fault_pin": 25,
    "max_power": 0.8,
    "pwm_freq_hz": 20000
  }
}
```

10.5 Network Transparency

Expose your component for remote access:

```
// On node with hardware
motor, _ := drv8833.New("left", cfg, pins)
nws.Wrap(node, motor, "gorai.motors.left")

// On remote node
motor := nwc.Motor(remoteNode, "gorai.motors.left")
motor.SetPower(ctx, 0.5) // Works transparently
```

Cross-reference: See Chapter 2 for NWS/NWC concepts.

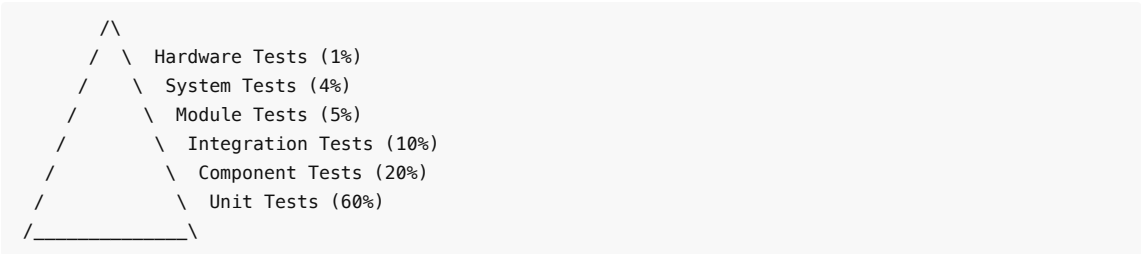
Chapter 11 covers testing these components thoroughly.

Chapter 11: Testing Strategies

Robots are safety-critical systems. Bugs can cause physical damage. Testing is not optional—it's essential.

11.1 The Testing Pyramid

Gorai follows a testing pyramid with more unit tests at the base:



Philosophy: Catch bugs at the lowest level possible. Unit tests are fast, reliable, and pinpoint problems. Higher-level tests catch integration issues but are slower and harder to debug.

11.2 Test Categories and Build Tags

Gorai uses build tags to organize tests:

Tag	Purpose	Speed	NATS	Hardware
(none)	Unit tests	<1s	No	No
component	Single component	1-5s	Embedded	No
integration	Multi-component	5-30s	Embedded	No
module	Full module lifecycle	30s+	Embedded	No
system	Complete robot config	Minutes	Real	Optional
hardware	Real hardware	Variable	Real	Yes

Run specific categories:

```

go test ./...                                # Unit only
go test -tags=component ./...                # Component
go test -tags=integration ./...              # Integration
go test -tags="component integration" ./...  # Both

```

11.3 Unit Testing Patterns

Table-Driven Tests

The Go idiom for comprehensive testing:

```

func TestCelsiusToFahrenheit(t *testing.T) {
    tests := []struct {
        name      string
        celsius   float64
        fahrenheit float64
    }{
        {"freezing", 0, 32},
        {"boiling", 100, 212},
        {"body temp", 37, 98.6},
    }
}

```

```

        {"negative", -40, -40}, // Same in both!
    }

    for _, tt := range tests {
        t.Run(tt.name, func(t *testing.T) {
            got := CelsiusToFahrenheit(tt.celsius)
            if math.Abs(got-tt.fahrenheit) > 0.1 {
                t.Errorf("CelsiusToFahrenheit(%v) = %v, want %v",
                    tt.celsius, got, tt.fahrenheit)
            }
        })
    }
}

```

Test Helpers

Mark helper functions with `t.Helper()` :

```

func assertNoError(t *testing.T, err error) {
    t.Helper() // Points to caller in failure output
    if err != nil {
        t.Fatalf("unexpected error: %v", err)
    }
}

func createTestSensor(t *testing.T) *TemperatureSensor {
    t.Helper()
    fake := fake.New()
    fake.SetTemperature(25.0)
    sensor, err := sensor.New(nil, fake, sensor.DefaultConfig())
    assertNoError(t, err)
    t.Cleanup(func() {
        sensor.Close(context.Background())
    })
    return sensor
}

```

Parallel Tests

Speed up test suites:

```

func TestMotor_SetPower(t *testing.T) {
    t.Parallel() // Run concurrently with other parallel tests

    motor := fake.NewMotor()
    // Test...
}

func TestMotor_Stop(t *testing.T) {
    t.Parallel() // Also runs in parallel

    motor := fake.NewMotor()
    // Test...
}

```

Avoid shared state in parallel tests.

11.4 Fake Implementations

Every component needs a fake for testing.

Hooks for Custom Behavior

```
type FakeMotor struct {
    mu sync.RWMutex

    // State
    power float64
    moving bool

    // Hooks for custom test behavior
    OnSetPower func(power float64) error
    OnStop     func() error
}

func (m *FakeMotor) SetPower(ctx context.Context, power float64) error {
    if m.OnSetPower != nil {
        return m.OnSetPower(power)
    }

    m.mu.Lock()
    m.power = power
    m.moving = power != 0
    m.mu.Unlock()
    return nil
}
```

Usage in tests:

```
func TestBehavior_StopsOnOverheat(t *testing.T) {
    motor := fake.NewMotor()

    // Track if stop was called
    stopCalled := false
    motor.OnStop = func() error {
        stopCalled = true
        return nil
    }

    // Run behavior with high temperature
    behavior := NewThermalSafety(motor, tempSensor)
    tempSensor.SetTemperature(95.0) // Overheat!
    behavior.Check()

    assert.True(t, stopCalled, "should stop motor on overheat")
}
```

Error Injection

```
func TestSensor_HandlesReadError(t *testing.T) {
    fake := fake.NewReader()
    fake.SetError("I2C timeout")

    sensor, _ := sensor.New(nil, fake, cfg)
    _, err := sensor.Readings(context.Background())

    assert.Error(t, err)
    assert.Contains(t, err.Error(), "I2C timeout")
}
```

11.5 Testing with NATS

Embedded NATS Server

For tests needing real NATS:

```
//go:build component

package sensor_test

import (
    "testing"
    "time"

    "github.com/nats-io/nats-server/v2/server"
    "github.com/nats-io/nats.go"
)

func startTestNATS(t *testing.T) *nats.Conn {
    t.Helper()

    opts := &server.Options{
        Host:      "127.0.0.1",
        Port:      -1, // Random port
        JetStream: true,
        StoreDir:  t.TempDir(),
    }

    ns, err := server.NewServer(opts)
    if err != nil {
        t.Fatal(err)
    }

    go ns.Start()
    if !ns.ReadyForConnections(10 * time.Second) {
        t.Fatal("NATS server not ready")
    }

    t.Cleanup(func() {
        ns.Shutdown()
    })

    nc, err := nats.Connect(ns.ClientURL())
    if err != nil {
        t.Fatal(err)
    }

    t.Cleanup(func() {
        nc.Close()
    })

    return nc
}
```

Testing Pub/Sub

```
func TestSensor_PublishesReadings(t *testing.T) {
    nc := startTestNATS(t)
```

```

// Subscribe before publishing
received := make(chan *sensor.Temperature, 1)
sub, _ := nc.Subscribe("gorai.sensors.temp.data", func(m *nats.Msg) {
    var temp sensor.Temperature
    proto.Unmarshal(m.Data, &temp)
    received <- &temp
})
defer sub.Unsubscribe()

// Create and start sensor
n, _ := node.New("test", node.WithConnection(nc))
sensor := createSensor(n)
sensor.Start(context.Background())

// Wait for message
select {
case msg := <-received:
    assert.InDelta(t, 25.0, msg.Temperature, 0.1)
case <-time.After(5 * time.Second):
    t.Fatal("timeout waiting for message")
}
}

```

Testing Request/Reply

```

func TestService_RespondsToRequest(t *testing.T) {
    nc := startTestNATS(t)

    // Set up service
    nc.Subscribe("test.service", func(m *nats.Msg) {
        m.Respond([]byte("pong"))
    })

    // Make request
    resp, err := nc.Request("test.service", []byte("ping"), time.Second)
    assert.NoError(t, err)
    assert.Equal(t, []byte("pong"), resp.Data)
}

```

11.6 Component Tests

Test a single component with real dependencies:

```

//go:build component

func TestTemperatureSensor_Component(t *testing.T) {
    nc := startTestNATS(t)
    n, _ := node.New("test", node.WithConnection(nc))
    defer n.Close()

    fake := fake.NewReader()
    fake.SetTemperature(42.0)

    cfg := sensor.Config{
        Name:      "test_temp",
        Interval:  100 * time.Millisecond,
        Topic:     "test.temp.data",
    }
}

```

```

s, _ := sensor.New(n, fake, cfg)
defer s.Close(context.Background())

// Test readings
readings, err := s.Readings(context.Background())
assert.NoError(t, err)
assert.Equal(t, 42.0, readings["temperature_celsius"])

// Test publishing
s.Start(context.Background())
time.Sleep(200 * time.Millisecond)

stats, _ := s.DoCommand(context.Background(), map[string]any{"command": "get_stats"})
assert.Greater(t, stats["reading_count"].(uint64), uint64(0))
}

```

11.7 Integration Tests

Test multiple components together:

```

//go:build integration

func TestVisionPipeline_Integration(t *testing.T) {
    nc := startTestNATS(t)
    n, _ := node.New("test", node.WithConnection(nc))
    defer n.Close()

    // Create fake camera
    camera := fake.NewCamera()
    camera.SetImage(testImage)

    // Create real vision service with fake camera
    vision, _ := vision.New(n, camera, visionConfig)
    defer vision.Close(context.Background())

    // Test detection pipeline
    img, _ := camera.Image(context.Background())
    detections, err := vision.DetectObjects(context.Background(), img)

    assert.NoError(t, err)
    assert.Greater(t, len(detections.Detections), 0)
}

```

11.8 Hardware Tests

Tests requiring real hardware:

```

//go:build hardware && linux

func TestLinuxThermalReader_Hardware(t *testing.T) {
    reader, err := reader.New()
    if err != nil {
        t.Skip("thermal zones not available:", err)
    }
    defer reader.Close()

    zones, _ := reader.Zones(context.Background())
    assert.Greater(t, len(zones), 0)
}

```

```
    reading, err := reader.Read(context.Background(), "")
    assert.NoError(t, err)
    assert.Greater(t, reading.TemperatureC, 0.0)
    assert.Less(t, reading.TemperatureC, 120.0) // Sanity check
}
```

Run with:

```
go test -tags=hardware ./...
```

11.9 Coverage Requirements

Package	Target
pkg/*	80%
component/*	75%
examples/*	80%

Check coverage:

```
go test -coverprofile=coverage.out ./...
go tool cover -html=coverage.out
```

11.10 CI/CD Integration

GitHub Actions workflow:

```
name: Test

on: [push, pull_request]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-go@v5
        with:
          go-version: '1.22'

      - name: Unit Tests
        run: go test -race ./...

      - name: Component Tests
        run: go test -race -tags=component ./...

      - name: Coverage
        run: |
          go test -coverprofile=coverage.out ./...
          go tool cover -func=coverage.out
```

Chapter 12 explores integrating AI/ML into your tested components.

Chapter 12: AI/ML Integration

Modern robots increasingly rely on ML for perception and decision-making. Gorai treats AI as a first-class capability.

12.1 The AI Opportunity in Robotics

Edge inference vs cloud:

- Edge: Low latency, works offline, privacy preserving
- Cloud: More compute, larger models, easier updates

Gorai focuses on edge inference—models running on the robot itself.

Real-time requirements:

- Object detection: 10-30 fps for navigation
- Voice commands: <500ms response
- Gesture recognition: <100ms for natural interaction

Power constraints:

- Mobile robots have limited battery
- Inference efficiency directly impacts runtime
- NPU/TPU provide 10x+ efficiency over CPU/GPU

12.2 Hardware Accelerators

NPU (Neural Processing Unit)

RK3588 NPU (6 TOPS):

- Found in Orange Pi 5, Rock 5B, Radxa
- Optimized for INT8 inference
- Gorai uses go-rknnlite bindings

```
import "github.com/gorai/gorai/accel/rknn"

acc, err := rknn.New()
if err != nil {
    log.Fatal("NPU not available:", err)
}
defer acc.Close()

model, err := acc.Load(ctx, "yolov5s.rknn")
if err != nil {
    log.Fatal("Failed to load model:", err)
}
defer model.Close()
```

GPU Acceleration

NVIDIA CUDA (Jetson):

```
import "github.com/gorai/gorai/accel/cuda"

acc, err := cuda.New()
// Requires CUDA 12.x and cuDNN 9.x
```

OpenCL (generic):

```
import "github.com/gorai/gorai/accel/openc1"
```

```
acc, err := openc1.New()  
// Works on various GPUs
```

TPU (Tensor Processing Unit)

Google Coral:

```
import "github.com/gorai/gorai/accel/coral"
```

```
acc, err := coral.New()  
// USB or M.2 Edge TPU
```

12.3 Gorai's Acceleration Layer

The `accel` package provides a unified interface:

```
// accel/accel.go  
type Accelerator interface {  
    Name() string  
    Device() string  
    Load(ctx context.Context, modelPath string) (Model, error)  
    Close() error  
}  
  
type Model interface {  
    Name() string  
    Metadata() Metadata  
    Infer(ctx context.Context, inputs map[string]Tensor) (map[string]Tensor, error)  
    Close() error  
}  
  
type Tensor struct {  
    Shape []int  
    DataType DataType // Float32, Uint8, Int8  
    Data any  
}
```

Benefits:

- Same code runs on different hardware
- Swap accelerators via configuration
- Fallback to CPU when accelerators unavailable

12.4 Common ML Tasks

Object Detection

Detect and locate objects in images:

```
func detectObjects(img image.Image) (*vision.Detections, error) {  
    // Preprocess  
    input := preprocess(img)  
  
    // Infer  
    outputs, err := model.Infer(ctx, map[string]accel.Tensor{  
        "images": input,  
    })  
}
```

```

    if err != nil {
        return nil, err
    }

    // Postprocess (NMS, decode boxes)
    detections := postprocess(outputs["output0"])
    return detections, nil
}

func preprocess(img image.Image) accel.Tensor {
    // Resize to model input size
    resized := resize.Resize(640, 640, img, resize.Lanczos3)

    // Convert to float32, normalize
    data := make([]float32, 640*640*3)
    for y := 0; y < 640; y++ {
        for x := 0; x < 640; x++ {
            r, g, b, _ := resized.At(x, y).RGBA()
            data[(y*640+x)*3+0] = float32(r>>8) / 255.0
            data[(y*640+x)*3+1] = float32(g>>8) / 255.0
            data[(y*640+x)*3+2] = float32(b>>8) / 255.0
        }
    }

    return accel.Tensor{
        Shape:    []int{1, 3, 640, 640},
        DataType: accel.Float32,
        Data:     data,
    }
}

```

Classification

Identify what's in an image:

```

func classify(img image.Image) (*vision.Classifications, error) {
    input := preprocessForClassification(img)

    outputs, _ := model.Infer(ctx, map[string]accel.Tensor{
        "input": input,
    })

    // Output is probability distribution
    probs := outputs["output"].([]float32)

    // Find top-k
    classifications := topK(probs, 5)
    return &vision.Classifications{
        Classifications: classifications,
    }, nil
}

```

Pose Estimation

Detect human/object poses:

```

type Pose struct {
    Keypoints []Keypoint
    Score     float64
}

```

```

type Keypoint struct {
    Name      string
    X, Y      float64
    Confidence float64
}

func estimatePose(img image.Image) ([]Pose, error) {
    input := preprocess(img)
    outputs, _ := model.Infer(ctx, inputs)

    // Decode keypoints
    poses := decodePoses(outputs)
    return poses, nil
}

```

12.5 Model Deployment

ONNX as Interchange

Convert models to ONNX for portability:

```

# PyTorch to ONNX
torch.onnx.export(model, dummy_input, "model.onnx")

# TensorFlow to ONNX
python -m tf2onnx.convert --saved-model model_dir --output model.onnx

```

Then convert to target format:

```

# ONNX to RKNN (for RK3588 NPU)
python rknn_convert.py model.onnx model.rknn

# ONNX to TensorRT (for NVIDIA)
trtexec --onnx=model.onnx --saveEngine=model.trt

```

Quantization for Edge

INT8 quantization reduces model size and speeds inference:

```

# RKNN quantization
rknn.config(quantized_dtype='asymmetric_quantized-8')
rknn.load_onnx(model='model.onnx')
rknn.build(do_quantization=True, dataset='./calibration_data.txt')

```

Typical speedup: 2-4x over FP32, with <1% accuracy loss.

Model Versioning

Track models like code:

```

models/
├─ yolov5s/
│  └─ v1.0.0/
│     └─ model.rknn
│     └─ metadata.json
│     └─ classes.txt
└─ v1.1.0/
   └─ ...

```

```
└─ mobilenet/
  └─ ...
```

Configuration references version:

```
{
  "model": "yolov5s",
  "version": "v1.0.0",
  "accelerator": "rknn"
}
```

12.6 Vision Service Integration

Combine camera, accelerator, and model:

```
type VisionService struct {
    camera Camera
    accel  accel.Accelerator
    model  accel.Model
}

func (v *VisionService) DetectObjects(ctx context.Context, img image.Image) (*Detections, error) {
    // Preprocess
    input := v.preprocess(img)

    // Infer
    outputs, err := v.model.Infer(ctx, map[string]accel.Tensor{
        "images": input,
    })
    if err != nil {
        return nil, err
    }

    // Postprocess
    return v.postprocess(outputs), nil
}

func (v *VisionService) DetectFromCamera(ctx context.Context) (*Detections, error) {
    img, err := v.camera.Image(ctx)
    if err != nil {
        return nil, err
    }
    return v.DetectObjects(ctx, img)
}
```

Streaming Inference

Process frames continuously:

```
func (v *VisionService) StreamDetections(ctx context.Context) (<-chan *Detections, error) {
    stream, err := v.camera.Stream(ctx)
    if err != nil {
        return nil, err
    }

    out := make(chan *Detections, 1)

    go func() {
        defer close(out)
    }
```

```

    for img := range stream {
        detections, err := v.DetectObjects(ctx, img)
        if err != nil {
            continue
        }
        select {
        case out <- detections:
        default:
            // Drop if consumer is slow
        }
    }
}()

return out, nil
}

```

12.7 Performance Optimization

Batching

Process multiple images together:

```

// Single image: 30ms
outputs, _ := model.Infer(ctx, inputs1)

// Batch of 4: 50ms total (12.5ms per image)
batchInputs := combineTensors(inputs1, inputs2, inputs3, inputs4)
outputs, _ := model.Infer(ctx, batchInputs)

```

Async Inference

Don't block on slow models:

```

type AsyncDetector struct {
    requests chan *DetectRequest
    results  chan *DetectResult
}

func (d *AsyncDetector) DetectAsync(img image.Image) <-chan *Detections {
    result := make(chan *Detections, 1)
    d.requests <- &DetectRequest{Image: img, Result: result}
    return result
}

func (d *AsyncDetector) worker(ctx context.Context) {
    for {
        select {
        case <-ctx.Done():
            return
        case req := <-d.requests:
            det, _ := d.detect(req.Image)
            req.Result <- det
            close(req.Result)
        }
    }
}

```

Memory Management

Reuse buffers to avoid allocation:

```
type InferencePool struct {
    inputBuffers sync.Pool
    outputBuffers sync.Pool
}

func (p *InferencePool) GetInputBuffer() []float32 {
    if buf := p.inputBuffers.Get(); buf != nil {
        return buf.([]float32)
    }
    return make([]float32, 640*640*3)
}

func (p *InferencePool) PutInputBuffer(buf []float32) {
    p.inputBuffers.Put(buf)
}
```

Chapter 13 covers organizing these components into maintainable projects.

Chapter 13: Project Organization

As your robot codebase grows, organization matters. This chapter covers best practices for Gorai projects.

13.1 The Gorai Monorepo Structure

The main Gorai repository:

```
github.com/gorai/gorai/
├── api/                    # Protocol definitions
│   ├── proto/             # .proto source files
│   │   └── gorai/
│   │       ├── std/
│   │       ├── geometry/
│   │       ├── sensor/
│   │       └── vision/
│   └── gen/                # Generated Go code
├── pkg/                   # Core packages
│   ├── node/
│   ├── pub/
│   ├── sub/
│   ├── resource/
│   └── registry/
├── component/             # Component interfaces
│   ├── component.go
│   ├── motor/
│   ├── camera/
│   └── sensor/
├── service/               # Service interfaces
│   ├── service.go
│   ├── vision/
│   └── navigation/
├── driver/                # Hardware drivers
│   ├── gpio/
│   ├── i2c/
│   └── serial/
├── accel/                 # Acceleration layer
│   ├── accel.go
│   └── rknn/
├── nws/                   # Network wrappers
├── examples/              # Example applications
├── internal/              # Private packages
├── scripts/               # Utility scripts
├── specs/                 # Specifications
└── docs/                  # Documentation
```

13.2 When to Use the Monorepo

Use the core monorepo for:

- Framework development
- Changes affecting multiple packages
- Core component/service interfaces
- Shared protocol definitions

13.3 When to Create Separate Repos

Custom Components

Hardware-specific drivers:

```
github.com/myorg/gorai-drv8833/
├─ drv8833.go          # Motor driver
├─ drv8833_test.go
├─ fake/
│   └─ fake.go
├─ go.mod              # Imports gorai/gorai
└─ README.md
```

go.mod :

```
module github.com/myorg/gorai-drv8833

go 1.22

require (
    github.com/gorai/gorai v0.2.0
)
```

Robot Applications

Complete robot packages:

```
github.com/myorg/my-robot/
├─ cmd/
│   └─ myrobot/
│       └─ main.go
├─ config/
│   ├── default.json
│   └─ production.json
├─ components/          # Custom components
│   └─ arm/
├─ services/            # Custom services
│   └─ behavior/
├─ internal/            # Private packages
├─ scripts/
└─ go.mod
```

Driver Packages

Drivers with external dependencies:

```
github.com/gorai/gorai-driver-v4l2/
├─ camera.go            # V4L2 camera driver
├─ camera_test.go
├─ go.mod               # CGo dependencies isolated
└─ README.md
```

Why separate?

- CGo adds build complexity
- Vendor-specific licenses
- Platform-specific code
- Optional functionality

13.4 Import Paths and Versioning

Semantic versioning:

```
v0.1.0 # Initial development
v0.2.0 # Breaking changes (pre-1.0)
v1.0.0 # First stable release
v1.1.0 # Backward-compatible features
```

```
v1.1.1 # Bug fixes
v2.0.0 # Breaking changes (new import path)
```

Go module versioning:

```
// v0.x and v1.x
import "github.com/gorai/gorai/pkg/node"

// v2+
import "github.com/gorai/gorai/v2/pkg/node"
```

13.5 Configuration Organization

```
config/
├─ default.json      # Development defaults
├─ production.json   # Production settings
├─ robots/
│   └─ robot1.json   # Robot-specific
│   └─ robot2.json
└─ components/
    └─ motor_left.json # Component-specific
```

Layered configuration:

```
// Load base, then overlay environment-specific
config := loadConfig("default.json")
config.Merge(loadConfig(os.Getenv("GORAI_ENV") + ".json"))
```

13.6 Example: Multi-Robot Fleet

```
github.com/myorg/robot-fleet/
├─ shared/           # Common code
│   └─ components/
│   └─ services/
├─ robots/
│   └─ scout/        # Scout robot type
│       └─ cmd/
│           └─ scout/
│           └─ config/
│   └─ carrier/      # Carrier robot type
│       └─ cmd/
│           └─ carrier/
│           └─ config/
├─ fleet-manager/    # Central coordination
│   └─ cmd/
│       └─ manager/
│       └─ api/
├─ deploy/           # Deployment configs
│   └─ docker/
│   └─ kubernetes/
└─ go.mod
```

13.7 Documentation Standards

README per Package

```
# package motor
```

Motor component interface for Gorai.

```
## Installation
```

```
go get github.com/gorai/gorai/component/motor
```

```
## Usage
```

```
motor := fake.NewMotor()
motor.SetPower(ctx, 0.5)
```

```
## Configuration
```

Field	Type	Description
<code>max_power</code>	<code>float64</code>	Maximum power (0-1)

GoDoc Comments

```
// Motor represents a controllable motor.
//
// Motors can be controlled via power (open-loop), velocity (closed-loop),
// or position (closed-loop). Use Properties to discover capabilities.
//
// Example:
//
//     motor, _ := drv8833.New("left", cfg, pins)
//     motor.SetPower(ctx, 0.5) // 50% forward
//     motor.Stop(ctx)
//
type Motor interface {
    // SetPower sets the motor power from -1.0 (full reverse) to 1.0 (full forward).
    //
    // Power values are clamped to configured max_power.
    // Returns an error if the motor is in a fault state.
    SetPower(ctx context.Context, power float64) error

    // ...
}
```

Example Files

```
// motor_example_test.go
package motor_test

func ExampleMotor_SetPower() {
    motor := fake.NewMotor()

    motor.SetPower(context.Background(), 0.5)
    // Motor runs at 50% power

    motor.Stop(context.Background())
    // Motor stops
}
```

```
// Output:  
}
```

Run examples as tests:

```
go test -v -run Example
```

Chapter 14 explores using AI tools to accelerate Gorai development.

Chapter 14: AI-Assisted Development

Gorai is designed with AI-assisted development in mind. Clear interfaces, consistent patterns, and comprehensive specifications make AI tools effective collaborators.

14.1 The AI Development Philosophy

From CLAUDE.md: "use AI assisted coding wherever possible"

This isn't about replacing developers—it's about amplifying them:

- **Speed:** Generate boilerplate, tests, documentation faster
- **Consistency:** Follow established patterns automatically
- **Exploration:** Quickly prototype ideas
- **Learning:** Understand unfamiliar code through explanation

14.2 Effective AI Prompting for Robotics

Component Generation

```
Create a Gorai motor component for the L298N dual H-bridge that implements
the motor.Motor interface. Include:
- SetPower with power clamping to configured max
- Direction control via IN1/IN2 pins
- PWM via ENA pin
- Thread-safe state management
- A fake implementation for testing
```

```
Follow the patterns in component/motor/ and examples/hello-sensor/.
```

Why this works:

- Specifies the hardware target
- Lists required features
- References existing patterns
- Requests the fake (often forgotten)

Test Generation

```
Write table-driven tests for the DRV8833 motor's SetPower method covering:
- Normal cases: 0, 0.5, 1.0, -0.5, -1.0
- Edge cases: values beyond range (clamp), exactly at limits
- Error conditions: motor in fault state, closed motor
Follow Gorai's testing patterns from specs/testing-approach.md
```

Protocol Buffer Design

```
Design a protobuf message for an ultrasonic range sensor that includes:
- Header with timestamp and frame_id
- Range measurement in meters
- Field of view in radians
- Min and max range capabilities
- Radiation type (ultrasound)
```

```
Follow conventions in api/proto/gorai/sensor/sensor.proto
```

14.3 Code Review with AI

Use AI to review before committing:

Review this motor driver implementation for:

- Interface compliance with motor.Motor
- Thread safety (all state access protected)
- Error handling (hardware failures, invalid inputs)
- Resource cleanup (Close properly releases resources)
- Testability (dependencies injectable)

Common issues AI catches:

- Missing mutex protection
- Unclosed resources
- Hardcoded values that should be config
- Missing context cancellation checks

14.4 Documentation Generation

GoDoc from Code

Generate GoDoc comments for all exported symbols in this file.
Follow the style in pkg/node/node.go – brief first sentence,
then details, then example if helpful.

README Generation

Generate a README.md for this motor driver package including:

- Brief description
- Installation instructions
- Usage example
- Configuration reference table
- Link to related specs

Keep it concise – under 100 lines.

14.5 Debugging Assistance

Error Analysis

This motor command is failing with "context deadline exceeded".
The motor is configured with these pins: IN1=17, IN2=18, PWM=12.
The GPIO driver is gpio.RPiDriver.

Help me debug:

1. What could cause this timeout?
2. How can I add logging to narrow it down?
3. What should I check in hardware?

Log Interpretation

These are the last 20 log lines before the crash.

The robot was navigating to waypoint 3.

Help me understand:

1. What was the sequence of events?
2. Where did things go wrong?
3. What additional logging would help?

NATS Message Inspection

```
I'm seeing these messages on "gorai.motors.left.command" but
the motor isn't responding. The motor is subscribed to this topic.
Help me debug the message flow.
```

14.6 Specification to Implementation

Gorai's specs are designed for AI consumption:

```
Implement the temperature sensor described in specs/hello-sensor-design.md.
Generate:
1. The reader interface and Linux implementation
2. The sensor component implementing resource.Sensor
3. Unit tests for both
4. A fake reader for testing

Use existing patterns from examples/hello-sensor/ as reference.
```

Workflow:

1. Write/review specification (human)
2. Generate initial implementation (AI)
3. Review and refine (human)
4. Generate tests (AI)
5. Run tests, fix issues (collaborative)

14.7 Limitations and Pitfalls

Hardware-Specific Knowledge Gaps

AI doesn't know:

- Your specific wiring
- Timing requirements of your hardware
- Environmental factors (EMI, temperature)

Always verify:

- Pin assignments match physical connections
- Timing meets hardware specs
- Edge cases tested on real hardware

Testing on Real Hardware Still Required

AI-generated code may pass unit tests but fail on hardware:

- GPIO timing issues
- I2C address conflicts
- Power supply limitations

Security Review Importance

AI may generate insecure patterns:

- Hardcoded credentials
- Unsafe command execution
- Unvalidated inputs

Review all AI-generated code for security implications.

Safety-Critical Code

Never use AI-generated code unreviewed for:

- Emergency stop logic
- Motor power limiting

- Collision detection
- Human safety interlocks

These require human verification and testing.

14.8 Workflow Integration

Editor Integration

VS Code with Copilot/Cody:

- Inline completions as you type
- Chat for questions and generation
- Reference Gorai patterns in prompts

CLI Tools

Use AI from command line:

```
# Generate component
ai "Create a Gorai component for BMP280 temperature/pressure sensor"

# Explain code
ai "Explain what this NATS subscription does" < code.go

# Debug
ai "Why might this test be flaky?" < test_output.txt
```

CI/CD Assistance

AI can help with:

- Debugging failing CI
- Optimizing build times
- Generating release notes

These CI tests passed locally but fail on GitHub Actions.
The difference is: local is macOS, CI is ubuntu-latest.
Help me understand platform-specific issues.

With all the technical foundations in place, Chapter 15 concludes with the Gorai vision and next steps.

Chapter 15: Conclusion

We've covered a lot of ground. Let's step back and see the whole picture.

15.1 What We've Covered

Foundations (Chapters 1-3):

- Why Gorai exists: filling the gap between heavy frameworks and from-scratch development
- The mental model: nodes, resources, distributed architecture
- NATS as the communication backbone: topics, services, actions, QoS

Components (Chapters 4-6):

- Sensors: the `Readings()` interface, built-in types, fake implementations
- Actuators: motors, servos, grippers, safety-first design
- Vision: cameras, image flow, depth sensing

Services (Chapter 7):

- The difference between components and services
- Vision, navigation, SLAM, motion planning
- Building custom services

Development (Chapters 8-11):

- Environment setup: Go, NATS, Protocol Buffers
- The hello-sensor deep dive: a complete working example
- Building custom components from scratch
- Testing at every level: unit, component, integration, hardware

Advanced Topics (Chapters 12-14):

- AI/ML integration: NPU, GPU, TPU acceleration
- Project organization: monorepo, satellites, versioning
- AI-assisted development: prompting, review, debugging

15.2 The Gorai Vision

Returning to Chapter 1's question: What if we designed a robotics framework for today?

Go-first: The language provides simplicity without sacrificing performance. Concurrency is natural. Deployment is trivial. The same language works from data center to microcontroller.

NATS-native: Cloud-proven messaging adapted for robotics. Simpler than DDS, more capable than MQTT. Persistence when you need it, speed when you don't.

AI-optimized: First-class support for edge inference. Unified acceleration interface. Models as components, not afterthoughts.

Modular by default: Loose coupling through messages. Hot-swappable components. Distributed from day one.

Low barrier to entry: Minutes to first sensor reading. Hours to custom component. Days to complete robot system.

Fun: The joy of building robots, not fighting tools.

15.3 Next Steps for Readers

Immediate

1. **Run hello-sensor:** If you haven't already, get it working on your machine
2. **Modify it:** Change the interval, add a reading, break it and fix it
3. **Subscribe with NATS CLI:** Watch messages flow in real-time

Short Term

4. **Build a custom sensor:** Pick hardware you have—a temperature sensor, a button, an LED

5. **Write tests:** Unit tests, fake implementation, component tests
6. **Connect hardware:** Deploy to a Raspberry Pi or Orange Pi

Medium Term

7. **Build a complete system:** Multiple components working together
8. **Add a service:** Vision, behavior state machine, or simple navigation
9. **Contribute:** Fix a bug, improve documentation, add an example

15.4 Roadmap Highlights

Gorai is actively developing:

Near term:

- More component drivers (common motors, sensors)
- Improved ML model support
- Better tooling for debugging and monitoring

Medium term:

- TinyGo serial gateway improvements
- Navigation and SLAM services
- Simulation environment

Longer term:

- Fleet management
- Cloud telemetry integration (optional)
- Certification support for production deployments

15.5 Getting Help

GitHub Issues: <https://github.com/gorai/gorai/issues>

- Bug reports
- Feature requests
- Questions

Documentation:

- specs/ directory for design documents
- docs/ for analysis and decisions
- Code comments for implementation details

Community:

- Discussions on GitHub
- Share your builds
- Contribute improvements

15.6 Contributing to Gorai

We welcome contributions:

Code contributions:

1. Fork the repository
2. Create a feature branch
3. Write code with tests
4. Submit a pull request

Documentation:

- Fix typos
- Clarify confusing sections
- Add examples

Specifications:

- Propose new components
- Design new services
- Review existing specs

Review process:

- All changes reviewed before merge
- CI must pass
- Documentation updates encouraged

15.7 Final Thoughts

Robotics should be joyful. The frustration of complex build systems, cryptic errors, and heavyweight frameworks steals that joy.

Gorai aims to restore it. Write Go code. Run it on your robot. See it work. Iterate quickly. Focus on the interesting problems—not the infrastructure.

The framework is young. There's much to build. But the foundations are solid: clean interfaces, consistent patterns, comprehensive testing. You can build on this.

Whether you're exploring robotics for the first time or simplifying a complex existing system, Gorai offers a path: **Go + NATS + AI = modern robotics.**

Welcome aboard. Let's build something amazing.

Gorai: Building Modern Robots with Go and NATS

Appendices

Appendix A: Command Reference

Gorai Scripts

Script	Purpose
scripts/start.sh	Start NATS and development services
scripts/stop.sh	Stop all services
scripts/hello.sh	Run hello-sensor example

NATS CLI Commands

Command	Purpose
nats sub ">"	Subscribe to all messages
nats sub "gorai.>"	Subscribe to Gorai messages
nats pub TOPIC DATA	Publish message
nats request TOPIC DATA	Request/reply
nats server info	Server information
nats stream list	List JetStream streams
nats stream view NAME	View stream messages
nats consumer list STREAM	List consumers

Go Commands

Command	Purpose
go build ./...	Build all packages
go test ./...	Run unit tests
go test -tags=component ./...	Run component tests
go test -race ./...	Test with race detector
go test -cover ./...	Test with coverage

Makefile Targets

Target	Purpose
make build	Build all binaries
make test	Run unit tests
make test-quick	Unit + component tests
make test-all	All test levels
make proto	Generate proto code
make lint	Run linters

Appendix B: Protocol Buffer Reference

Standard Messages (gorai/std)

```
message Header {
    Timestamp stamp = 1;
    string frame_id = 2;
    uint32 seq = 3;
}

message Timestamp {
    int64 seconds = 1;
    int32 nanos = 2;
}

message Duration {
    int64 seconds = 1;
    int32 nanos = 2;
}

message DiagnosticStatus {
    uint32 level = 1;      // OK=0, WARN=1, ERROR=2, STALE=3
    string name = 2;
    string message = 3;
    string hardware_id = 4;
}
```

Geometry Messages (gorai/geometry)

```
message Vector3 {
    double x = 1;
    double y = 2;
    double z = 3;
}

message Point {
    double x = 1;
    double y = 2;
    double z = 3;
}

message Quaternion {
    double x = 1;
    double y = 2;
    double z = 3;
    double w = 4;
}

message Pose {
    Point position = 1;
    Quaternion orientation = 2;
}

message Twist {
    Vector3 linear = 1;
    Vector3 angular = 2;
}

message Transform {
```

```
Vector3 translation = 1;
Quaternion rotation = 2;
}
```

Sensor Messages (gorai/sensor)

```
message Imu {
    std.Header header = 1;
    geometry.Quaternion orientation = 2;
    repeated double orientation_covariance = 3;
    geometry.Vector3 angular_velocity = 4;
    repeated double angular_velocity_covariance = 5;
    geometry.Vector3 linear_acceleration = 6;
    repeated double linear_acceleration_covariance = 7;
}

message Image {
    std.Header header = 1;
    uint32 height = 2;
    uint32 width = 3;
    string encoding = 4;
    uint32 step = 5;
    bytes data = 6;
}

message LaserScan {
    std.Header header = 1;
    float angle_min = 2;
    float angle_max = 3;
    float angle_increment = 4;
    float time_increment = 5;
    float scan_time = 6;
    float range_min = 7;
    float range_max = 8;
    repeated float ranges = 9;
    repeated float intensities = 10;
}

message NavSatFix {
    std.Header header = 1;
    int32 status = 2;
    uint32 service = 3;
    double latitude = 4;
    double longitude = 5;
    double altitude = 6;
    repeated double position_covariance = 7;
    uint32 position_covariance_type = 8;
}
```

Appendix C: Hardware Compatibility

Supported SBCs

Board	CPU	RAM	NPU	GPIO	Notes
Raspberry Pi 5	Cortex-A76	8GB	No	Yes	Best starter
Orange Pi 5	RK3588S	8-16GB	6 TOPS	Yes	Best for AI
Jetson Orin Nano	Cortex-A78	8GB	GPU	Yes	CUDA support

BeagleBone AI-64	TDA4VM	4GB	8 TOPS	Yes	Real-time PRUs
Rock 5B	RK3588	8-16GB	6 TOPS	Yes	PCIe support

Supported Microcontrollers (TinyGo)

Board	CPU	RAM	Flash	Notes
RP2040 (Pico)	Cortex-M0+	264KB	2MB	Dual core
ESP32-C3	RISC-V	400KB	4MB	WiFi/BLE
STM32F4	Cortex-M4	128KB+	512KB+	Industrial

Common Sensors

Sensor	Interface	Gorai Support
MPU6050	I2C	Example available
BME280	I2C/SPI	Example available
GPS (NMEA)	UART	Example available
HC-SR04	GPIO	Example available
Encoders	GPIO	Included

Common Actuators

Actuator	Interface	Gorai Support
DC Motors	PWM+GPIO	Included
Steppers	GPIO	Example available
RC Servos	PWM	Included
DRV8833	PWM+GPIO	Example available

Appendix D: Troubleshooting

NATS Connectivity

Problem: Can't connect to NATS server

```
failed to connect to NATS: nats: no servers available for connection
```

Solutions:

- 1. Check NATS server is running: `nats server info`
- 2. Check URL: default is `nats://localhost:4222`
- 3. Check firewall allows port 4222
- 4. Verify network connectivity

Problem: JetStream not available

```
JetStream not available
```

Solutions:

- 1. Start NATS with `-js` flag
- 2. Check JetStream is enabled in config

Hardware Access Issues

Problem: Permission denied for GPIO

```
open /sys/class/gpio/export: permission denied
```

Solutions:

- 1. Add user to gpio group: `sudo usermod -aG gpio $USER`
- 2. Logout and login
- 3. Or run with sudo (not recommended for production)

Problem: I2C device not found

```
no I2C device at address 0x68
```

Solutions:

- 1. Check wiring
- 2. Verify I2C enabled: `sudo raspi-config`
- 3. Scan bus: `i2cdetect -y 1`
- 4. Check address in datasheet

Build Problems

Problem: Module not found

```
cannot find module providing package github.com/gorai/gorai/...
```

Solutions:

- 1. Run `go mod download`
- 2. Check Go version ≥ 1.21
- 3. Verify network access to github.com

Problem: CGo errors

```
cgo: C compiler "gcc" not found
```

Solutions:

- 1. Install build-essential: `sudo apt install build-essential`
- 2. Or use pure Go alternatives

Appendix E: Glossary

Term	Definition
Actuator	Component that performs physical actions (motors, servos)
Component	Hardware abstraction in Gorai
Fake	Test implementation that simulates real hardware
JetStream	NATS persistence layer
Node	Gorai process managing resources and NATS connection
NPU	Neural Processing Unit for ML inference
NWC	Network Wrapper Client - consumes remote resources
NWS	Network Wrapper Server - exposes resources
QoS	Quality of Service for message delivery

Resource	Base interface for components and services
Sensor	Component that provides readings
Service	Software capability (vision, navigation)
TinyGo	Go compiler for microcontrollers
Topic	NATS subject for pub/sub messaging

End of Gorai Book